# Lisan al-Gaib

Architecture & Implementation Guide

v1.0 — January 2026

A Self-Evolving Multi-Agent Orchestration System

# Part I: Executive Preamble

## Lisan al-Gaib: A Self-Evolving Multi-Agent Orchestration System

---

## 1. Vision Statement

Lisan al-Gaib is a hybrid multi-agent orchestration architecture that combines Claude Code's tiered intelligence (Haiku/Sonnet/Opus) with Gemini Flash's million-token context window, unified by deterministic bash-hook enforcement and file-based state management. The system is designed to evolve: it observes human orchestration decisions, discovers behavioral patterns, and progressively automates routine workflows while maintaining human oversight for critical decisions.

**The core thesis:** Most AI agent frameworks optimize for developer convenience at the cost of transparency, cost control, and auditability. Lisan al-Gaib inverts this priority—transparency and determinism come first, with sophistication emerging from composition rather than abstraction.

**What makes this different:**

1. **Code-enforced routing** — Bash scripts calculate complexity scores and enforce tier selection. The LLM cannot route itself to expensive tiers; the code decides. This eliminates prompt injection risks in routing and provides deterministic cost control.

2. **File-based memory** — All state is human-readable YAML and JSON files. No database dependencies, no opaque vector stores. grep and find are valid debugging tools. Git tracks all changes.

3. **Hybrid context architecture** — Claude handles orchestration and generation within its native context limits; Gemini handles large-context operations (codebase analysis, document synthesis) via CLI pipes. Each model operates in its optimal regime.

4. **Progressive autonomy** — The system captures human decisions, identifies patterns, and proposes automation. Humans approve. Over time, routine decisions automate while novel situations maintain human oversight.

This is not a framework. It is an architecture pattern with concrete implementation—a set of interlocking components designed to grow more capable through use.

---

## 2. Architecture at a Glance

```
┌──────────────
```

```
                    │
        ┌───────────┼───────────┐
        │           │           │
        ▼           ▼           ▼
┌─────────────┐  ┌─────────────┐  ┌─────────────┐
│  BASH HOOKS │  │ STATE FILES │  │   MEMORY    │
│             │  │             │  │             │
│ PreToolUse  │  │ scout_metrics│  │ decisions/  │
│ PostToolUse │  │ complexity  │  │ sharp-edges/│
│ SubagentStop│  │ routing_log │  │ facts/      │
│ SessionEnd  │  │ handoff.json│  │ preferences/│
│             │  │             │  │             │
│ ENFORCEMENT │  │ .claude/tmp/ │  │ .claude/    │
│   LAYER     │  │             │  │  memory/    │
└─────────────┘  └─────────────┘  └─────────────┘
```

### Component Summary

| Component | Role | Implementation | Cost Profile |
|---|---|---|---|
| **Claude Haiku** | Scout, quick QA, simple tasks | Native subagent | $0.25/1M tokens |
| **Claude Sonnet** | Architecture, planning, standard work | Primary tier | $3/1M tokens |
| **Claude Opus** | Complex reasoning, critical decisions | Escalation tier | $15/1M tokens |
| **Gemini Flash** | Large context (>100K), codebase analysis | CLI pipe integration | ~$0.075/1M tokens |
| **Bash Hooks** | Routing enforcement, validation, logging | Shell scripts in `.claude/hooks/` | Zero marginal cost |
| **State Files** | Inter-agent communication, checkpoints | JSON/YAML in `.claude/tmp/` | Zero marginal cost |
| **Memory System** | Persistent learning, decision history | Markdown/YAML in `.claude/memory/` | Zero marginal cost |

### Data Flow Summary

1. **User request** enters the orchestrator
2. **Haiku scout** (optional) performs reconnaissance on large codebases
3. **Complexity calculator** (bash) scores the task: `(tokens/10000) + (files/5) + (modules*2)`
4. **PreToolUse hook** enforces tier ceiling based on score
5. **Selected agent** executes with appropriate context
6. **PostToolUse hook** logs outcomes, captures decisions
7. **Memory archivist** (periodic) processes decisions into persistent memory
8. **Weekly review** synthesizes learnings, proposes system improvements

---

## 3. Competitive Differentiation

### vs. LangChain/LangGraph

| Dimension | LangChain/LangGraph | Lisan al-Gaib |
|---|---|---|
| **Routing** | LLM-based or manual | Code-enforced (bash arithmetic) |
| **State** | PostgreSQL + Redis required | File-based (YAML/JSON) |
| **Debugging** | Graph visualization, traces | grep, find, git diff |
| **Dependencies** | Heavy (framework + infrastructure) | Minimal (bash, jq, bc) |
| **Latency overhead** | ~14ms per graph invocation | <1ms (file I/O) |
| **Token overhead** | 15-50% increase documented | Controlled (no framework prompts) |
| **Tiered routing** | Not native, must implement | First-class feature |
| **Lock-in risk** | High (API instability history) | Zero (standard tools) |

**Key finding from framework analysis:** LangChain consumed 2.7x expected costs in documented cases due to hidden multi-call patterns. Teams that migrated from LangChain to bespoke solutions reported being “happier and more productive.”

### vs. Pure API Integration

| Dimension | Direct API Calls | Lisan al-Gaib |
|---|---|---|
| **Cost control** | Manual, per-call | Automatic tier routing |
| **Context management** | Developer responsibility | Hybrid architecture handles |
| **Memory/learning** | None | File-based RAG + evolution |
| **Human oversight** | Ad-hoc | Structured (weekly review, approvals) |
| **Scalability** | Linear cost growth | Optimized routing reduces costs |

### Cost Efficiency Thesis

Research on intelligent routing (RouteLLM, FrugalGPT) demonstrates that **85% cost reduction** is achievable while maintaining 95% of top-tier performance through appropriate model selection. Lisan al-Gaib implements this via:

1. **Scout-first protocol** — Haiku assesses complexity before expensive operations
2. **Complexity-based routing** — Deterministic formula prevents over-escalation
3. **Cascade fallback** — Start cheap, escalate only on verified need
4. **External offload** — Gemini handles context-heavy work at lower cost

Target session cost: **$3-5** for typical development sessions touching 5-20 files.

---

## 4. Intellectual Property Summary

Lisan al-Gaib introduces **four novel architectural patterns** that represent significant innovations in multi-agent system design. These should be considered for IP protection.

### Innovation 1: Emergent Schema Discovery

**What it is:** Rather than pre-defining schemas for decision capture, the system accumulates raw behavioral observations. When sufficient data exists (200+ observations, 3+ detectable clusters), a specialized subagent analyzes patterns and *proposes* a schema. Humans approve before activation.

**Why it's novel:** Existing RLHF approaches require pre-defined feedback schemas. This inverts the process—the schema emerges from observed behavior, similar to unsupervised learning applied to system architecture evolution.

**Competitive advantage:** Systems using this pattern adapt to user workflows rather than forcing users into predefined categories.

### Innovation 2: Gap Analysis → Subagent Spawning

**What it is:** Weekly architecture review identifies capability gaps (e.g., "document synthesis failed 4 times due to context boundaries"). For gaps that cannot be resolved by configuration changes, the system generates subagent templates—complete agent definitions ready for deployment.

**Why it's novel:** Multi-agent systems typically have fixed agent populations. This creates a *generative architecture* that grows capabilities based on observed deficiencies.

**Competitive advantage:** The system becomes more capable over time without manual architecture redesign.

### Innovation 3: Apprenticeship Learning Model ("Agent-in-the-Loop" Inversion)

**What it is:** Traditional human-in-the-loop has agents propose, humans approve. This system inverts the model: humans act, the system observes and learns. Over time, a "QC subagent" emerges that replicates human decision-making. The human progressively steps back as the agent demonstrates competence in specific decision categories.

**Why it's novel:** This is apprenticeship learning applied to AI orchestration. The agent learns to *become* the human orchestrator for routine decisions.

**Competitive advantage:** Achieves automation without sacrificing the nuanced decision-making that humans provide during the learning period.

### Innovation 4: Code-Enforced Deterministic Routing

**What it is:** Routing decisions are made by bash scripts using arithmetic formulas, not by LLMs interpreting prompts. The LLM cannot convince itself to use a more expensive tier—the code enforces the ceiling.

**Why it's novel:** Most agent frameworks rely on LLM-based routing, which is susceptible to prompt injection, inconsistent interpretation, and cost overruns. Code-enforced routing is deterministic, auditable, and manipulation-resistant.

**Competitive advantage:** Predictable costs, auditable decisions, zero routing latency, security against prompt injection attacks on routing.

---

## 5. Current Maturity & Capabilities

### What Works Today (v2.1)

| Capability | Status | Notes |
|---|---|---|
| Tiered agent routing | ✅ Production | Haiku/Sonnet/Opus + Gemini |
| Complexity calculation | ✅ Production | Bash formula implemented |
| PreToolUse enforcement | ✅ Production | Tier ceiling enforced |
| Scout-first protocol | ✅ Production | Haiku reconnaissance |
| Gemini CLI integration | ✅ Production | Large context offload |
| File-based state | ✅ Production | JSON/YAML in .claude/tmp/ |
| Memory directory structure | ✅ Production | decisions/, sharp-edges/, etc. |
| Cross-platform install | ✅ Production | Bash + PowerShell installers |
| Explore workflow (7 phases) | ✅ Production | Full implementation |

### Partial Implementation

| Capability | Status | Gap |
|---|---|---|
| PostToolUse logging | 🟡 Partial | Needs structured decision capture |
| SubagentStop validation | 🟡 Partial | Hook exists, validation logic needed |
| Memory retrieval | 🟡 Partial | grep-based, needs BM25 upgrade |
| Weekly review | 🟡 Partial | Manual process, needs automation |

### Not Yet Implemented

| Capability | Status | Roadmap Position |
|---|---|---|
| Observation accumulation | ❌ Planned | Phase 4 (Months 4-5) |
| Emergent schema discovery | ❌ Planned | Phase 4 (Months 4-5) |
| Gap analysis automation | ❌ Planned | Phase 5 (Months 6-7) |
| Subagent spawning | ❌ Planned | Phase 5 (Months 6-7) |
| Autonomy level progression | ❌ Planned | Phase 6 (Months 8-10) |
| Self-improving loop | ❌ Planned | Phase 7 (Months 11-12) |

### Production Readiness Assessment

**Current state:** Solid foundation for cost-controlled multi-agent development work. The routing and enforcement layers are production-ready. The evolution mechanisms (the novel IP) are designed but not implemented.

**Risk level:** Low for current capabilities; the architecture is conservative and failure modes are well-understood.

**Scaling readiness:** Tested with sessions touching 5-20 files. Architecture supports larger scale; Gemini offload handles context growth.

---

## 6. Strategic Positioning

### Target Use Cases

1. **Cost-sensitive AI-assisted development** — Teams with budget constraints who need intelligent routing to control costs while maintaining quality for complex tasks.

2. **Auditable AI workflows** — Organizations requiring explainable AI decisions with full audit trails (regulated industries, security-sensitive environments).

3. **Long-running development relationships** — Solo developers or small teams where the AI assistant should learn and improve over months/years of collaboration.

4. **Hybrid model architectures** — Teams wanting to leverage multiple AI providers (Anthropic + Google) without framework lock-in.

### Development Philosophy

1. **Transparency over abstraction** — Every decision is traceable to human-readable files.

2. **Composition over inheritance** — Components are independent and composable; no deep inheritance hierarchies.

3. **Conservative automation** — Automate only what has been proven safe; maintain human oversight for novel situations.

4. **Local-first** — No mandatory cloud dependencies; everything runs on developer machines.

5. **Incremental evolution** — The system grows capabilities through observation and adaptation, not wholesale replacement.

### 12-Month Vision

By end of year, Lisan al-Gaib will be a **self-evolving development assistant** that:

- Recognizes your decision patterns and proposes automation for routine choices
- Spawns specialized subagents when capability gaps are identified
- Maintains transparent human oversight for all critical decisions
- Operates at predictable, optimized costs through intelligent routing
- Accumulates organizational knowledge in searchable, version-controlled memory

The human orchestrator progressively shifts from operator to supervisor to consultant—not because the AI "takes over," but because the AI has *learned* the human's approach and can be trusted with routine execution.

---

## Summary

Lisan al-Gaib represents a fundamentally different approach to AI agent orchestration:

- **Not a framework** — An architecture pattern with concrete implementation
- **Not opaque** — Every decision traceable in human-readable files
- **Not static** — Designed to evolve through observation and adaptation
- **Not expensive** — Intelligent routing controls costs without sacrificing capability

The four novel innovations—emergent schema discovery, generative subagent spawning, apprenticeship learning, and code-enforced routing—position this architecture as a unique entry in the multi-agent systems space, with potential for significant intellectual property value.

The following sections provide the technical depth required for implementation.

---

*Continue to Part II: Technical Architecture* → # Part II: Technical Architecture

## Comprehensive System Reference

---

## 1. System Topology

### 1.1 Complete Component Map

```
┌──────────────────────────────────────────────────────────────────────────────
│                              LISAN AL-GAIB SYSTEM TOPOLOGY
│
```

```
└──────────────
```

```
protocols/      │
│ PostToolUse.sh  │  │ .json           │  │ sharp-edges/    │  │
handover.md     │
│ SubagentStop.sh │  │                 │  │ facts/          │  │
│
│ SessionEnd.sh   │  │ complexity_     │  │ preferences/    │  │
config.yaml     │
│                 │  │ score           │  │ observations/   │  │
│
│ validate-       │  │                 │  │                 │  │
CLI interface   │
│ routing.sh      │  │ recommended_    │  │ YAML frontmatter│  │
via pipes       │
│                 │  │ tier            │  │ + markdown body │  │
│
│ calculate-      │  │                 │  │                 │  │
│
│ complexity.sh   │  │ handoff.json    │  │ BM25 indexable  │  │
│
│                 │  │                 │  │                 │  │
│
│ attention-      │  │ routing_log     │  │ git versioned   │  │
│
│ gate.sh         │  │ .jsonl          │  │                 │  │
│
│                 │  │                 │  │                 │  │
│
└─────────────────┘  └─────────────────┘  └─────────────────┘
└─────────────────┘
```

### 1.2 Component Responsibilities

| Component | Primary Responsibility | Secondary Responsibilities |
|---|---|---|
| **Claude Code Orchestrator** | Conversation management, task decomposition | Tool invocation, context management |
| **Hook Layer** | Routing enforcement, validation | Logging, metrics capture, blocking |
| **State Layer** | Inter-agent communication | Checkpointing, handoff persistence |
| **Memory Layer** | Long-term learning storage | Pattern retrieval, decision history |
| **External Layer** | Large context operations | Codebase analysis, document synthesis |

### 1.3 File System Layout

```
~/.claude/
├── hooks/
│   ├── PreToolUse.sh           # Routing enforcement (runs before
every tool)
│   ├── PostToolUse.sh          # Outcome logging (runs after every
tool)
│   ├── SubagentStop.sh         # Subagent completion validation
│   └── SessionEnd.sh           # Session cleanup, memory archival
trigger
│
├── scripts/
│   ├── calculate-complexity.sh # Complexity scoring formula
│   ├── validate-routing.sh     # Tier permission checker
│   ├── attention-gate.sh       # Focus management
│   └── query-memory.sh         # Memory retrieval interface
│
├── agents/
│   ├── haiku-scout/
│   │   ├── agent.md            # Agent definition
│   │   └── agent.yaml          # Configuration
│   ├── architect/
│   ├── memory-archivist/
│   └── [spawned-agents]/       # Future dynamically created agents
│
├── skills/
│   └── explore/
│       └── SKILL.md            # Explore workflow definition
│
├── tmp/                        # Ephemeral state (cleared per
session)
│   ├── scout_metrics.json
│   ├── complexity_score
│   ├── recommended_tier
│   └── handoff.json
│
├── memory/                     # Persistent learning (git
versioned)
│   ├── decisions/              # Approved architectural decisions
│   ├── sharp-edges/            # Known pitfalls and workarounds
│   ├── facts/                  # Verified project facts
│   ├── preferences/            # User preferences and patterns
│   └── observations/           # Raw behavioral observations
(future)
│
├── schemas/                    # Schema definitions (future)
```

```
│   └── [version]/
│
├── settings.json               # Global configuration
├── routing-schema.json         # Routing rules and thresholds
└── agents-index.json           # Agent registry with status

~/.gemini-slave/
├── protocols/
│   ├── handover-protocol.md    # Claude→Gemini handoff format
│   ├── codebase-analysis.md    # Large codebase handling
│   └── document-synthesis.md   # Multi-document summarization
└── config.yaml                 # Gemini CLI configuration
```

---

# 2. Tier Structure

## 2.1 Tier Definitions

**Tier 0: Haiku (Scout/Quick Operations)**

**Purpose:** Fast, cheap reconnaissance and simple tasks **Cost:** $0.25/1M input, $1.25/1M output **Context:** 200K tokens **Latency:** Fastest

**Use cases:** - Initial codebase reconnaissance (scout protocol) - Simple file lookups - Quick validation checks - Formatting and linting decisions - Yes/no classification tasks

**Routing trigger:** Complexity score < 2

**Tier 1: Sonnet (Standard Operations)**

**Purpose:** Primary workhorse for most development tasks **Cost:** $3/1M input, $15/1M output **Context:** 200K tokens **Latency:** Fast

**Use cases:** - Architecture planning - Code generation and modification - Code review - Documentation writing - Multi-file refactoring (up to 10 files) - Standard problem-solving

**Routing trigger:** Complexity score 2-10

**Tier 2: Opus (Complex Operations)**

**Purpose:** Most capable reasoning for difficult problems **Cost:** $15/1M input, $75/1M output **Context:** 200K tokens **Latency:** Slower

**Use cases:** - Complex architectural decisions - Edge case reasoning - Cross-cutting refactors - Critical security reviews - Novel problem-solving

**Routing trigger:** Complexity score > 10, or explicit `--force-tier opus`

**External: Gemini Flash (Large Context Operations)**

**Purpose:** Operations requiring context beyond Claude's window **Cost:** ~$0.075/1M input (free tier available) **Context:** 1M tokens **Latency:** Variable

**Use cases:** - Full codebase analysis - Large document synthesis - Multi-file context assembly - Repository-wide search and analysis

**Routing trigger:** Estimated tokens > 50K, or explicit Gemini protocol invocation

## 2.2 Tier Selection Matrix

| Task Type | Files | Complexity | Recommended Tier |
|---|---|---|---|
| Simple lookup | 1 | Low | Haiku |
| Bug fix | 1-3 | Low-Medium | Sonnet |
| Feature implementation | 3-10 | Medium | Sonnet |
| Architecture refactor | 5-15 | High | Opus |
| Codebase analysis | 20+ | Context-bound | Gemini |
| Security audit | Variable | Critical | Opus |
| Documentation | Variable | Low-Medium | Sonnet |

---

# 3. Request Lifecycle

## 3.1 Six-Phase Request Processing

```
┌──────────────────────────────────────────────────────────────────────────────
│                          COMPLETE REQUEST LIFECYCLE
│
└──────────────────────────────────────────────────────────────────────────────
```

```
PHASE 1: INTENT CLASSIFICATION
┌──────────────
```

```
│  │   Example: 45000 tokens, 15 files, 3 modules
│   │
│  │   SCORE = 4.5 + 3.0 + 6.0 = 13.5
│   │
│  │
│   │
│
└──────────────────────────────────────────────────────────────────────────
   │
│
│
│  Output files:
│
│  • .claude/tmp/complexity_score  → "13.50"
│
│  • .claude/tmp/recommended_tier  → "opus"
│
│
│
│  Threshold mapping:
│
│  • Score < 2     → haiku
│
│  • Score 2-10    → sonnet
│
│  • Score > 10    → opus
│
│  • Tokens > 50K  → FORCE gemini (regardless of score)
│
│
│
└──────────────────────────────────────────────────────────────────────────

                                        │
                                        ▼
PHASE 4: ROUTING ENFORCEMENT
┌──────────────────────────────────────────────────────────────────────────
│
│
│  PreToolUse.sh hook executes BEFORE any tool invocation:
│
│
│
│
┌──────────────────────────────────────────────────────────────────────────
   │
│  │
│   │
│  │   1. Check for --force-tier escape hatch
│   │
│  │      └─ If present and valid, PERMIT regardless of score
│   │
│  │
│   │
│  │   2. Check scout_metrics freshness (< 5 minutes)
│   │
│  │      └─ If stale, trigger re-scout or use conservative estimate
│   │
│  │
│   │
│  │   3. Read recommended_tier from state file
│   │
│  │
│   │
│  │   4. Compare requested operation tier to ceiling
│   │
│  │      └─ If operation tier ≤ ceiling: PERMIT (exit 0)
│   │
│  │      └─ If operation tier > ceiling: BLOCK (exit 2)
│   │
│  │
│   │
│  │   5. Log routing decision to routing_log.jsonl
│   │
│  │
│   │
│
└──────────────────────────────────────────────────────────────────────────
   │
│
│
│  Exit codes:
│
│  • 0 = PERMIT (continue with tool execution)
│
│  • 1 = ERROR (hook failure, blocks execution)
│
│  • 2 = BLOCK (routing violation, blocks with message)
│
│
│
└──────────────────────────────────────────────────────────────────────────

                                        │
                                        ▼
PHASE 5: EXECUTION
```

```
┌──────────────
```

```
                    | YES                                 | NO
                    ▼                                     ▼
            ┌─────────────────┐
┌───────────────┐
            | Validate tier   |                     | Scout metrics
|
            | is permitted    |                     | exist & fresh?
|
            | (user auth)     |                     | (< 5 min old)
|
            └─────────────────┘
└───────────────┘
                    |                                     |
          ┌─────────┴─────────┐
┌─────────────────┴─────────────────┐
          | VALID             | INVALID      | YES
| NO
          ▼                   ▼              ▼
▼
    ┌─────────┐         ┌─────────┐   ┌─────────────────┐
┌─────────────────┐
    | PERMIT  |         | BLOCK   |   | Read complexity |         |
Trigger scout   |
    | (log    |         | (invalid|   | from state file |         | OR
use default  |
    | override)|        | escape) |   └─────────────────┘         |
conservative    |
    └─────────┘         └─────────┘             |                 |
estimate        |
                                                ▼
└─────────────────┘
                                      ┌─────────────────┐
|
                                      | calculate-      |
|
                                      | complexity.sh
|◄────────────────┘
                                      | (if needed)     |
                                      └─────────────────┘
                                              |
                                              ▼
                                      ┌─────────────────┐
                                      | Read            |
                                      | recommended_    |
                                      | tier            |
                                      └─────────────────┘
                                              |
                                              ▼
                                      ┌─────────────────┐
                                      | Requested tier  |
                                      | ≤ recommended?  |
                                      └─────────────────┘
                                              |
                                ┌─────────────┴─────────────┐
                                | YES                       |
NO
                                ▼                           ▼
                        ┌─────────┐
┌─────────┐
                        | PERMIT  |                   | BLOCK
|
                        | exit 0  |                   | exit 2
|
                        |         |                   |
+message|
                        └─────────┘
└─────────┘
                                |                           |
                                └───────────┬───────────────┘
                                            ▼
                                    ┌─────────────────┐
                                    | Log decision to |
                                    | routing_log.    |
                                    | jsonl           |
                                    └─────────────────┘
```

### 4.2 Routing Log Format

```
{
  "timestamp": "2026-01-13T10:23:45Z",
  "session_id": "abc123",
  "tool_name": "Edit",
  "requested_tier": "opus",
  "calculated_tier": "sonnet",
  "complexity_score": 8.5,
  "scout_metrics": {
    "files": 12,
    "tokens": 35000,
    "modules": 2
  },
  "decision": "BLOCK",
  "reason": "requested_tier exceeds calculated ceiling",
  "force_override": false
}
```

---

# 5. The Explore Workflow

## 5.1 Seven-Phase Breakdown

```
┌──────────────
```

```
│  Agent: Sonnet or Opus (based on routing)
│
│
│  Tasks:
│
│  • Analyze scout report
│
│  • Create implementation plan
│
│  • Break into discrete phases
│
│  • Assign agent type per phase
│
│  • Estimate cost and duration
│
│
│  Output: specs.md with structure:
│  ```
│  # Implementation Plan
│  ## Phase 1: [Name]
│  - Agent: sonnet
│  - Files: [list]
│  - Tasks: [list]
│  - Estimated tokens: X
│  ## Phase 2: [Name]
│  ...
│  ```
│
│
│  Duration: 10-60s
└──────────────────────────────────────────────────────────────────────────

                                          │
                                          ▼
PHASE 5: APPROVE
┌──────────────────────────────────────────────────────────────────────────
│  Agent: Human
│
│
│  User reviews specs.md and:
│
│  • APPROVE → Continue to Phase 6
│
│  • MODIFY → Edit specs.md, return to Phase 4 for replan
│
│  • REJECT → Abort workflow, capture reason in sharp-edges/
│
│
│  Duration: User dependent (could be immediate or hours)
└──────────────────────────────────────────────────────────────────────────

                                          │
                                          ▼
PHASE 6: EXECUTE
┌──────────────────────────────────────────────────────────────────────────
│  Agents: Per-phase as specified in specs.md
│
│
│  Agent selection by task type:
│
│
┌──────────────────────────────┬──────────────┬──────────────────────────
   │
│  │ Task Type                  │ Agent        │ Rationale
│   │
│
├──────────────────────────────┼──────────────┼──────────────────────────
   │
│  │ File creation (simple)     │ Sonnet       │ Standard generation
│   │
│  │ File creation (complex)    │ Opus         │ Architectural
decisions         │   │
│  │ Refactoring (< 5 files)    │ Sonnet       │ Bounded scope
│   │
│  │ Refactoring (> 5 files)    │ Opus         │ Cross-cutting
concerns          │   │
│  │ Test writing               │ Sonnet       │ Pattern-based
```

```
│   │
│  │ Documentation               │ Sonnet      │ Standard prose
│   │
│  │ Security review             │ Opus        │ Critical analysis
│   │
│  │ Large file analysis         │ Gemini      │ Context
requirements            │   │
│
└──────────────────────────────┴─────────────┴──────────────────────────
   │
│
│
│  Each phase execution:
│
│  1. Load phase context from specs.md
│
│  2. PreToolUse hook validates tier permission
│
│  3. Execute with selected agent
│
│  4. PostToolUse hook logs outcome
│
│  5. Validate completion (SubagentStop hook)
│
│  6. Proceed to next phase or flag for human intervention
│
│
│
│  Duration: Varies by task complexity (minutes to hours)
│
└──────────────────────────────────────────────────────────────────────

                                        │
                                        ▼
PHASE 7: ARCHIVE
┌──────────────────────────────────────────────────────────────────────

│  Agent: Memory Archivist (Haiku or Sonnet)
│
│
│
│  Sources processed:
│
│  • specs.md (architectural decisions)
│
│  • Execution logs (what worked, what failed)
│
│  • Human modifications (corrections, overrides)
│
│  • Sharp edge detections (errors encountered)
│
│
│
│  Outputs:
│
│  • .claude/memory/decisions/[date]-[topic].md
│
│  • .claude/memory/sharp-edges/[date]-[issue].md
│
│  • .claude/memory/facts/[topic].md (updates)
│
│
│
│  Duration: 5-30s (background, non-blocking)
│
└──────────────────────────────────────────────────────────────────────
```

---

# 6. Memory Pipeline

## 6.1 Memory Architecture

```
┌──────────────────────────────────────────────────────────────────────

│                              MEMORY PIPELINE
│
└──────────────────────────────────────────────────────────────────────


SOURCES                        PROCESSING
STORAGE
──────────────────────────────────────────────────────────────────────


 ┌──────────────┐
 │ Architect    │
 │ specs.md     │──┐
 └──────────────┘  │
                   │
 ┌──────────────┐  │         ┌──────────────┐
 ┌──────────────┐
 │ Sharp Edge   │  │         │              │         │
decisions/      │
 │ Detector     │──┼────────→│    MEMORY    │────────→│
```

```
│
└──────────────────┘  │         │   ARCHIVIST     │         │ 2026-
01-13-     │
                      │         │                 │         │ auth-
refactor   │
┌──────────────────┐  │         │ (Haiku/Sonnet)  │         │ .md
│
│ Session          │──┤         │                 │
└──────────────────┘
│ Context          │  │         │ • Extracts      │
└──────────────────┘  │         │   learnings     │
┌──────────────────┐
                      │         │ • Categorizes   │         │ sharp-
edges/    │
┌──────────────────┐  │         │ • Formats with  │────────→│
│
│ Human            │──┤         │   frontmatter   │         │ jwt-
expiry-     │
│ Corrections      │  │         │ • Commits to    │         │
gotcha.md       │
└──────────────────┘  │         │   git           │
└──────────────────┘
                      │         │                 │
┌──────────────────┐  │         └─────────────────┘
┌──────────────────┐
│ Tool Execution   │──┘                                     │ facts/
│
│ Logs             │                                  ──────→│
│
└──────────────────┘                                         │
project-        │
                                                             │
structure.md    │
└──────────────────┘


┌──────────────────┐
                                                             │
preferences/    │
                                                  ──────→│
│
                                                             │ coding-
style    │
                                                             │ .md
│
└──────────────────┘
```

```
                          RETRIEVAL
                     ────────────────────────────

                     ┌──────────────────┐
                     │ query-memory.sh  │
                     │                  │
                     │ • BM25 search    │
                     │ • Frontmatter    │
                     │   filtering      │
                     │ • Recency        │
                     │   weighting      │
                     └──────────────────┘
                              │
                              ▼
                     ┌──────────────────┐
                     │ Context          │
                     │ Assembly         │──────→ Agent receives
relevant memory
                     └──────────────────┘
```

### 6.2 Memory File Format

All memory files use YAML frontmatter with markdown body:

```
---
title: "Decision: JWT Token Refresh Strategy"
created: 2026-01-13
updated: 2026-01-13
category: architecture
tags: [authentication, jwt, security]
related: [./oauth-implementation.md, ../sharp-edges/jwt-expiry-
gotcha.md]
status: active
confidence: high
source: explore-session-2026-01-13
summary: "Chose sliding window refresh over fixed expiry for better
UX"
---

## Context

During authentication refactor, needed to decide between fixed JWT
expiry
(requiring re-login) and sliding window refresh (extending on
activity).

## Decision
```

```
Implemented sliding window refresh with:
- 15-minute access token
- 7-day refresh token
- Refresh on any authenticated request within 5 minutes of expiry

## Rationale

- Better user experience (no unexpected logouts during active
sessions)
- Security acceptable for this application's threat model
- Aligns with OAuth 2.0 best practices

## Consequences

- Must handle token refresh failures gracefully
- Need to implement refresh token rotation
- Slightly more complex token validation logic
```

### 6.3 Retrieval Mechanism

Current implementation uses grep-based search with YAML frontmatter filtering:

```
# query-memory.sh simplified logic
search_memory() {
    local query="$1"
    local category="${2:-}"

    # Find files matching category filter
    if [ -n "$category" ]; then
        files=$(grep -l "category: $category"
~/.claude/memory/**/*.md)
    else
        files=$(find ~/.claude/memory -name "*.md")
    fi

    # BM25-style term matching (simplified)
    for file in $files; do
        score=$(calculate_bm25_score "$file" "$query")
        echo "$score $file"
    done | sort -rn | head -10
}
```

**Planned upgrade path:** Implement proper BM25 via `rank_bm25` Python library, with optional sqlite-vec for semantic search when pattern matching proves insufficient.

---

## 7. Inter-Agent Communication

### 7.1 State File Specifications

#### scout_metrics.json

```
{
  "schema_version": "1.0.0",
  "generated_at": "2026-01-13T10:23:45Z",
  "scout_agent": "haiku",
  "scout_report": {
    "scope_metrics": {
      "total_files": 15,
      "file_types": {
        ".py": 10,
        ".md": 3,
        ".json": 2
      },
      "estimated_tokens": 45000,
      "largest_file": {
        "path": "src/auth/handlers.py",
        "tokens": 8500
      }
    },
    "complexity_signals": {
      "cross_file_dependencies": 8,
      "module_count": 3,
      "circular_imports": 0,
      "test_coverage_files": 5
    },
    "recommendations": {
      "suggested_tier": "sonnet",
      "gemini_offload_candidates": [],
      "risk_factors": ["large handlers.py may need splitting"]
    }
  }
}
```

#### handoff.json (Inter-agent handoff)

```
{
  "schema_version": "1.0.0",
  "handoff_id": "uuid-here",
  "from_agent": "architect",
  "to_agent": "executor",
```

```
          "created_at": "2026-01-13T10:25:00Z",
          "context": {
            "task_summary": "Implement OAuth refresh token rotation",
            "files_in_scope": ["src/auth/tokens.py",
"src/auth/middleware.py"],
            "critical_constraints": [
              "Must maintain backward compatibility with existing tokens",
              "Refresh rotation must be atomic"
            ],
            "success_criteria": [
              "All existing tests pass",
              "New rotation tests added",
              "No breaking API changes"
            ]
          },
          "artifacts": {
            "specs_path": ".claude/tmp/specs.md",
            "scout_metrics_path": ".claude/tmp/scout_metrics.json"
          },
          "metadata": {
            "estimated_tokens": 25000,
            "estimated_duration_minutes": 15,
            "tier_ceiling": "sonnet"
          }
        }
```

### 7.2 Handoff Protocol

```
┌──────────────┐                    ┌──────────────┐
│   Agent A    │                    │   Agent B    │
│  (Architect) │                    │  (Executor)  │
└──────────────┘                    └──────────────┘
        │                                  │
        │  1. Complete task                │
        │                                  │
        │  2. Write handoff.json           │
        │     - Task summary               │
        │     - Context                    │
        │     - Constraints                │
        │     - Success criteria           │
        │                                  │
        │  3. Write completion signal      │
        ├──────────────────────────────────►│
        │                                  │
        │                            4. Read handoff.json
        │                                  │
        │                            5. Validate schema
        │                                  │
        │                            6. Load context
        │                                  │
        │                            7. Execute task
        │                                  │
        │  8. SubagentStop hook            │
        ◄──────────────────────────────────┤
        │     validates completion         │
        │                                  │
```

---

## 8. External Engine Integration (Gemini)

### 8.1 Integration Architecture

```
┌──────────────────────────────────
```

**Pattern 1: Large Codebase Analysis**

```
# Orchestrator detects token estimate > 50K
# Invokes Gemini via CLI pipe

gemini-cli analyze \
  --protocol codebase-analysis \
  --input-dir ./src \
  --output .claude/tmp/codebase_analysis.json \
  --max-tokens 100000
```

**Pattern 2: Document Synthesis (Parallel Workers)**

```
# For documents exceeding single-call limits
# Spin up parallel Gemini workers with overlapping windows

for chunk in $(split_document_with_overlap "$DOCUMENT" 40000 10000);
do
    gemini-cli summarize \
      --input "$chunk" \
      --output ".claude/tmp/summary_${i}.json" &
done
wait

# Synthesis phase
gemini-cli synthesize \
  --inputs ".claude/tmp/summary_*.json" \
  --output ".claude/tmp/final_synthesis.md"
```

### 8.3 Handover Protocol Format

```
# Gemini Handover Protocol

## Task
[Clear description of what Gemini should accomplish]

## Context
[Relevant background information]

## Input Files
- path/to/file1.py
- path/to/file2.py

## Constraints
- Maximum output tokens: 10000
- Focus areas: [specific aspects]
- Exclude: [what to ignore]

## Expected Output Format
[Structured format specification]

## Success Criteria
- [ ] Criterion 1
- [ ] Criterion 2
```

### 8.4 When to Route to Gemini

| Condition | Action |
|---|---|
| Token estimate > 50,000 | Route to Gemini |
| Token estimate > 100,000 | FORCE Gemini |
| Files > 20 | Consider Gemini scout first |
| Full repo analysis requested | Gemini required |
| Document synthesis > 30 pages | Parallel Gemini workers |

---

## Summary

This technical architecture provides:

1. **Deterministic control** — Bash hooks enforce routing without LLM interpretation
2. **Transparent state** — All inter-agent communication via human-readable files
3. **Optimal model allocation** — Each model tier serves its appropriate use cases
4. **Scalable context** — Gemini integration handles workloads beyond Claude's limits
5. **Persistent learning** — Memory pipeline captures and retrieves organizational knowledge

The architecture is designed for evolution. The following sections describe the novel innovations that enable progressive autonomy and the implementation roadmap for realizing the full vision.

---

*Continue to Part III: Intellectual Property & Novel Innovations* → #
Part III: Intellectual Property & Novel Innovations

## Four Architectural Patterns Representing Significant Innovation

---

## Overview

Lisan al-Gaib introduces four architectural innovations that, to our knowledge, have no direct precedent in the multi-agent systems literature or commercial frameworks. These innovations address fundamental limitations in current approaches to AI agent orchestration:

| Innovation | Problem Addressed | Current Approaches | Our Solution |
|---|---|---|---|
| Emergent Schema Discovery | Pre-defined schemas constrain learning | RLHF with fixed feedback categories | Discover schemas from accumulated observations |
| Subagent Spawning | Fixed agent populations limit adaptability | Manual agent creation | Generate agents from gap analysis |
| Apprenticeship Learning | Human-in-the-loop doesn't scale | Humans approve/reject agent proposals | System learns to replicate human decisions |
| Code-Enforced Routing | LLM routing is non-deterministic | LLM-based or keyword routing | Bash arithmetic enforces routing |

Each innovation is detailed below with architectural specifications, implementation guidance, novelty assessment, and competitive differentiation.

---

## Innovation 1: Emergent Schema Discovery

### 1.1 Concept Definition

**Traditional approach:** Define a schema for capturing user feedback or system decisions, then collect data against that schema. The schema constrains what can be learned.

**Our approach:** Accumulate raw behavioral observations without schema constraints. Periodically analyze accumulated observations for patterns. When statistically significant clusters emerge, propose a schema that describes the discovered patterns. Human approves before activation.

This is **unsupervised learning applied to system architecture evolution**. The system discovers its own feedback categories rather than being told what to measure.

### 1.2 Architectural Specification

```
┌──────────────
```

```
│      "complexity_score": 8.5,
│
│      "recommended_tier": "sonnet",
│
│      "files_in_scope": 12
│
│    },
│
│    "action": {
│
│      "type": "tier_override",
│
│      "from": "sonnet",
│
│      "to": "opus",
│
│      "reason_provided": "security-critical code"
│
│    },
│
│    "outcome": {
│
│      "success": true,
│
│      "duration_seconds": 180
│
│    }
│
│  }
│
│
│
│  Key principle: Capture WHAT happened, not what category it
belongs to.            │
│  Categories emerge later.
│
│
│
└──────────────────────────────────────────────────────────────────────

                                          │
                                          │ Trigger: 200+
observations
                                          │ OR manual /schema-
discovery
                                          ▼
PHASE 2: PATTERN DETECTION
┌──────────────────────────────────────────────────────────────────────
│
│
│  Schema Discovery Agent (Sonnet with extended thinking)
│
│
│  Input: All observations from past N days/sessions
│
│
│  Task:
│
│  "Analyze these behavioral observations. Identify recurring
patterns.              │
│   A valid pattern requires:
│
│   - At least 10 instances
│
│   - Distinguishable characteristics from other patterns
│
│   - Predictive value (pattern membership predicts outcome or
action)               │
│
│
│   For each discovered pattern, provide:
│
│   - Proposed name
│
│   - Defining characteristics
│
│   - Instance count
│
│   - Confidence score
│
│   - Suggested schema fields to capture this pattern"
│
│
│
│  Statistical validation:
│
│  - Minimum 30 observations per cluster for 80% statistical power
│
│  - Silhouette score > 0.5 for cluster validity
│
│  - Bootstrap stability > 80% reproducibility
│
│
│
```

```
│  Output: pattern_analysis.json
│
│  {
│
│    "analysis_date": "2026-01-13",
│
│    "observations_analyzed": 234,
│
│    "patterns_discovered": [
│
│      {
│
│        "name": "scope_reduction",
│
│        "instance_count": 47,
│
│        "confidence": 0.87,
│
│        "characteristics": [
│
│          "Human modified plan to reduce scope",
│

│          "Original scope > 5 files",
│

│          "Reduction typically 40-60%"
│

│        ],
│

│        "suggested_schema_fields": {
│

│          "original_scope": "integer",
│

│          "final_scope": "integer",
│

│          "reduction_reason": "enum[complexity|time|risk]"
│

│        }
│

│      },
│

│      {
│

│        "name": "tier_override_up",
│

│        "instance_count": 31,
│

│        ...
│

│      }
│

│    ],
│

│    "schema_proposal": { ... }
│

│  }
│
│
│
└──────────────────────────────────────────────────────────────────────────────
                                          │
                                          │ Human review required
                                          ▼
PHASE 3: SCHEMA PROPOSAL & APPROVAL
┌──────────────────────────────────────────────────────────────────────────────

│
│
│  Orchestrator presents findings to human:
│
│
│  "📊 SCHEMA DISCOVERY RESULTS
│
│
│   I analyzed 234 observations and found 4 distinct patterns:
│
│
│   1. scope_reduction (47 instances, 87% confidence)
│

│      You frequently reduce task scope when > 5 files involved
│
│
│   2. tier_override_up (31 instances, 82% confidence)
│

│      You escalate to Opus for security-related code
│
│
│   3. clarification_request (28 instances, 79% confidence)
│

│      You ask for clarification on ambiguous requirements
│
│
```

```
│
│   4. quick_approval (89 instances, 94% confidence)
│
│      Simple tasks are approved without modification
│
│
│
│   I propose activating a decision schema to capture these
patterns                 │
│   more precisely. This will enable me to:
│
│   - Predict when you'll want scope reduction
│
│   - Auto-escalate security-critical code
│
│   - Eventually automate quick_approval decisions
│
│
│
│   [View full schema] [Approve] [Modify] [Reject]"
│
│
│
│  Human response determines next action:
│
│  - Approve: Schema activated, future observations captured
against it               │
│  - Modify: Human edits schema, then activates
│
│  - Reject: Continue raw observation accumulation
│
│
│
└──────────────────────────────────────────────────────────────────────

                                          │
                                          │ On approval
                                          ▼
PHASE 4: SCHEMA ACTIVATION & BACKFILL
┌──────────────────────────────────────────────────────────────────────

│
│
│  1. Create schema definition file:
│
│     .claude/schemas/decisions-v1.0.json
│
│
│
│  2. Update observation capture to use new schema
│
│
│
│  3. Backfill existing observations (async, Haiku-tier):
│
│     - Classify each historical observation into discovered
patterns                 │
│     - Add pattern_classification field
│
│     - Maintain original raw data
│
│
│
│  4. Enable pattern-based features:
│
│     - Predictive suggestions ("Based on pattern, you may want
to...")               │
│     - Confidence tracking per pattern
│
│     - Autonomy level candidates identified
│
│
│
└──────────────────────────────────────────────────────────────────────
```

### 1.3 Novelty Assessment

**Existing approaches in RLHF/feedback systems:**

| System | Schema Approach | Limitation |
|---|---|---|
| OpenAI RLHF | Pre-defined helpfulness/harmlessness scales | Categories fixed at design time |
| Constitutional AI | Pre-defined principles | Principles must be articulated upfront |
| LangSmith feedback | User-defined categories | Categories manually specified |
| Thumbs up/down | Binary feedback | No nuance, no category discovery |

**Our differentiation:**

1. **No a priori categories** — The system discovers what matters by observing behavior

2. **Statistical validation** — Patterns must meet significance thresholds, not just exist
3. **Human approval gate** — Discovery is automated, activation is human-controlled
4. **Backfill capability** — Historical data becomes more valuable over time

**Prior art search findings:** No published systems implement emergent schema discovery for agent feedback. The closest analog is unsupervised topic modeling (LDA, etc.) applied to text, but not applied to behavioral observation streams in agent systems.

### 1.4 Competitive Advantage

1. **Adapts to any workflow** — The system learns YOUR patterns, not generic patterns
2. **Reveals unknown unknowns** — Discovers behavioral patterns the user wasn't aware of
3. **Reduces schema maintenance** — No need to manually update feedback categories
4. **Enables organic evolution** — New patterns emerge as usage evolves

### 1.5 Implementation Priority

**Phase 4 in roadmap (Months 4-5)**

Dependencies: - Observation capture infrastructure (Phase 2) - Memory file standardization (Phase 2) - Weekly review process (Phase 3)

---

## Innovation 2: Gap Analysis → Subagent Spawning

### 2.1 Concept Definition

**Traditional approach:** Multi-agent systems have fixed agent populations defined at design time. Adding capabilities requires manual agent development.

**Our approach:** Weekly architecture review identifies capability gaps from actual usage patterns. For gaps that cannot be resolved by configuration changes, the system generates complete subagent definitions ready for deployment. Humans approve before activation.

This creates a **generative architecture** — the system grows capabilities based on observed deficiencies rather than anticipated needs.

### 2.2 Architectural Specification

```
┌─────────────
```

```
│
│  5. Identify routing inefficiencies
│
│
│
│  For each gap, determine:
│
│  - Can this be resolved by configuration change?
│
│  - Does this require new subagent capability?
│
│  - Is this a one-off or recurring pattern?
│
│
│
└──────────────────────────────────────────────────────────────────────────────

                                          │
                                          ▼
PHASE 2: RECOMMENDATION GENERATION
┌──────────────────────────────────────────────────────────────────────────────
│
│
│  Gap Classification:
│
│
│
│
┌──────────────────────────────────────────────────────────────────────────────
   │
│  │ Gap Type              │ Resolution              │ Output
│   │
│
├──────────────────────────┼─────────────────────────┼──────────────────────────
   │
│  │ Routing threshold     │ Config change           │ routing-
schema.json diff   │   │
│  │ Missing agent type    │ Subagent spawn          │ Agent
definition files     │   │
│  │ Memory retrieval      │ Query enhancement       │ query-
memory.sh update     │   │
│  │ Context limits        │ Gemini integration      │ Protocol
addition          │   │
│  │ User workflow         │ Skill addition          │ SKILL.md
template          │   │
│
└──────────────────────────────────────────────────────────────────────────────
   │
│
│
│  For subagent spawning, generate:
│
│  .claude/agents/[agent-name]/
│
│  ├── agent.md           # Agent definition and purpose
│
│  ├── agent.yaml         # Configuration (tier, triggers,
constraints)              │
│  └── CLAUDE.md          # Instructions for this agent
│
│
│
└──────────────────────────────────────────────────────────────────────────────

                                          │
                                          ▼
PHASE 3: HUMAN REVIEW & APPROVAL
┌──────────────────────────────────────────────────────────────────────────────

│
│
│  Orchestrator presents recommendations:
│
│
│
│  "🔍 GAP ANALYSIS RESULTS
│
│
│
│   This week I identified 3 capability gaps:
│
│
│
│   GAP 1: Document synthesis failures (4 occurrences)
│
│   Root cause: Context boundaries causing information loss
│
│   Recommendation: Spawn 'parallel-summarizer' subagent swarm
│
│   [View agent definition] [Approve] [Modify] [Reject]
│
│
│
│   GAP 2: Excessive Opus routing (12 tasks could have been Sonnet)
│
│   Root cause: Complexity threshold too aggressive
```

```
│
│   Recommendation: Raise opus threshold from 10 to 12
│
│   [View diff] [Approve] [Modify] [Reject]
│
│
│   GAP 3: Memory retrieval misses (3 relevant decisions not found)
│
│   Root cause: Keyword matching insufficient for conceptual queries
│
│   Recommendation: Add BM25 indexing to query-memory.sh
│
│   [View implementation] [Approve] [Modify] [Reject]"
│
│
└──────────────────────────────────────────────────────────────────────

                                        │
                                        ▼
PHASE 4: SHADOW DEPLOYMENT (For new agents)
┌──────────────────────────────────────────────────────────────────────
│
│
│  New agents enter shadow mode before production:
│
│
│  agents-index.json entry:
│
│  {
│
│    "parallel-summarizer": {
│
│      "status": "shadow",
│
│      "created": "2026-01-13",
│
│      "shadow_start": "2026-01-13",
│
│      "promotion_criteria": {
│
│        "min_invocations": 10,
│
│        "success_rate_threshold": 0.9,
│
│        "max_shadow_duration_days": 14
│
│      },
│
│      "shadow_metrics": {
│
│        "invocations": 0,
│
│        "successes": 0,
│
│        "failures": 0
│
│      }
│
│    }
│
│  }
│
│
│  Shadow behavior:
│
│  - Agent runs in parallel with existing solution
│
│  - Output compared but not used
│
│  - Metrics tracked for promotion decision
│
│  - Human can force-promote or reject
│
│
│  Promotion criteria (from research):
│
│  - Minimum 10 invocations
│
│  - Success rate ≥ 90%
│
│  - Error rate ≤ 0.1% above baseline
│
│  - Maximum 14 days in shadow
│
│
└──────────────────────────────────────────────────────────────────────

                                        │
                                        ▼
PHASE 5: LIFECYCLE MANAGEMENT
```

```
┌──────────────
```

evolve

**Prior art search findings:** No published systems implement automated subagent generation from gap analysis. The closest analog is AutoML for model architecture search, but not applied to agent architecture.

### 2.5 Competitive Advantage

1. **Self-healing architecture** — System addresses its own deficiencies
2. **Reduced maintenance burden** — No manual agent development for common gaps
3. **Organic capability growth** — Architecture evolves with usage patterns
4. **Risk-managed deployment** — Shadow mode prevents untested agents from causing harm

### 2.6 Implementation Priority

**Phase 5 in roadmap (Months 6-7)**

Dependencies: - Weekly review process (Phase 3) - Systems Architect agent (Phase 3) - Agent registry infrastructure (Phase 1)

---

## Innovation 3: Apprenticeship Learning Model ("Agent-in-the-Loop" Inversion)

### 3.1 Concept Definition

**Traditional human-in-the-loop:**

```
Agent proposes action → Human approves/rejects → Agent executes (or
not)
```

The agent is the actor; the human is the gatekeeper.

**Our approach (Apprenticeship Learning):**

```
Human acts → System observes → Patterns emerge → Agent learns to
replicate →
Human validates agent's replication → Agent assumes routine
decisions →
Human handles novel cases
```

The human is the teacher; the agent is the apprentice learning to become the human (for routine decisions).

### 3.2 Architectural Specification

```
┌──────────────────────────────────────────────────────────────────────────────
│                        APPRENTICESHIP LEARNING MODEL
│
└──────────────────────────────────────────────────────────────────────────────


CONCEPT: The system learns to replicate the human orchestrator's
decision-making
         for routine decisions, progressively reducing human
involvement.

┌──────────────────────────────────────────────────────────────────────────────
│
│
│                             LEARNING PROGRESSION
│
│
│
│  LEVEL 1: OPERATOR
│
│
┌──────────────────────────────────────────────────────────────────────────────
   │
│  │ Human approves ALL decisions
│   │
│  │ System observes and captures
│   │
│  │ Decision patterns begin accumulating
│   │
│  │ No automation
│   │
│  │
│   │
│  │ Duration: Until 100+ decisions captured
│   │
│
└──────────────────────────────────────────────────────────────────────────────
   │
│                                         │
│
│                                         ▼ Pattern detection
```

```
triggers                |
|  LEVEL 2: COLLABORATOR
|
|
┌──────────────────────────────────────────────────────────────────────────
   |
|  | System suggests decisions based on patterns
|   |
|  | Human confirms, modifies, or rejects
|   |
|  | Confirmation/rejection feeds back to learning
|   |
|  | Still no autonomous action
|   |
|  |
|   |
|  | Duration: Until 95% success rate on suggestions over 200
decisions           |   |
|
└──────────────────────────────────────────────────────────────────────────
   |
|                                          |
|
|                                          ▼ Success threshold met
|
|  LEVEL 3: CONSULTANT
|
|
┌──────────────────────────────────────────────────────────────────────────
   |
|  | System auto-approves HIGH-CONFIDENCE decisions (>95% pattern
match)          |   |
|  | Human consulted on MEDIUM-CONFIDENCE decisions
|   |
|  | Human decides on LOW-CONFIDENCE / novel situations
|   |
|  | All auto-decisions logged for audit
|   |
|  |
|   |
|  | Category-specific: Different decision types progress
independently           |   |
|  | Duration: Until 98% success rate over 500 decisions in category
|   |
|
└──────────────────────────────────────────────────────────────────────────
   |
|                                          |
|
|                                          ▼ Extended success
demonstrated             |
|  LEVEL 4: APPROVER
|
|
┌──────────────────────────────────────────────────────────────────────────
   |
|  | System handles routine decisions autonomously
|   |
|  | Human reviews summary reports (daily/weekly)
|   |
|  | Human intervenes only on flagged anomalies
|   |
|  | Human approves CRITICAL decisions (security, architecture)
|   |
|  |
|   |
|  | Duration: Indefinite with periodic validation
|   |
|
└──────────────────────────────────────────────────────────────────────────
   |
|                                          |
|
|                                          ▼ Full trust established
|
|  LEVEL 5: OBSERVER
|
|
┌──────────────────────────────────────────────────────────────────────────
   |
|  | System operates autonomously for defined domain
|   |
|  | Human observes via dashboard/reports
|   |
|  | Human can intervene at any time
|   |
|  | System flags when it encounters novel situations
|   |
|  |
|   |
|  | Note: Level 5 only for specific, well-defined decision
categories           |   |
|  | Most domains remain at Level 3-4
|   |
|
└──────────────────────────────────────────────────────────────────────────
   |
```

```
│
│
└──────────────────────────────────────────────────────────────────────────────
```

### 3.3 Decision Capture Schema

To enable apprenticeship learning, decisions must be captured with sufficient context for pattern matching:

```
{
  "decision_id": "uuid",
  "timestamp": "2026-01-13T10:23:45Z",
  "session_id": "session-uuid",

  "decision_category": "routing_override",

  "context": {
    "task_type": "code_modification",
    "task_description": "Refactor authentication module",
    "complexity_score": 8.5,
    "files_in_scope": 12,
    "estimated_tokens": 35000,
    "tags": ["security", "authentication", "refactor"]
  },

  "system_recommendation": {
    "action": "route_to_sonnet",
    "confidence": 0.82,
    "reasoning": "Complexity score 8.5 within sonnet range"
  },

  "human_decision": {
    "action": "route_to_opus",
    "reasoning_provided": "Security-critical code needs careful
review",
    "time_to_decide_seconds": 12
  },

  "outcome": {
    "task_success": true,
    "quality_rating": null,
    "issues_encountered": [],
    "would_recommend_same_decision": null
  },

  "learning_metadata": {
    "autonomy_level_at_time": 2,
    "pattern_match_candidates": ["security_escalation",
"complexity_override"],
    "should_automate_similar": false,
    "requires_human_always": false
  }
}
```

### 3.4 Autonomy Level Tracking

```yaml
# .claude/autonomy-levels.yaml
schema_version: 1.0.0
updated: 2026-01-13

categories:
  quick_approval:
    current_level: 3  # CONSULTANT - auto-approving high-confidence
    decision_count: 127
    success_rate: 0.98
    last_human_override: 2026-01-10
    auto_approve_threshold: 0.95

  routing_decisions:
    current_level: 2  # COLLABORATOR - suggesting but human confirms
    decision_count: 89
    success_rate: 0.91
    promotion_blocked_reason: "success rate below 95% threshold"

  security_decisions:
    current_level: 1  # OPERATOR - human approves all
    decision_count: 23
    success_rate: null  # Not enough data
    notes: "Security decisions will likely remain human-approved"
    max_allowed_level: 2  # Never fully automate

  scope_modifications:
    current_level: 2  # COLLABORATOR
    decision_count: 47
    success_rate: 0.87
    promotion_blocked_reason: "success rate below 95% threshold"
```

### 3.5 Novelty Assessment

**Existing approaches:**

| System | Learning Model | Human Role |
|---|---|---|
| Standard RLHF | Human rates outputs | Feedback provider |

| | | |
|---|---|---|
| Constitutional AI | Human defines principles | Rule definer |
| Active Learning | Human labels uncertain cases | Data labeler |
| Imitation Learning | Human demonstrates behavior | Demonstrator (batch) |

**Our differentiation:**

1. **Continuous apprenticeship** — Learning happens during normal use, not separate training
2. **Category-specific progression** — Different decision types progress independently
3. **Transparent level tracking** — User always knows automation level per category
4. **Reversible autonomy** — Levels can decrease if success rate drops
5. **Graceful degradation** — Novel situations always surface to human

**Prior art search findings:** Apprenticeship learning exists in robotics (learning from demonstration), but applying this model to AI orchestration with progressive autonomy levels is novel. The closest analog is Levels of Automation taxonomies from human factors engineering, but those describe states, not transitions.

### 3.6 Competitive Advantage

1. **Personalized automation** — System learns YOUR decision-making, not generic rules
2. **Earned trust** — Automation is earned through demonstrated competence
3. **Maintained oversight** — Human never fully out of the loop; can intervene anytime
4. **Category-specific control** — Automate routine decisions while keeping critical ones human-controlled

### 3.7 Implementation Priority

**Phase 6 in roadmap (Months 8-10)**

Dependencies: - Decision capture schema (Phase 2) - Pattern detection (Phase 4) - Weekly review with autonomy assessment (Phase 3)

---

## Innovation 4: Code-Enforced Deterministic Routing

### 4.1 Concept Definition

**Traditional approach:** Use the LLM itself to decide which model tier to use, or use keyword matching.

**Our approach:** Bash scripts calculate complexity scores using arithmetic formulas. The code enforces tier ceilings. The LLM cannot route itself to expensive tiers regardless of what it "decides."

### 4.2 Implementation

```bash
#!/bin/bash
# calculate-complexity.sh

METRICS_FILE="$HOME/.claude/tmp/scout_metrics.json"

# Extract values from scout report
TOKENS=$(jq '.scout_report.scope_metrics.estimated_tokens' "$METRICS_FILE")
FILES=$(jq '.scout_report.scope_metrics.total_files' "$METRICS_FILE")
MODULES=$(jq '.scout_report.complexity_signals.module_count // 1' "$METRICS_FILE")

# Calculate complexity score using ARITHMETIC, not LLM interpretation
SCORE=$(echo "scale=2; ($TOKENS / 10000) + ($FILES / 5) + ($MODULES * 2)" | bc)

# Determine tier using NUMERIC COMPARISON, not LLM judgment
if (( $(echo "$SCORE < 2" | bc -l) )); then
    TIER="haiku"
elif (( $(echo "$SCORE < 10" | bc -l) )); then
    TIER="sonnet"
else
    TIER="opus"
fi

# Force Gemini for large context regardless of score
if (( TOKENS > 50000 )); then
    TIER="gemini"
fi
```

```
# Write results (code decides, not LLM)
echo "$SCORE" > "$HOME/.claude/tmp/complexity_score"
echo "$TIER" > "$HOME/.claude/tmp/recommended_tier"
```

### 4.3 Why This Matters

| Risk | LLM-Based Routing | Code-Enforced Routing |
|---|---|---|
| Prompt injection | Attacker can trick LLM into expensive tier | Code ignores prompts entirely |
| Cost overrun | LLM may rationalize expensive choices | Arithmetic doesn't rationalize |
| Inconsistency | Same task may route differently | Deterministic for same inputs |
| Auditability | "Why did it choose Opus?" — unclear | Formula is explicit and logged |
| Latency | 100-500ms for routing decision | <10ms file I/O |

### 4.4 Novelty Assessment

**Existing approaches:**

| Framework | Routing Method |
|---|---|
| LangChain | LLM-based or manual |
| AutoGPT | LLM self-selects |
| Claude native | Single model, no routing |
| RouteLLM | ML classifier (still model-based) |

**Our differentiation:**

Code-enforced routing using bash arithmetic is, to our knowledge, unique in production multi-agent systems. The pattern of using shell scripts as an enforcement layer between user intent and model selection is novel.

### 4.5 Competitive Advantage

1. **Predictable costs** — Formula determines cost, not LLM mood
2. **Zero routing latency** — No LLM call needed for routing
3. **Fully auditable** — Every routing decision traceable to formula
4. **Manipulation-resistant** — Prompt injection cannot affect routing
5. **Simple to adjust** — Change thresholds in one file

---

## 5. IP Protection Recommendations

### 5.1 Documentation Requirements

To establish prior art and potential patent claims:

1. **Timestamp all documents** — This guide serves as dated documentation
2. **Maintain git history** — All implementation changes tracked
3. **Publish technical descriptions** — Consider blog posts or technical papers
4. **Record conception dates** — Brain dump messages are dated evidence

### 5.2 Provisional Patent Candidates

| Innovation | Patentability Assessment | Priority |
|---|---|---|
| Emergent Schema Discovery | High — Novel method, clear utility | High |
| Subagent Spawning | High — Novel method, clear utility | High |
| Apprenticeship Learning | Medium — Builds on existing concepts | Medium |
| Code-Enforced Routing | Medium — Novel application, simple method | Low |

### 5.3 Trade Secret Considerations

Some implementation details may be better protected as trade secrets than patents:

- Specific threshold values and their derivation
- Observation capture heuristics
- Pattern detection algorithms tuned to specific domains

### 5.4 Recommended Actions

1. **Immediate:** File provisional patent applications for Innovations 1 and 2
2. **Near-term:** Document implementation details sufficient for reproduction

3. **Ongoing:** Maintain dated development records
4. **Consider:** Academic publication for prior art establishment

---

## Summary

These four innovations represent genuine architectural novelty in the multi-agent systems space:

1. **Emergent Schema Discovery** — The system discovers what to measure, not just how to measure it
2. **Subagent Spawning** — The architecture grows capabilities based on observed needs
3. **Apprenticeship Learning** — The system learns to become the human orchestrator
4. **Code-Enforced Routing** — Deterministic control that LLMs cannot circumvent

Together, they create an architecture that is **self-evolving** — it improves through use rather than requiring manual enhancement. This positions Lisan al-Gaib as a fundamentally different approach to AI orchestration, one where the human trains the system through normal use rather than explicit configuration.

---

*Continue to Part IV: Gap Analysis & Current State Assessment →* #
Part IV: Gap Analysis & Current State Assessment

# Systematic Evaluation of Architectural Gaps and Enhancement Priorities

---

## 1. Methodology

### 1.1 Evaluation Frameworks Applied

This gap analysis synthesizes findings from three evaluation approaches:

**1. TOGAF Architecture Development Method (ADM)** - Establishes baseline architecture (current state) - Defines target architecture (desired state) - Performs gap analysis between states - Prioritizes remediation based on business value

**2. MAST Failure Taxonomy (UC Berkeley, 2025)** - Comprehensive classification of multi-agent system failure modes - Research finding: 41-86.7% of multi-agent systems fail in production - Dominant failure categories: - Inter-agent misalignment (35% of failures) - Task verification failures (25% of failures) - Specification failures (22% of failures) - Memory/context failures (18% of failures)

**3. CLASSic Evaluation Framework** - **C**ost: Cost per successful task by tier - **L**atency: Response time by complexity level - **A**ccuracy: Task completion rate - **S**tability: Output variance for same input - **S**ecurity: Resistance to prompt injection and manipulation

### 1.2 Assessment Scope

| Aspect | Included | Excluded |
|---|---|---|
| Core orchestration | ✅ | |
| Routing enforcement | ✅ | |
| State management | ✅ | |
| Memory pipeline | ✅ | |
| Hook architecture | ✅ | |
| Gemini integration | ✅ | |
| UI/presentation layer | | ❌ (out of scope) |
| Deployment infrastructure | | ❌ (local-first) |
| External integrations | | ❌ (beyond Gemini) |

---

## 2. Current State Summary

### 2.1 Architecture Strengths

| Component | Assessment | Evidence |
|---|---|---|
| **Routing enforcement** | Strong | Bash hooks provide deterministic control; formula-based scoring eliminates LLM routing variability |
| **Tier structure** | Strong | Clear separation of concerns; cost optimization |

| | | |
|---|---|---|
| | | through appropriate tier selection |
| **File-based state** | Strong | Transparent, auditable, git-versioned; no infrastructure dependencies |
| **Gemini integration** | Strong | Effective large-context offload; CLI pipe architecture avoids tight coupling |
| **Explore workflow** | Strong | Comprehensive 7-phase process with human approval gates |

### 2.2 Architecture Neutral Areas

| Component | Assessment | Notes |
|---|---|---|
| **Memory retrieval** | Adequate | grep-based search works for current scale (<500 files); upgrade path clear |
| **Subagent definitions** | Adequate | Current agents fit purpose; spawning mechanism not yet implemented |
| **Hook coverage** | Adequate | Core hooks implemented; advanced hooks (SubagentStop, SessionEnd) underutilized |

### 2.3 Architecture Gaps

| Gap | Severity | Impact | Priority |
|---|---|---|---|
| Inter-agent validation | High | Task failures, incorrect handoffs | P1 |
| Observability depth | High | Cannot optimize what isn't measured | P1 |
| Memory hardening | Medium | Data integrity risks | P2 |
| Decision capture | Medium | Blocks evolution features | P2 |
| Schema validation | Low | Potential state corruption | P3 |

---

## 3. Priority 1: Inter-Agent Validation

### 3.1 Gap Description

**Current state:** Agents hand off work via state files, but there is no structured validation that: - The receiving agent understands the handoff correctly - The sending agent completed its task fully - Required artifacts exist and are valid - Success criteria are met before proceeding

**Risk exposure:** The MAST taxonomy identifies inter-agent misalignment as responsible for **35% of multi-agent system failures**. Without validation checkpoints, errors propagate through the pipeline undetected.

### 3.2 Failure Modes

| Failure Mode | Description | Current Detection |
|---|---|---|
| Incomplete handoff | Agent A writes partial state, Agent B proceeds | None |
| Schema mismatch | Agent B expects fields Agent A didn't provide | Runtime error (if at all) |
| Stale state | Agent B reads outdated state | None |

| | | |
|---|---|---|
| | file | |
| Success criteria drift | Agent A's definition of "done" differs from workflow's | None |
| Artifact missing | Referenced files don't exist | File not found error |

### 3.3 Recommended Enhancements

**Enhancement 1: SubagentStop Hook Validation**

```bash
#!/bin/bash
# hooks/SubagentStop.sh

AGENT_NAME="$1"
STATE_FILE="$HOME/.claude/tmp/handoff.json"

# Validate handoff file exists and is valid JSON
if ! jq empty "$STATE_FILE" 2>/dev/null; then
    echo "ERROR: Invalid or missing handoff state" >&2
    exit 2  # Block - validation failed
fi

# Validate required fields present
REQUIRED_FIELDS=("task_summary" "files_in_scope" "success_criteria")
for field in "${REQUIRED_FIELDS[@]}"; do
    if ! jq -e ".context.$field" "$STATE_FILE" >/dev/null 2>&1; then
        echo "ERROR: Missing required field: $field" >&2
        exit 2
    fi
done

# Validate referenced artifacts exist
ARTIFACTS=$(jq -r '.artifacts | to_entries[] | .value' "$STATE_FILE"
2>/dev/null)
for artifact in $ARTIFACTS; do
    if [[ ! -f "$artifact" ]]; then
        echo "ERROR: Referenced artifact missing: $artifact" >&2
        exit 2
    fi
done

# Validate completion status
STATUS=$(jq -r '.status' "$STATE_FILE")
if [[ "$STATUS" != "completed" ]]; then
    echo "WARNING: Handoff status is '$STATUS', not 'completed'" >&2
    # Continue but log warning
fi

exit 0  # Validation passed
```

**Enhancement 2: Schema Validation on State Files**

```python
# scripts/validate_state.py
from pydantic import BaseModel, validator
from typing import List, Optional
import json

class ScopeMetrics(BaseModel):
    total_files: int
    estimated_tokens: int

class ScoutReport(BaseModel):
    scope_metrics: ScopeMetrics
    complexity_signals: dict

class ScoutMetrics(BaseModel):
    schema_version: str
    generated_at: str
    scout_report: ScoutReport

def validate_scout_metrics(file_path: str) -> bool:
    with open(file_path) as f:
        data = json.load(f)
    ScoutMetrics(**data)  # Raises ValidationError if invalid
    return True
```

**Enhancement 3: Handoff Protocol Checksums**

```jsonc
{
  "handoff_id": "uuid",
  "integrity": {
    "state_checksum": "sha256:abc123...",
    "artifacts_checksums": {
      "specs.md": "sha256:def456...",
      "scout_metrics.json": "sha256:789ghi..."
    }
  }
}
```

### 3.4 Implementation Effort

| Task | Effort | Dependencies |
|---|---|---|
| SubagentStop hook validation | 2-3 hours | None |

| | | |
|---|---|---|
| Pydantic schemas for state files | 4-6 hours | None |
| Checksum generation/verification | 2-3 hours | None |
| Integration testing | 4-6 hours | Above tasks |
| **Total** | **12-18 hours** | |

---

# 4. Priority 2: Observability Depth

## 4.1 Gap Description

**Current state:** Basic logging exists, but there is no structured observability for: - Cost tracking per tier, per session, per task type - Routing decision audit trail - Performance metrics (latency, token usage) - Inter-agent communication patterns - Memory retrieval effectiveness

**Risk exposure:** Without observability, optimization is guesswork. Cost overruns go undetected. Routing threshold tuning lacks data support.

## 4.2 Current vs. Target Observability

| Metric | Current State | Target State |
|---|---|---|
| Cost per session | Unknown | Tracked, visualized |
| Cost per tier | Unknown | Tracked, compared to baseline |
| Routing decisions | Partially logged | Fully logged with reasoning |
| Token usage | Not tracked | Tracked per tool call |
| Latency | Not tracked | P50, P95, P99 tracked |
| Memory retrieval recall | Not measured | Measured against ground truth |
| Human override rate | Not tracked | Tracked per decision category |

## 4.3 Recommended Enhancements

### Enhancement 1: OpenTelemetry Integration

Claude Code supports native OpenTelemetry tracing via environment variable:

```
export CLAUDE_CODE_ENABLE_TELEMETRY=1
```

Captured metrics include: - `claude_code.session.count` — CLI sessions - `claude_code.lines_of_code.count` — Lines modified - Token usage per API call - Latency per operation

### Enhancement 2: Routing Decision Log

Enhance `routing_log.jsonl` to capture full decision context:

```
{
  "timestamp": "2026-01-13T10:23:45Z",
  "session_id": "abc123",
  "decision_id": "uuid",

  "input": {
    "task_description": "Refactor authentication",
    "estimated_tokens": 35000,
    "files_in_scope": 12,
    "complexity_score": 8.5
  },

  "routing": {
    "calculated_tier": "sonnet",
    "requested_tier": "opus",
    "final_tier": "sonnet",
    "decision": "BLOCK",
    "reason": "requested_tier exceeds calculated ceiling"
  },

  "cost": {
    "estimated_input_tokens": 35000,
    "estimated_output_tokens": 5000,
    "estimated_cost_usd": 0.12
  },

  "outcome": {
    "actual_input_tokens": null,
    "actual_output_tokens": null,
    "actual_cost_usd": null,
    "task_success": null,
    "filled_by_post_hook": true
  }
}
```

### Enhancement 3: Cost Tracking Dashboard Data

```
# scripts/generate-cost-report.sh
#!/bin/bash

# Aggregate routing log for cost analysis
jq -s '
  group_by(.routing.final_tier) |
  map({
    tier: .[0].routing.final_tier,
    count: length,
    total_cost: (map(.outcome.actual_cost_usd // 0) | add),
    avg_cost: (map(.outcome.actual_cost_usd // 0) | add / length)
  })
' ~/.claude/tmp/routing_log.jsonl
```

**Enhancement 4: Memory Retrieval Metrics**

```
{
  "query": "authentication JWT refresh",
  "timestamp": "2026-01-13T10:30:00Z",
  "results_returned": 5,
  "results_used_by_agent": 2,
  "user_feedback": null,
  "retrieval_method": "bm25",
  "latency_ms": 45
}
```

### 4.4 Implementation Effort

| Task | Effort | Dependencies |
|---|---|---|
| Enable OpenTelemetry | 1 hour | None |
| Enhanced routing log schema | 2-3 hours | None |
| PostToolUse outcome capture | 3-4 hours | Routing log |
| Cost aggregation script | 2-3 hours | Routing log |
| Memory retrieval metrics | 2-3 hours | query-memory.sh |
| **Total** | **10-14 hours** | |

---

## 5. Priority 3: Memory Hardening

### 5.1 Gap Description

**Current state:** Memory files are human-readable YAML/markdown with no integrity verification. No access controls. No audit logging for reads/writes.

**Risk exposure:** MAST taxonomy identifies memory poisoning as a failure mode. Corrupted or manipulated memory files could degrade system behavior over time.

### 5.2 Risk Assessment

| Risk | Likelihood | Impact | Mitigation Priority |
|---|---|---|---|
| Accidental corruption | Medium | Medium | P2 |
| Malicious manipulation | Low | High | P3 |
| Stale data retrieval | Medium | Low | P2 |
| Inconsistent state | Low | Medium | P3 |

### 5.3 Recommended Enhancements

**Enhancement 1: File Integrity Checksums**

```
# Memory file with integrity header
---
title: "Decision: JWT Token Strategy"
created: 2026-01-13
integrity:
  content_hash: "sha256:abc123..."
  last_verified: 2026-01-13T10:00:00Z
---
# Content follows...
```

**Enhancement 2: Audit Logging**

```
// .claude/memory/audit.jsonl
{
  "timestamp": "2026-01-13T10:30:00Z",
  "operation": "read",
  "file": "decisions/jwt-strategy.md",
  "agent": "architect",
  "session_id": "abc123"
}
{
  "timestamp": "2026-01-13T10:35:00Z",
  "operation": "write",
  "file": "decisions/jwt-strategy.md",
  "agent": "memory-archivist",
  "session_id": "abc123",
  "change_type": "update",
```

```
  "previous_hash": "sha256:abc123...",
  "new_hash": "sha256:def456..."
}
```

**Enhancement 3: Periodic Integrity Verification**

```
# scripts/verify-memory-integrity.sh
#!/bin/bash

MEMORY_DIR="$HOME/.claude/memory"
ERRORS=0

find "$MEMORY_DIR" -name "*.md" | while read -r file; do
    # Extract stored hash from frontmatter
    stored_hash=$(grep "content_hash:" "$file" | awk '{print $2}')

    # Calculate actual hash (excluding frontmatter)
    actual_hash=$(sed '1,/^---$/d' "$file" | sha256sum | awk '{print
"sha256:" $1}')

    if [[ "$stored_hash" != "$actual_hash" ]]; then
        echo "INTEGRITY FAILURE: $file"
        ERRORS=$((ERRORS + 1))
    fi
done

exit $ERRORS
```

### 5.4 Implementation Effort

| Task | Effort | Dependencies |
|---|---|---|
| Add integrity headers to memory files | 2-3 hours | None |
| Implement audit logging | 3-4 hours | None |
| Verification script | 2-3 hours | Integrity headers |
| Hook integration for audit | 2-3 hours | Audit logging |
| **Total** | **9-13 hours** | |

---

## 6. Framework Comparison Summary

### 6.1 Evaluation Results

Based on comprehensive analysis of LangChain/LangGraph vs. the bespoke architecture:

| Dimension | LangChain/LangGraph | Lisan al-Gaib | Winner |
|---|---|---|---|
| Routing control | LLM-based, configurable | Code-enforced, deterministic | Lisan |
| State transparency | PostgreSQL opaque | File-based, human-readable | Lisan |
| Infrastructure requirements | PostgreSQL + Redis | None (files only) | Lisan |
| Latency overhead | ~14ms per graph | <1ms file I/O | Lisan |
| Token overhead | 15-50% documented | Zero framework overhead | Lisan |
| Human-in-the-loop | Sophisticated interrupts | File-based, manual | LangGrap |
| Observability tooling | LangSmith (commercial) | DIY (flexible) | Tie |
| Agentic RAG patterns | Native support | Requires implementation | LangGrap |
| Learning curve | Steep (framework concepts) | Moderate (shell/files) | Lisan |
| Lock-in risk | High (API instability) | Zero | Lisan |

### 6.2 Selective Adoption Recommendations

**Adopt:** - `langchain-anthropic` adapter for cleaner model provider abstraction - LlamaIndex for advanced retrieval (if/when vector embeddings added) - LangSmith for observability (optional, via environment variable)

**Avoid:** - Full LangGraph adoption (infrastructure conflicts) - LangChain memory modules (token-inefficient) - Complex chain nesting (debugging nightmare)

### 6.3 Migration Risk Assessment

| Migration Path | Risk | Recommendation |
|---|---|---|
| | | Avoid — conflicts with file- |

| | | |
|---|---|---|
| Adopt full LangGraph | High | based philosophy |
| Adopt LangChain adapters | Low | Consider — clean model interface |
| Adopt LangSmith | Low | Consider — observability value |
| Keep fully bespoke | Low | Current recommendation |

---

# 7. Anti-Pattern Risk Assessment

## 7.1 Known Anti-Patterns from Research

The MAST taxonomy and multi-agent systems literature identify these anti-patterns:

| Anti-Pattern | Risk to Lisan | Mitigation |
|---|---|---|
| **Tool overload** (>5 tools per agent) | Medium | Monitor tool count per agent; refactor if exceeded |
| **Circular dependencies** | Low | Explicit termination conditions in explore workflow |
| **Stateless reasoning** | Medium | Session state persisted; memory system provides continuity |
| **Excessive agency** | Low | Complexity-based routing limits autonomy |
| **Unbounded loops** | Low | Hook enforcement provides circuit breakers |
| **Context overflow** | Low | Gemini offload handles large contexts |

## 7.2 Lisan-Specific Risks

| Risk | Current Status | Monitoring |
|---|---|---|
| Subagent proliferation | Not yet applicable | Population cap when spawning implemented |
| Memory bloat | Low (manual curation) | File count alerts if >1000 |
| Routing threshold drift | Not monitored | Add threshold effectiveness tracking |
| Schema version conflicts | Low (single version) | Version validation when schemas activated |

## 7.3 Recommended Monitoring

```yaml
# monitoring-thresholds.yaml
alerts:
  - name: high_opus_usage
    condition: opus_invocations_per_session > 5
    action: review routing thresholds

  - name: memory_file_growth
    condition: memory_file_count > 500
    action: trigger memory curation

  - name: agent_count_limit
    condition: active_agents > 12
    action: block new agent spawning

  - name: session_cost_overrun
    condition: session_cost_usd > 10
    action: alert user, log for review
```

---

# 8. Gap Remediation Priority Matrix

## 8.1 Prioritized Enhancement List

| Priority | Gap | Effort | Impact | Dependencies |
|---|---|---|---|---|
| P1.1 | SubagentStop validation | 4h | High | None |
| P1.2 | State file schema validation | 6h | High | None |
| | OpenTelemetry | | | |

| | | | | |
|---|---|---|---|---|
| P1.3 | enablement | 1h | Medium | None |
| P1.4 | Enhanced routing log | 4h | High | None |
| P2.1 | Decision capture schema | 6h | High | P1.4 |
| P2.2 | Memory integrity headers | 3h | Medium | None |
| P2.3 | Audit logging | 4h | Medium | None |
| P2.4 | Cost aggregation reporting | 3h | Medium | P1.4 |
| P3.1 | Memory retrieval metrics | 3h | Low | None |
| P3.2 | Checksum verification | 3h | Low | P2.2 |

### 8.2 Implementation Sequence

```
Week 1:
├── P1.1 SubagentStop validation
├── P1.3 OpenTelemetry enablement
└── P1.4 Enhanced routing log

Week 2:
├── P1.2 State file schema validation
├── P2.2 Memory integrity headers
└── P2.4 Cost aggregation reporting

Week 3:
├── P2.1 Decision capture schema
├── P2.3 Audit logging
└── P3.1 Memory retrieval metrics

Week 4:
├── P3.2 Checksum verification
├── Integration testing
└── Documentation updates
```

### 8.3 Success Criteria

| Gap Category | Success Metric | Target |
|---|---|---|
| Inter-agent validation | Handoff failures detected | 100% |
| Observability | Cost tracked per session | 100% |
| Memory hardening | Integrity verified | Weekly |
| Schema validation | Invalid state prevented | 100% |

---

## Summary

The gap analysis reveals a fundamentally sound architecture with specific enhancement opportunities:

**Immediate priorities (Phase 0-1):** 1. Inter-agent validation via SubagentStop hooks 2. Observability through enhanced logging and OpenTelemetry 3. Schema validation for state files

**Near-term priorities (Phase 2):** 1. Decision capture schema for apprenticeship learning 2. Memory hardening with integrity verification 3. Cost tracking and reporting

**Framework comparison conclusion:** Maintain bespoke architecture; selectively adopt model adapters and observability tooling only.

The identified gaps, once addressed, will provide the foundation for implementing the novel innovations (emergent schema discovery, subagent spawning, apprenticeship learning) described in Part III.

---

*Continue to Part V: Implementation Roadmap* → # Part V: Implementation Roadmap

# 12-Month Development Plan for Self-Evolving Multi-Agent Architecture

---

## Roadmap Overview

```
┌──────────────────────────────────────────────────────────────────────────────
│                              12-MONTH IMPLEMENTATION ROADMAP
│
└──────────────────────────────────────────────────────────────────────────────


FOUNDATION                      CAPABILITY
EVOLUTION
──────────────────────────────────────────────────────────────────────────────
```

```
Weeks 1-2     Month 1       Month 2       Month 3       Months 4-5
Months 6-7
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
┌──────────┐
│ PHASE 0 │  │ PHASE 1 │  │ PHASE 2 │  │ PHASE 3 │  │  PHASE 4  │  │
PHASE 5  │
│          │  │          │  │          │  │          │  │           │  │
│
│ Founda- │→│ Observ- │→│ Memory  │→│ Weekly  │→│ Emergent  │→│
Subagent  │
│ tion     │  │ ability │  │ Enhance │  │ Review  │  │ Schema    │  │
Spawning  │
│          │  │          │  │          │  │          │  │           │  │
│
│ • Telem │  │ • Logs  │  │ • BM25  │  │ • Synth │  │ • Observe │  │
• Gap     │
│ • Valid │  │ • Costs │  │ • Capt  │  │ • Arch  │  │ • Detect  │  │
• Generate│
│ • Schema│  │ • Hooks │  │ • Front │  │ • HITL  │  │ • Propose │  │
• Shadow  │
└──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘
└──────────┘

                                                         Months 8-10
Months 11-12
                                                         ┌──────────┐
┌──────────┐
                                                         │  PHASE 6  │
│  PHASE 7  │
                                                         │           │
│           │
                                                         │ Autonomy  │→│
Self-     │
                                                         │ Progress  │
│ Improving │
                                                         │           │
│           │
                                                         │ • Track   │
│ • Auto    │
                                                         │ • Promote │
│ • Deprec  │
                                                         │ • Audit   │
│ • Loop    │
                                                         └──────────┘
└──────────┘
```

### Phase Summary

| Phase | Timeline | Focus | Key Deliverable |
|---|---|---|---|
| 0 | Weeks 1-2 | Foundation | Telemetry, validation, schemas |
| 1 | Month 1 | Observability | Full logging, cost tracking |
| 2 | Month 2 | Memory | BM25 retrieval, decision capture |
| 3 | Month 3 | Weekly Review | Automated analysis pipeline |
| 4 | Months 4-5 | Schema Discovery | Pattern detection, schema proposal |
| 5 | Months 6-7 | Subagent Spawning | Agent generation, shadow deployment |
| 6 | Months 8-10 | Autonomy | Level progression, earned trust |
| 7 | Months 11-12 | Self-Improvement | Continuous evolution loop |

---

## Phase 0: Foundation (Weeks 1-2)

### Objectives

1. Enable telemetry for baseline metrics
2. Implement inter-agent validation
3. Establish schema validation on state files
4. Create foundation for all subsequent phases

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Telemetry enablement | `~/.profile` or `~/.zshrc` | Environment variable configuration |
| SubagentStop hook | `.claude/hooks/SubagentStop.sh` | Validation of agent handoffs |
| State schemas | `.claude/schemas/state/` | Pydantic models for state files |
| Schema validator | `.claude/scripts/validate-state.py` | Validation script |

| | | |
|---|---|---|
| Updated settings | `.claude/settings.json` | Telemetry and validation flags |

## Implementation Steps

### Step 0.1: Enable OpenTelemetry (1 hour)

```
# Add to ~/.profile or ~/.zshrc
export CLAUDE_CODE_ENABLE_TELEMETRY=1

# Verify telemetry is active
claude --version  # Should show telemetry indicator
```

### Step 0.2: Implement SubagentStop Validation (4 hours)

```
# .claude/hooks/SubagentStop.sh
#!/bin/bash
set -euo pipefail

AGENT_NAME="${1:-unknown}"
STATE_FILE="$HOME/.claude/tmp/handoff.json"
LOG_FILE="$HOME/.claude/tmp/validation_log.jsonl"

log_validation() {
    local status="$1"
    local message="$2"
    echo "{\"timestamp\":\"$(date -
Iseconds)\",\"agent\":\"$AGENT_NAME\",\"status\":\"$status\",\"message\":\"$message\"}"
 >> "$LOG_FILE"
}

# Check handoff file exists
if [[ ! -f "$STATE_FILE" ]]; then
    log_validation "WARN" "No handoff file found"
    exit 0  # Don't block if no handoff expected
fi

# Validate JSON syntax
if ! jq empty "$STATE_FILE" 2>/dev/null; then
    log_validation "ERROR" "Invalid JSON in handoff file"
    exit 2  # Block - invalid state
fi

# Validate required fields
REQUIRED=("context.task_summary" "context.success_criteria"
"status")
for field in "${REQUIRED[@]}"; do
    if ! jq -e ".$field" "$STATE_FILE" >/dev/null 2>&1; then
        log_validation "ERROR" "Missing required field: $field"
        exit 2
    fi
done

# Validate referenced artifacts exist
if jq -e '.artifacts' "$STATE_FILE" >/dev/null 2>&1; then
    while IFS= read -r artifact; do
        if [[ -n "$artifact" && ! -f "$artifact" ]]; then
            log_validation "ERROR" "Missing artifact: $artifact"
            exit 2
        fi
    done < <(jq -r '.artifacts | to_entries[] | .value // empty'
"$STATE_FILE")
fi

log_validation "OK" "Validation passed"
exit 0

# Make executable
chmod +x ~/.claude/hooks/SubagentStop.sh
```

### Step 0.3: Create State File Schemas (4 hours)

```
# .claude/schemas/state/scout_metrics.py
from pydantic import BaseModel, Field
from typing import Dict, List, Optional
from datetime import datetime

class ScopeMetrics(BaseModel):
    total_files: int = Field(ge=0)
    estimated_tokens: int = Field(ge=0)
    file_types: Optional[Dict[str, int]] = None
    largest_file: Optional[Dict[str, any]] = None

class ComplexitySignals(BaseModel):
    cross_file_dependencies: int = Field(ge=0, default=0)
    module_count: int = Field(ge=1, default=1)
    circular_imports: int = Field(ge=0, default=0)

class ScoutReport(BaseModel):
    scope_metrics: ScopeMetrics
    complexity_signals: ComplexitySignals
    recommendations: Optional[Dict[str, any]] = None

class ScoutMetrics(BaseModel):
    schema_version: str = "1.0.0"
```

```python
    generated_at: datetime
    scout_agent: str = "haiku"
    scout_report: ScoutReport

# .claude/schemas/state/handoff.py
from pydantic import BaseModel, Field
from typing import Dict, List, Optional
from datetime import datetime
from enum import Enum

class HandoffStatus(str, Enum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"

class HandoffContext(BaseModel):
    task_summary: str
    files_in_scope: List[str] = []
    critical_constraints: List[str] = []
    success_criteria: List[str]

class HandoffMetadata(BaseModel):
    estimated_tokens: int = Field(ge=0)
    estimated_duration_minutes: int = Field(ge=0)
    tier_ceiling: str

class Handoff(BaseModel):
    schema_version: str = "1.0.0"
    handoff_id: str
    from_agent: str
    to_agent: str
    created_at: datetime
    status: HandoffStatus
    context: HandoffContext
    artifacts: Dict[str, str] = {}
    metadata: HandoffMetadata
```

### Step 0.4: Create Schema Validator Script (2 hours)

```python
#!/usr/bin/env python3
# .claude/scripts/validate-state.py
"""Validate state files against schemas."""

import sys
import json
import importlib.util
from pathlib import Path

SCHEMA_DIR = Path.home() / ".claude" / "schemas" / "state"
STATE_DIR = Path.home() / ".claude" / "tmp"

SCHEMA_MAP = {
    "scout_metrics.json": "scout_metrics.ScoutMetrics",
    "handoff.json": "handoff.Handoff",
}

def load_schema_class(module_path: str):
    """Dynamically load a Pydantic model from schema directory."""
    module_name, class_name = module_path.rsplit(".", 1)
    spec = importlib.util.spec_from_file_location(
        module_name,
        SCHEMA_DIR / f"{module_name}.py"
    )
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return getattr(module, class_name)

def validate_file(file_path: Path) -> tuple[bool, str]:
    """Validate a single state file."""
    if file_path.name not in SCHEMA_MAP:
        return True, f"No schema defined for {file_path.name}"

    try:
        with open(file_path) as f:
            data = json.load(f)

        schema_class = load_schema_class(SCHEMA_MAP[file_path.name])
        schema_class(**data)
        return True, "Valid"

    except json.JSONDecodeError as e:
        return False, f"Invalid JSON: {e}"
    except Exception as e:
        return False, f"Validation failed: {e}"

def main():
    if len(sys.argv) > 1:
        # Validate specific file
        file_path = Path(sys.argv[1])
        valid, message = validate_file(file_path)
        print(f"{file_path.name}: {message}")
        sys.exit(0 if valid else 1)
    else:
        # Validate all known state files
        errors = 0
        for filename in SCHEMA_MAP:
```

```
            file_path = STATE_DIR / filename
            if file_path.exists():
                valid, message = validate_file(file_path)
                status = "✓" if valid else "✗"
                print(f"{status} {filename}: {message}")
                if not valid:
                    errors += 1
        sys.exit(errors)

if __name__ == "__main__":
    main()
```

### Verification Commands

```
# Verify telemetry enabled
env | grep CLAUDE_CODE_ENABLE_TELEMETRY

# Test SubagentStop hook
echo '{"status":"completed","context":
{"task_summary":"test","success_criteria":["done"]}}' >
~/.claude/tmp/handoff.json
~/.claude/hooks/SubagentStop.sh test_agent
echo "Exit code: $?"  # Should be 0

# Test schema validation
python3 ~/.claude/scripts/validate-state.py
~/.claude/tmp/handoff.json
```

### Success Criteria

| Criterion | Verification |
|---|---|
| Telemetry active | `env \| grep TELEMETRY` returns value |
| SubagentStop validates | Valid handoff returns exit 0 |
| SubagentStop blocks invalid | Missing fields returns exit 2 |
| Schema validator works | Validates known state files |

### Estimated Effort: 12-15 hours

---

## Phase 1: Observability & Validation (Month 1)

### Objectives

1. Complete hook coverage for all lifecycle events
2. Implement comprehensive routing decision logging
3. Create cost tracking per session and tier
4. Establish baseline metrics for optimization

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Enhanced PreToolUse | `.claude/hooks/PreToolUse.sh` | Full routing logging |
| Enhanced PostToolUse | `.claude/hooks/PostToolUse.sh` | Outcome capture |
| SessionEnd hook | `.claude/hooks/SessionEnd.sh` | Session summary |
| Routing log schema | `.claude/schemas/routing_log.json` | Log format definition |
| Cost report script | `.claude/scripts/generate-cost-report.sh` | Cost aggregation |
| Metrics dashboard data | `.claude/tmp/metrics/` | Aggregated metrics |

### Implementation Steps

#### Step 1.1: Enhanced Routing Log (4 hours)

```
# .claude/hooks/PreToolUse.sh (enhanced)
#!/bin/bash
set -euo pipefail

TOOL_NAME="${CLAUDE_TOOL_NAME:-unknown}"
SESSION_ID="${CLAUDE_SESSION_ID:-$(date +%s)}"
LOG_FILE="$HOME/.claude/tmp/routing_log.jsonl"
METRICS_FILE="$HOME/.claude/tmp/scout_metrics.json"
SCORE_FILE="$HOME/.claude/tmp/complexity_score"
TIER_FILE="$HOME/.claude/tmp/recommended_tier"

# Read current state
COMPLEXITY_SCORE=$(cat "$SCORE_FILE" 2>/dev/null || echo "0")
RECOMMENDED_TIER=$(cat "$TIER_FILE" 2>/dev/null || echo "sonnet")

# Determine requested tier from tool (simplified - actual
implementation varies)
REQUESTED_TIER="${CLAUDE_REQUESTED_TIER:-sonnet}"
```

```
        # Make routing decision
        if [[ "$REQUESTED_TIER" == "opus" && "$RECOMMENDED_TIER" != "opus"
]]; then
            DECISION="BLOCK"
            REASON="requested opus but ceiling is $RECOMMENDED_TIER"
            EXIT_CODE=2
        else
            DECISION="PERMIT"
            REASON="within tier ceiling"
            EXIT_CODE=0
        fi

        # Estimate cost (simplified)
        case "$RECOMMENDED_TIER" in
            haiku)  COST_PER_1K=0.00025 ;;
            sonnet) COST_PER_1K=0.003 ;;
            opus)   COST_PER_1K=0.015 ;;
            *)      COST_PER_1K=0.003 ;;
        esac

        ESTIMATED_TOKENS=$(jq -r
'.scout_report.scope_metrics.estimated_tokens // 10000'
"$METRICS_FILE" 2>/dev/null || echo "10000")
        ESTIMATED_COST=$(echo "scale=4; $ESTIMATED_TOKENS / 1000 *
$COST_PER_1K" | bc)

        # Log decision
        cat >> "$LOG_FILE" << EOF
        {"timestamp":"$(date -
Iseconds)","session_id":"$SESSION_ID","tool_name":"$TOOL_NAME","routing":
{"complexity_score":$COMPLEXITY_SCORE,"calculated_tier":"$RECOMMENDED_TIER","requested_tier":"$REQUESTE
{"estimated_tokens":$ESTIMATED_TOKENS,"estimated_cost_usd":$ESTIMATED_COST},"outcome":
{"actual_tokens":null,"actual_cost_usd":null,"task_success":null}}
        EOF

        exit $EXIT_CODE
```

**Step 1.2: PostToolUse Outcome Capture (4 hours)**

```
        # .claude/hooks/PostToolUse.sh
        #!/bin/bash
        set -euo pipefail

        TOOL_NAME="${CLAUDE_TOOL_NAME:-unknown}"
        EXIT_CODE="${CLAUDE_TOOL_EXIT_CODE:-0}"
        SESSION_ID="${CLAUDE_SESSION_ID:-unknown}"
        LOG_FILE="$HOME/.claude/tmp/routing_log.jsonl"

        # Determine success
        if [[ "$EXIT_CODE" == "0" ]]; then
            SUCCESS="true"
        else
            SUCCESS="false"
        fi

        # Update the last log entry with outcome
        # (In practice, would use a proper log ID; this is simplified)
        TEMP_FILE=$(mktemp)
        head -n -1 "$LOG_FILE" > "$TEMP_FILE" 2>/dev/null || true

        # Get last entry, update outcome
        LAST_ENTRY=$(tail -1 "$LOG_FILE")
        UPDATED_ENTRY=$(echo "$LAST_ENTRY" | jq --arg success "$SUCCESS"
'.outcome.task_success = ($success == "true")')

        cat "$TEMP_FILE" > "$LOG_FILE"
        echo "$UPDATED_ENTRY" >> "$LOG_FILE"
        rm "$TEMP_FILE"

        exit 0
```

**Step 1.3: SessionEnd Hook (3 hours)**

```
        # .claude/hooks/SessionEnd.sh
        #!/bin/bash
        set -euo pipefail

        SESSION_ID="${CLAUDE_SESSION_ID:-$(date +%s)}"
        LOG_FILE="$HOME/.claude/tmp/routing_log.jsonl"
        SUMMARY_DIR="$HOME/.claude/tmp/session_summaries"
        mkdir -p "$SUMMARY_DIR"

        # Generate session summary
        SUMMARY_FILE="$SUMMARY_DIR/${SESSION_ID}.json"

        # Aggregate session metrics
        jq -s --arg sid "$SESSION_ID" '
        {
            session_id: $sid,
            timestamp: (now | todate),
            metrics: {
                total_tool_calls: length,
                by_tier: (group_by(.routing.final_tier) | map({(.
[0].routing.final_tier): length}) | add),
                total_estimated_cost: (map(.cost.estimated_cost_usd // 0) |
```

```
add),
                success_rate: ((map(select(.outcome.task_success == true)) |
length) / (length + 0.001)),
                blocked_requests: (map(select(.routing.decision == "BLOCK"))
| length)
            }
        }' "$LOG_FILE" > "$SUMMARY_FILE"

        # Archive routing log for this session
        mv "$LOG_FILE" "$HOME/.claude/tmp/routing_log_${SESSION_ID}.jsonl"

        echo "Session summary: $SUMMARY_FILE"
```

**Step 1.4: Cost Report Generator (3 hours)**

```
        #!/bin/bash
        # .claude/scripts/generate-cost-report.sh

        SUMMARY_DIR="$HOME/.claude/tmp/session_summaries"
        OUTPUT_FILE="$HOME/.claude/tmp/metrics/cost_report.json"
        mkdir -p "$(dirname "$OUTPUT_FILE")"

        # Aggregate all session summaries
        find "$SUMMARY_DIR" -name "*.json" -mtime -7 | \
        xargs cat 2>/dev/null | \
        jq -s '
        {
            report_date: (now | todate),
            period: "last_7_days",
            sessions: length,
            total_cost_usd: (map(.metrics.total_estimated_cost // 0) | add),
            avg_cost_per_session: ((map(.metrics.total_estimated_cost // 0)
| add) / (length + 0.001)),
            by_tier: (
                map(.metrics.by_tier // {}) |
                add |
                to_entries |
                map({(.key): .value}) |
                add
            ),
            overall_success_rate: ((map(.metrics.success_rate // 0) | add) /
(length + 0.001)),
            total_blocked: (map(.metrics.blocked_requests // 0) | add)
        }' > "$OUTPUT_FILE"

        echo "Cost report generated: $OUTPUT_FILE"
        cat "$OUTPUT_FILE" | jq .
```

### Verification Commands

```
        # Test full hook chain
        echo '{"scout_report":{"scope_metrics":
{"total_files":10,"estimated_tokens":25000},"complexity_signals":
{"module_count":2}}}' > ~/.claude/tmp/scout_metrics.json
        ~/.claude/scripts/calculate-complexity.sh
        ~/.claude/hooks/PreToolUse.sh  # Should permit
        cat ~/.claude/tmp/routing_log.jsonl | jq .

        # Generate cost report
        ~/.claude/scripts/generate-cost-report.sh
```

### Success Criteria

| Criterion | Verification |
|---|---|
| All hooks execute | No errors on tool invocation |
| Routing logged | `routing_log.jsonl` contains entries |
| Cost tracked | Cost report shows tier breakdown |
| Session summaries | Summary files created per session |

### Estimated Effort: 20-25 hours

---

## Phase 2: Memory Enhancement (Month 2)

### Objectives

1. Implement BM25-based memory retrieval
2. Deploy decision capture schema
3. Standardize YAML frontmatter across memory files
4. Enable structured pattern accumulation

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| BM25 retriever | `.claude/scripts/query-memory-bm25.py` | Improved retrieval |
| Decision capture | `.claude/schemas/decision.py` | Decision schema |
| Frontmatter standard | `.claude/docs/memory-format.md` | Format specification |

| | | |
|---|---|---|
| Observation logger | `.claude/scripts/log-observation.py` | Raw event capture |
| Memory index | `.claude/memory/index.json` | Searchable index |

## Implementation Steps

### Step 2.1: BM25 Memory Retrieval (6 hours)

```python
#!/usr/bin/env python3
# .claude/scripts/query-memory-bm25.py
"""BM25-based memory retrieval."""

import sys
import json
import re
from pathlib import Path
from typing import List, Dict, Tuple
from dataclasses import dataclass

# Install: pip install rank-bm25 pyyaml
from rank_bm25 import BM25Okapi
import yaml

MEMORY_DIR = Path.home() / ".claude" / "memory"
INDEX_FILE = MEMORY_DIR / "index.json"

@dataclass
class MemoryDocument:
    path: Path
    title: str
    content: str
    category: str
    tags: List[str]
    created: str

def parse_frontmatter(content: str) -> Tuple[Dict, str]:
    """Extract YAML frontmatter and body from markdown."""
    if not content.startswith("---"):
        return {}, content

    parts = content.split("---", 2)
    if len(parts) < 3:
        return {}, content

    try:
        frontmatter = yaml.safe_load(parts[1])
        body = parts[2].strip()
        return frontmatter or {}, body
    except yaml.YAMLError:
        return {}, content

def load_memory_documents() -> List[MemoryDocument]:
    """Load all memory documents."""
    documents = []

    for md_file in MEMORY_DIR.rglob("*.md"):
        if md_file.name.startswith("."):
            continue

        content = md_file.read_text()
        frontmatter, body = parse_frontmatter(content)

        doc = MemoryDocument(
            path=md_file,
            title=frontmatter.get("title", md_file.stem),
            content=body,
            category=frontmatter.get("category", "uncategorized"),
            tags=frontmatter.get("tags", []),
            created=frontmatter.get("created", "")
        )
        documents.append(doc)

    return documents

def tokenize(text: str) -> List[str]:
    """Simple tokenization."""
    return re.findall(r'\w+', text.lower())

def search(query: str, top_k: int = 5, category: str = None) ->
List[Dict]:
    """Search memory using BM25."""
    documents = load_memory_documents()

    # Filter by category if specified
    if category:
        documents = [d for d in documents if d.category == category]

    if not documents:
        return []

    # Build corpus
    corpus = [tokenize(f"{d.title} {d.content} {' '.join(d.tags)}")
for d in documents]

    # Create BM25 index
```

```
            bm25 = BM25Okapi(corpus)

            # Search
            query_tokens = tokenize(query)
            scores = bm25.get_scores(query_tokens)

            # Rank results
            ranked = sorted(zip(documents, scores), key=lambda x: x[1],
reverse=True)

            results = []
            for doc, score in ranked[:top_k]:
                if score > 0:
                    results.append({
                        "path": str(doc.path.relative_to(MEMORY_DIR)),
                        "title": doc.title,
                        "category": doc.category,
                        "tags": doc.tags,
                        "score": round(score, 3),
                        "preview": doc.content[:200] + "..." if
len(doc.content) > 200 else doc.content
                    })

            return results

        def main():
            import argparse
            parser = argparse.ArgumentParser(description="Search memory with
BM25")
            parser.add_argument("query", help="Search query")
            parser.add_argument("-k", "--top-k", type=int, default=5,
help="Number of results")
            parser.add_argument("-c", "--category", help="Filter by
category")
            parser.add_argument("--json", action="store_true", help="Output
as JSON")

            args = parser.parse_args()
            results = search(args.query, args.top_k, args.category)

            if args.json:
                print(json.dumps(results, indent=2))
            else:
                for i, r in enumerate(results, 1):
                    print(f"\n{i}. [{r['score']}] {r['title']}")
                    print(f"   Category: {r['category']} | Tags: {',
'.join(r['tags'])}")
                    print(f"   Path: {r['path']}")
                    print(f"   {r['preview']}")

        if __name__ == "__main__":
            main()
```

### Step 2.2: Decision Capture Schema (4 hours)

```
        # .claude/schemas/decision.py
        from pydantic import BaseModel, Field
        from typing import List, Dict, Optional, Any
        from datetime import datetime
        from enum import Enum

        class DecisionCategory(str, Enum):
            ROUTING = "routing_override"
            SCOPE = "scope_modification"
            APPROVAL = "plan_approval"
            REJECTION = "plan_rejection"
            ESCALATION = "human_escalation"
            DELEGATION = "task_delegation"

        class SystemRecommendation(BaseModel):
            action: str
            confidence: float = Field(ge=0, le=1)
            reasoning: str

        class HumanDecision(BaseModel):
            action: str
            reasoning_provided: Optional[str] = None
            time_to_decide_seconds: Optional[int] = None

        class DecisionOutcome(BaseModel):
            task_success: Optional[bool] = None
            quality_rating: Optional[int] = Field(None, ge=1, le=5)
            issues_encountered: List[str] = []
            would_recommend_same: Optional[bool] = None

        class LearningMetadata(BaseModel):
            autonomy_level_at_time: int = Field(ge=1, le=5)
            pattern_match_candidates: List[str] = []
            should_automate_similar: Optional[bool] = None
            requires_human_always: bool = False

        class Decision(BaseModel):
            """Schema for capturing human decisions for apprenticeship
learning."""

            schema_version: str = "1.0.0"
            decision_id: str
```

```
    timestamp: datetime
    session_id: str

    decision_category: DecisionCategory

    context: Dict[str, Any] = Field(
        description="Task context at decision time"
    )

    system_recommendation: Optional[SystemRecommendation] = None
    human_decision: HumanDecision
    outcome: DecisionOutcome = DecisionOutcome()
    learning_metadata: LearningMetadata
```

**Step 2.3: Observation Logger (3 hours)**

```
#!/usr/bin/env python3
# .claude/scripts/log-observation.py
"""Log raw behavioral observations for pattern discovery."""

import json
import sys
from datetime import datetime
from pathlib import Path
from typing import Dict, Any

OBSERVATIONS_DIR = Path.home() / ".claude" / "memory" /
"observations"

def log_observation(
    event_type: str,
    context: Dict[str, Any],
    action: Dict[str, Any],
    outcome: Dict[str, Any] = None
) -> str:
    """Log a single observation event."""

    OBSERVATIONS_DIR.mkdir(parents=True, exist_ok=True)

    today = datetime.now().strftime("%Y-%m-%d")
    log_file = OBSERVATIONS_DIR / f"{today}-observations.jsonl"

    observation = {
        "timestamp": datetime.now().isoformat(),
        "event_type": event_type,
        "context": context,
        "action": action,
        "outcome": outcome or {}
    }

    with open(log_file, "a") as f:
        f.write(json.dumps(observation) + "\n")

    return str(log_file)

def main():
    """CLI interface for logging observations."""
    import argparse

    parser = argparse.ArgumentParser(description="Log behavioral
observation")
    parser.add_argument("event_type", help="Type of event")
    parser.add_argument("--context", type=json.loads, default={},
help="Context JSON")
    parser.add_argument("--action", type=json.loads, default={},
help="Action JSON")
    parser.add_argument("--outcome", type=json.loads, default={},
help="Outcome JSON")

    args = parser.parse_args()

    log_file = log_observation(
        args.event_type,
        args.context,
        args.action,
        args.outcome
    )

    print(f"Logged to: {log_file}")

if __name__ == "__main__":
    main()
```

**Step 2.4: Memory Format Standard (2 hours)**

```
# .claude/docs/memory-format.md
# Memory File Format Standard

All memory files in `.claude/memory/` MUST follow this format.

## Required YAML Frontmatter

```yaml
---
title: "Brief descriptive title"
created: YYYY-MM-DD
```
```

```yaml
category: decisions|sharp-edges|facts|preferences
tags: [tag1, tag2, tag3]
status: active|deprecated|archived
summary: "One-line searchable summary for BM25 retrieval"
---
```

## Optional Frontmatter Fields

```yaml
updated: YYYY-MM-DD
related: [./other-file.md, ../category/file.md]
confidence: high|medium|low
source: session-id or "manual"
expires: YYYY-MM-DD  # For time-sensitive facts
```

## Body Format

Use standard markdown. Structure recommendations:

### For Decisions

- Context (why decision was needed)
- Decision (what was decided)
- Rationale (why this choice)
- Consequences (what follows from this)

### For Sharp Edges

- Problem (what went wrong)
- Symptoms (how it manifested)
- Solution (how to fix/avoid)
- Prevention (how to prevent recurrence)

### For Facts

- Statement (the fact)
- Evidence (how we know)
- Scope (when this applies)

### For Preferences

- Preference (what is preferred)
- Context (when this applies)
- Rationale (why this preference)

### Verification Commands

```bash
# Test BM25 search
python3 ~/.claude/scripts/query-memory-bm25.py "authentication JWT"
--json

# Log test observation
python3 ~/.claude/scripts/log-observation.py "test_event" \
    --context '{"task":"test"}' \
    --action '{"type":"test"}'

# Verify observation logged
cat ~/.claude/memory/observations/$(date +%Y-%m-%d)-
observations.jsonl
```

### Success Criteria

| Criterion | Verification |
|---|---|
| BM25 returns relevant results | Test query returns expected files |
| Decision schema validates | Sample decision passes Pydantic |
| Observations accumulating | JSONL files growing |
| Frontmatter standard documented | Format doc exists |

### Estimated Effort: 20-25 hours

---

## Phase 3: Weekly Review System (Month 3)

### Objectives

1. Implement Memory Synthesis agent
2. Implement Systems Architect agent
3. Create human interview protocol
4. Establish automated recommendation generation

### Deliverables

---

| Deliverable | Path | Description |
|---|---|---|
| Memory Synthesis agent | `.claude/agents/memory-synthesis/` | Aggregation agent |
| Systems Architect agent | `.claude/agents/systems-architect/` | Analysis agent |
| Review orchestrator | `.claude/scripts/weekly-review.sh` | Review trigger |
| Interview template | `.claude/docs/review-interview.md` | Human interaction |
| Recommendation schema | `.claude/schemas/recommendation.py` | Output format |

## Implementation Steps

### Step 3.1: Memory Synthesis Agent (6 hours)

```yaml
# .claude/agents/memory-synthesis/agent.yaml
name: memory-synthesis
version: 1.0.0
tier: sonnet
purpose: |
  Aggregate and synthesize memory artifacts from the past week
  to prepare input for systems architect analysis.

triggers:
  - manual: "/weekly-review"
  - scheduled: "0 9 * * 1"  # Monday 9 AM

inputs:
  - .claude/memory/decisions/*.md (last 7 days)
  - .claude/memory/sharp-edges/*.md (last 7 days)
  - .claude/tmp/session_summaries/*.json (last 7 days)
  - .claude/tmp/routing_log_*.jsonl (last 7 days)
  - .claude/memory/observations/*.jsonl (last 7 days)

outputs:
  - .claude/tmp/weekly_synthesis.json

constraints:
  - max_input_tokens: 50000
  - max_output_tokens: 5000
  - timeout_minutes: 5

# .claude/agents/memory-synthesis/CLAUDE.md
# Memory Synthesis Agent

You are the Memory Synthesis agent. Your role is to aggregate and
summarize
the past week's system activity to prepare input for architectural
analysis.

## Your Task

1. Read all provided memory files from the past week
2. Read all session summaries and routing logs
3. Read accumulated observations

4. Produce a synthesis document containing:
   - Total sessions and tool invocations
   - Cost breakdown by tier
   - Success rate overall and by task type
   - Key decisions made and their outcomes
   - Sharp edges encountered
   - Human override patterns
   - Observation counts by event type

## Output Format

Produce JSON matching this structure:
```json
{
  "period": {"start": "YYYY-MM-DD", "end": "YYYY-MM-DD"},
  "sessions": {"count": N, "avg_duration_minutes": N},
  "costs": {"total_usd": N, "by_tier": {"haiku": N, "sonnet": N,
"opus": N}},
  "performance": {"success_rate": N, "blocked_requests": N},
  "decisions": [{"title": "", "outcome": "", "category": ""}],
  "sharp_edges": [{"title": "", "severity": "", "resolved": bool}],
  "overrides": {"count": N, "patterns": ["pattern1", "pattern2"]},
  "observations": {"total": N, "by_type": {"type1": N, "type2": N}}
}
```

Be concise. Focus on patterns, not individual events.

#### Step 3.2: Systems Architect Agent (8 hours)

```yaml
# .claude/agents/systems-architect/agent.yaml
name: systems-architect
version: 1.0.0
tier: opus
purpose: |
  Analyze weekly synthesis and generate architectural
```
```

```
recommendations
  including gap identification and improvement proposals.

triggers:
  - after: memory-synthesis
  - manual: "/architect-review"

inputs:
  - .claude/tmp/weekly_synthesis.json
  - .claude/memory/observations/*.jsonl (for pattern analysis)
  - .claude/schemas/ (current schema definitions)
  - .claude/agents/ (current agent registry)

outputs:
  - .claude/tmp/architect_report.md
  - .claude/tmp/recommendations.json

constraints:
  - extended_thinking: true
  - max_input_tokens: 100000
  - max_output_tokens: 10000
  - timeout_minutes: 10

        # .claude/agents/systems-architect/CLAUDE.md
        # Systems Architect Agent

        You are the Systems Architect agent. Your role is to perform deep
analysis
        of system behavior and generate actionable improvement
recommendations.

        ## Your Responsibilities

        ### 1. Gap Analysis
        - Identify recurring failure patterns (≥3 occurrences)
        - Flag tasks that consistently exceed time/cost budgets
        - Detect human overrides that suggest missing capabilities
        - Find context boundary issues causing synthesis gaps

        ### 2. For Each Gap, Determine
        - Can this be resolved by configuration change? → Generate config
diff
        - Does this require new subagent capability? → Generate agent
template
        - Is this a one-off or recurring pattern? → Prioritize accordingly

        ### 3. Schema Emergence Check (if observations ≥ 200)
        - Analyze observation patterns
        - Identify clusters with ≥10 instances
        - Propose schema if patterns are statistically significant

        ### 4. Autonomy Assessment
        - Review decision categories
        - Calculate success rates per category
        - Recommend level promotions where thresholds met

        ## Output Format

        ### architect_report.md
        Human-readable analysis with sections:
        - Executive Summary
        - Gap Analysis Findings
        - Schema Emergence Status
        - Autonomy Assessment
        - Recommendations Summary

        ### recommendations.json
        ```json
        {
          "generated_at": "ISO8601",
          "recommendations": [
            {
              "id": "uuid",
              "type":
"config_change|new_agent|schema_activation|autonomy_promotion",
              "priority": "high|medium|low",
              "title": "Brief title",
              "description": "Detailed description",
              "implementation": {
                "files_affected": ["path1", "path2"],
                "diff_or_template": "...",
                "effort_hours": N
              },
              "evidence": ["observation1", "observation2"]
            }
          ]
        }
```

Be thorough but actionable. Every recommendation must be implementable.

```
#### Step 3.3: Review Orchestrator (4 hours)

```bash
#!/bin/bash
# .claude/scripts/weekly-review.sh
```

```
set -euo pipefail

echo
"═══════════════════════════════════════════════════════════"
echo "                    WEEKLY REVIEW PROCESS
"
echo
"═══════════════════════════════════════════════════════════"

REVIEW_DATE=$(date +%Y-%m-%d)
REVIEW_DIR="$HOME/.claude/tmp/reviews/$REVIEW_DATE"
mkdir -p "$REVIEW_DIR"

# Phase 1: Memory Synthesis
echo ""
echo "Phase 1: Memory Synthesis"
echo "──────────────────────────"

# Invoke memory synthesis agent (placeholder - actual implementation
via Claude)
claude --agent memory-synthesis --output
"$REVIEW_DIR/synthesis.json"

if [[ ! -f "$REVIEW_DIR/synthesis.json" ]]; then
    echo "ERROR: Memory synthesis failed"
    exit 1
fi

echo "✓ Synthesis complete: $REVIEW_DIR/synthesis.json"

# Phase 2: Systems Architect Analysis
echo ""
echo "Phase 2: Systems Architect Analysis"
echo "───────────────────────────────────"

claude --agent systems-architect \
    --input "$REVIEW_DIR/synthesis.json" \
    --output-report "$REVIEW_DIR/architect_report.md" \
    --output-recommendations "$REVIEW_DIR/recommendations.json"

echo "✓ Analysis complete"

# Phase 3: Human Interview
echo ""
echo "Phase 3: Review Findings"
echo "────────────────────────"
echo ""

cat "$REVIEW_DIR/architect_report.md"

echo ""
echo
"═══════════════════════════════════════════════════════════"
echo "                    RECOMMENDATIONS
"
echo
"═══════════════════════════════════════════════════════════"

# Present recommendations interactively
jq -r '.recommendations[] | "[\(.priority)] \(.title)\n    \
(.description)\n"' \
    "$REVIEW_DIR/recommendations.json"

echo ""
echo "Review complete. Recommendations saved to:"
echo "  $REVIEW_DIR/recommendations.json"
echo ""
echo "To implement recommendations, run:"
echo "  claude --implement-recommendation <id>"
```

### Verification Commands

```
# Test weekly review (after accumulating data)
~/.claude/scripts/weekly-review.sh

# View generated recommendations
cat ~/.claude/tmp/reviews/$(date +%Y-%m-%d)/recommendations.json |
jq .
```

### Success Criteria

| Criterion | Verification |
|---|---|
| Synthesis agent produces output | synthesis.json exists and valid |
| Architect agent analyzes | architect_report.md exists |
| Recommendations generated | recommendations.json has entries |
| Human review possible | Report is readable and actionable |

### Estimated Effort: 25-30 hours

---

## Phase 4: Emergent Schema Discovery (Months 4-5)

### Objectives

1. Implement observation accumulation infrastructure
2. Create pattern detection subagent
3. Build schema proposal mechanism
4. Establish human approval workflow

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Schema Discovery agent | `.claude/agents/schema-discovery/` | Pattern analyzer |
| Pattern detector | `.claude/scripts/detect-patterns.py` | Statistical analysis |
| Schema proposer | `.claude/scripts/propose-schema.py` | Schema generation |
| Approval workflow | `.claude/docs/schema-approval.md` | Human process |

*Detailed implementation in Technical Specifications (Part VI)*

### Key Thresholds (from research)

| Threshold | Value | Source |
|---|---|---|
| Minimum observations for analysis | 200 | Statistical power research |
| Observations per cluster | 30 | 80% statistical power |
| Silhouette score threshold | 0.5 | Cluster validity |
| Bootstrap stability | 80% | Reproducibility requirement |

### Estimated Effort: 40-50 hours

---

## Phase 5: Subagent Spawning (Months 6-7)

### Objectives

1. Implement gap-to-agent generation
2. Create shadow deployment infrastructure
3. Build promotion criteria system
4. Establish deprecation lifecycle

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Agent generator | `.claude/scripts/generate-agent.py` | Template creation |
| Shadow runner | `.claude/scripts/shadow-deploy.sh` | Parallel execution |
| Promotion evaluator | `.claude/scripts/evaluate-promotion.py` | Criteria check |
| Agent registry | `.claude/agents-index.json` | Lifecycle tracking |

*Detailed implementation in Technical Specifications (Part VI)*

### Key Thresholds (from research)

| Threshold | Value | Source |
|---|---|---|
| Shadow period minimum | 10 invocations | Statistical significance |
| Success rate for promotion | 90% | Quality gate |
| Error rate vs baseline | ≤0.1% increase | Risk management |
| Maximum shadow duration | 14 days | Prevent limbo |
| Agent population cap | 15 active | Coordination overhead |

### Estimated Effort: 50-60 hours

---

## Phase 6: Autonomy Progression (Months 8-10)

### Objectives

1. Implement decision category tracking
2. Build success rate monitoring per category
3. Create level promotion logic
4. Establish audit trail system

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Autonomy tracker | `.claude/autonomy-levels.yaml` | Level state |
| Category analyzer | `.claude/scripts/analyze-categories.py` | Success rates |
| Promotion engine | `.claude/scripts/promote-autonomy.py` | Level changes |
| Audit logger | `.claude/memory/autonomy-audit.jsonl` | Decision trail |

*Detailed implementation in Technical Specifications (Part VI)*

### Key Thresholds (from research)

| Level Transition | Requirement | Source |
|---|---|---|
| L1 → L2 | 100 decisions captured | Sufficient data |
| L2 → L3 | 95% success over 200 decisions | Collaborator trust |
| L3 → L4 | 98% success over 500 decisions | Consultant trust |
| L4 → L5 | Domain-specific, human-defined | Full trust |

### Estimated Effort: 40-50 hours

---

## Phase 7: Self-Improving System (Months 11-12)

### Objectives

1. Implement schema versioning automation
2. Create subagent deprecation lifecycle
3. Build continuous improvement loop
4. Establish human oversight dashboard

### Deliverables

| Deliverable | Path | Description |
|---|---|---|
| Schema migrator | `.claude/scripts/migrate-schema.py` | Version handling |
| Deprecation manager | `.claude/scripts/manage-deprecation.py` | Lifecycle |
| Improvement loop | `.claude/scripts/continuous-improve.sh` | Automation |
| Dashboard data | `.claude/tmp/dashboard/` | Visualization data |

*Detailed implementation in Technical Specifications (Part VI)*

### Estimated Effort: 35-45 hours

---

## Dependency Graph

```
┌─────────────
```

```
           │       │
           │       └──► Phase 3
──────────────────────────────────────────────────────────────►
           │               │
           │               │ Weekly Review, Synthesis, Architect
           │               │
           │               ├──► Phase 4
──────────────────────────────────────────────────────────►
           │               │       │
           │               │       │ Schema Discovery, Pattern
Detection
           │               │       │
           │               │       └──► Phase 5
──────────────────────────────────────────────────────►
           │               │               │
           │               │               │ Subagent Spawning,
Shadow Deploy
           │               │               │
           │               │               └──► Phase 6
──────────────────────────────────────────────►
           │               │                       │
           │               │                       │ Autonomy
Progression
           │               │                       │
           │               │                       └──► Phase 7
──────────────────────────────►
           │               │                               │
           │               │                               │ Self-
Improving Loop
           │               │                               │
           │               │                               ▼
           │               │                           COMPLETE
           │               │
           │               │ (Weekly Review continues independently)
           │
└──────────────────────────────────────────────────────────►
           │
           │ (Observability continues independently)

└──────────────────────────────────────────────────────────────────────────────


LEGEND:
──► Sequential dependency (must complete before next)
│   Parallel workstream (continues independently)
```

---

## Total Effort Estimate

| Phase | Timeline | Effort (hours) |
|---|---|---|
| Phase 0 | Weeks 1-2 | 12-15 |
| Phase 1 | Month 1 | 20-25 |
| Phase 2 | Month 2 | 20-25 |
| Phase 3 | Month 3 | 25-30 |
| Phase 4 | Months 4-5 | 40-50 |
| Phase 5 | Months 6-7 | 50-60 |
| Phase 6 | Months 8-10 | 40-50 |
| Phase 7 | Months 11-12 | 35-45 |
| **Total** | **12 months** | **242-300 hours** |

At 10-15 hours/week of focused development: **20-30 weeks of active work** spread across 12 months, allowing for iteration, testing, and refinement.

---

*Continue to Part VI: Technical Specifications* → # Part VI: Technical Specifications

## Complete Reference for Implementation

---

## 1. Threshold Reference Table

All numeric thresholds derived from research, with justification and adjustment guidelines.

### 1.1 Routing Thresholds

| Threshold | Value | Justification | Adjustment Trigger |
|---|---|---|---|
| Haiku ceiling | Score < 2 | Simple tasks, minimal context | Increase if Haiku failures >5% |
| Sonnet ceiling | Score 2-10 | Standard development work | Adjust based on cost/quality tradeoff |
| Opus floor | Score > 10 | Complex reasoning required | Decrease if over-routing detected |

| | | | |
|---|---|---|---|
| Gemini trigger | Tokens > 50,000 | Context window limits | Fixed (model constraint) |
| Gemini force | Tokens > 100,000 | Safety margin | Fixed (model constraint) |

### 1.2 Complexity Formula

```
SCORE = (tokens / 10000) + (files / 5) + (modules × 2)
```

| Component | Weight | Rationale |
|---|---|---|
| Tokens | 1 per 10K | Primary context consumption metric |
| Files | 1 per 5 | Cross-file coordination overhead |
| Modules | 2 per module | Architectural complexity indicator |

**Example calculations:**

| Scenario | Tokens | Files | Modules | Score | Tier |
|---|---|---|---|---|---|
| Simple fix | 5,000 | 1 | 1 | 0.5 + 0.2 + 2 = 2.7 | Sonnet |
| Feature add | 25,000 | 8 | 2 | 2.5 + 1.6 + 4 = 8.1 | Sonnet |
| Refactor | 45,000 | 15 | 4 | 4.5 + 3.0 + 8 = 15.5 | Opus |
| Codebase analysis | 150,000 | 50 | 10 | 15 + 10 + 20 = 45 | Gemini |

### 1.3 Pattern Detection Thresholds

| Threshold | Value | Source | Purpose |
|---|---|---|---|
| Min observations for analysis | 200 | Statistical power research | Trigger schema discovery |
| Observations per cluster | 30 | 80% statistical power | Valid cluster minimum |
| Cluster count formula | N = 30 × k | k = expected clusters | Sample size planning |
| Silhouette score minimum | 0.5 | Cluster validity standard | Pattern quality gate |
| Bootstrap stability | 80% | Reproducibility standard | Pattern reliability |
| Confidence interval | 99% (α = 0.01) | Production ML standard | Decision threshold |

### 1.4 Autonomy Progression Thresholds

| Transition | Decisions Required | Success Rate | Rationale |
|---|---|---|---|
| L1 → L2 | 100 | N/A | Sufficient observation data |
| L2 → L3 | 200 | 95% | Collaborator trust earned |
| L3 → L4 | 500 | 98% | Consultant trust earned |
| L4 → L5 | Human-defined | 99%+ | Domain-specific full trust |

**Demotion triggers:** - Success rate drops 5% below promotion threshold - 3+ consecutive failures in category - Human requests demotion

### 1.5 Shadow Deployment Thresholds

| Threshold | Value | Source | Purpose |
|---|---|---|---|
| Min shadow invocations | 10 | Statistical significance | Minimum sample |
| Success rate for promotion | 90% | Quality gate | Promotion criterion |
| Error rate vs baseline | ≤ 0.1% increase | Risk management | Safety margin |
| Max shadow duration | 14 days | Prevent deployment limbo | Time bound |
| Traffic percentage (shadow) | 0% live | Shadow definition | No user impact |
| Traffic percentage (canary) | 1-5% | Progressive rollout | Initial exposure |

### 1.6 Memory and Retrieval Thresholds

| Threshold | Value | Purpose |
|---|---|---|
| BM25 top-k default | 5 | Balance |

| | | |
|---|---|---|
| | | relevance/noise |
| Memory file limit warning | 500 files | Curation trigger |
| Memory file limit hard | 1000 files | Performance concern |
| Stale memory threshold | 90 days | Review for archival |
| Frontmatter required fields | 5 | title, created, category, tags, summary |

### 1.7 Cost Control Thresholds

| Threshold | Value | Action |
|---|---|---|
| Session cost warning | $5 | Alert user |
| Session cost limit | $10 | Require confirmation |
| Weekly budget default | $50 | Review trigger |
| Opus calls per session warning | 5 | Review routing |
| Weekly review budget | $2 | Infrastructure investment |

### 1.8 Document Processing Thresholds

| Threshold | Value | Source | Purpose |
|---|---|---|---|
| Chunk size (factoid) | 256-512 tokens | Industry standard | Q&A retrieval |
| Chunk size (analytical) | 1024+ tokens | Industry standard | Complex analysis |
| Chunk size (default) | 512 tokens | Starting point | General use |
| Overlap percentage | 10-20% | NVIDIA research | Boundary coverage |
| Overlap optimal | 15% | FinanceBench 2024 | Best recall |
| Parallel workers max | 5 | Rate limit safety | Gemini calls |

---

## 2. Schema Specifications

### 2.1 Observation Event Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ObservationEvent",
  "description": "Raw behavioral observation for pattern discovery",
  "type": "object",
  "required": ["timestamp", "event_type", "context", "action"],
  "properties": {
    "timestamp": {
      "type": "string",
      "format": "date-time",
      "description": "ISO 8601 timestamp"
    },
    "event_type": {
      "type": "string",
      "enum": [
        "human_override",
        "plan_approval",
        "plan_rejection",
        "scope_modification",
        "tier_escalation",
        "tier_override",
        "clarification_request",
        "task_completion",
        "task_failure",
        "memory_retrieval",
        "agent_handoff"
      ]
    },
    "context": {
      "type": "object",
      "properties": {
        "task": {"type": "string"},
        "complexity_score": {"type": "number"},
        "recommended_tier": {"type": "string"},
        "files_in_scope": {"type": "integer"},
        "session_id": {"type": "string"},
        "tags": {"type": "array", "items": {"type": "string"}}
      }
    },
    "action": {
      "type": "object",
      "properties": {
        "type": {"type": "string"},
        "from": {"type": "string"},
        "to": {"type": "string"},
        "reason_provided": {"type": "string"}
      },
      "required": ["type"]
    },
    "outcome": {
      "type": "object",
```

```
            "properties": {
              "success": {"type": "boolean"},
              "duration_seconds": {"type": "integer"},
              "error_message": {"type": "string"}
            }
          }
        }
      }
```

### 2.2 Decision Capture Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Decision",
  "description": "Captured decision for apprenticeship learning",
  "type": "object",
  "required": [
    "schema_version",
    "decision_id",
    "timestamp",
    "session_id",
    "decision_category",
    "context",
    "human_decision",
    "learning_metadata"
  ],
  "properties": {
    "schema_version": {
      "type": "string",
      "const": "1.0.0"
    },
    "decision_id": {
      "type": "string",
      "format": "uuid"
    },
    "timestamp": {
      "type": "string",
      "format": "date-time"
    },
    "session_id": {
      "type": "string"
    },
    "decision_category": {
      "type": "string",
      "enum": [
        "routing_override",
        "scope_modification",
        "plan_approval",
        "plan_rejection",
        "human_escalation",
        "task_delegation",
        "memory_curation",
        "agent_selection"
      ]
    },
    "context": {
      "type": "object",
      "properties": {
        "task_type": {"type": "string"},
        "task_description": {"type": "string"},
        "complexity_score": {"type": "number"},
        "files_in_scope": {"type": "integer"},
        "estimated_tokens": {"type": "integer"},
        "tags": {"type": "array", "items": {"type": "string"}}
      }
    },
    "system_recommendation": {
      "type": "object",
      "properties": {
        "action": {"type": "string"},
        "confidence": {"type": "number", "minimum": 0, "maximum":
1},
        "reasoning": {"type": "string"}
      }
    },
    "human_decision": {
      "type": "object",
      "required": ["action"],
      "properties": {
        "action": {"type": "string"},
        "reasoning_provided": {"type": "string"},
        "time_to_decide_seconds": {"type": "integer"}
      }
    },
    "outcome": {
      "type": "object",
      "properties": {
        "task_success": {"type": "boolean"},
        "quality_rating": {"type": "integer", "minimum": 1,
"maximum": 5},
        "issues_encountered": {"type": "array", "items": {"type":
"string"}},
        "would_recommend_same": {"type": "boolean"}
      }
    },
    "learning_metadata": {
```

```
              "type": "object",
              "required": ["autonomy_level_at_time"],
              "properties": {
                "autonomy_level_at_time": {"type": "integer", "minimum": 1,
"maximum": 5},
                "pattern_match_candidates": {"type": "array", "items":
{"type": "string"}},
                "should_automate_similar": {"type": "boolean"},
                "requires_human_always": {"type": "boolean"}
              }
            }
          }
        }
```

### 2.3 Agent Definition Schema

```
        {
          "$schema": "http://json-schema.org/draft-07/schema#",
          "title": "AgentDefinition",
          "description": "Subagent configuration and metadata",
          "type": "object",
          "required": ["name", "version", "tier", "purpose"],
          "properties": {
            "name": {
              "type": "string",
              "pattern": "^[a-z][a-z0-9-]*$",
              "maxLength": 50
            },
            "version": {
              "type": "string",
              "pattern": "^\\d+\\.\\d+\\.\\d+$"
            },
            "status": {
              "type": "string",
              "enum": ["proposed", "shadow", "active", "deprecated",
"archived"],
              "default": "proposed"
            },
            "tier": {
              "type": "string",
              "enum": ["haiku", "sonnet", "opus", "gemini"]
            },
            "purpose": {
              "type": "string",
              "maxLength": 500
            },
            "created": {
              "type": "string",
              "format": "date"
            },
            "trigger_conditions": {
              "type": "array",
              "items": {
                "type": "object",
                "properties": {
                  "condition": {"type": "string"},
                  "value": {}
                }
              }
            },
            "configuration": {
              "type": "object",
              "additionalProperties": true
            },
            "input_schema": {
              "type": "object",
              "additionalProperties": {"type": "string"}
            },
            "output_schema": {
              "type": "object",
              "additionalProperties": {"type": "string"}
            },
            "constraints": {
              "type": "object",
              "properties": {
                "max_input_tokens": {"type": "integer"},
                "max_output_tokens": {"type": "integer"},
                "timeout_minutes": {"type": "integer"},
                "extended_thinking": {"type": "boolean"}
              }
            },
            "metrics": {
              "type": "object",
              "properties": {
                "invocations": {"type": "integer", "default": 0},
                "successes": {"type": "integer", "default": 0},
                "failures": {"type": "integer", "default": 0},
                "avg_duration_seconds": {"type": "number"},
                "avg_cost_usd": {"type": "number"}
              }
            }
          }
        }
```

### 2.4 Memory File Schema (YAML Frontmatter)

```yaml
        # Required fields
        title: string           # Brief descriptive title (max 100 chars)
        created: date           # YYYY-MM-DD format
        category: enum          # decisions | sharp-edges | facts |
preferences | observations
        tags: array[string]     # Searchable tags (max 10)
        summary: string         # One-line searchable summary (max 200
chars)

        # Optional fields
        updated: date           # Last modification date
        related: array[string]  # Relative paths to related files
        confidence: enum        # high | medium | low
        status: enum            # active | deprecated | archived (default:
active)
        source: string          # Session ID or "manual"
        expires: date           # For time-sensitive facts
        integrity:
          content_hash: string  # sha256:... of body content
          last_verified: datetime
```

**Validation rules:** - `title` required, max 100 characters - `created` required, valid ISO date - `category` required, must be enum value - `tags` required, 1-10 items, each max 30 chars - `summary` required, max 200 characters

---

# 3. Hook Implementation Specifications

## 3.1 Hook Lifecycle

```
┌─────────────────────────────────────────────────────────────────────────┐
│                              HOOK EXECUTION ORDER                       │
└─────────────────────────────────────────────────────────────────────────┘


SESSION START
     │
     ▼
┌──────────────┐
│ SessionStart.sh │ ← Load context, verify environment
│ (optional)   │
└──────────────┘
        │
        ▼
    ┌─────────┐
    │  TASK   │
    │  LOOP   │◄──────────────────────────────────────┐
    └─────────┘                                       │
        │                                             │
        ▼                                             │
┌──────────────┐                                      │
│ PreToolUse.sh   │ ← Routing enforcement, validation    │
│ (required)   │                                      │
└──────────────┘                                      │
        │                                             │
    ┌─────────┐                                       │
    │ exit 0  │ PERMIT                                │
    │ exit 2  │ BLOCK ──────────────────────────┐     │
    └─────────┘                                 │     │
        │                                       │     │
        ▼                                       │     │
┌──────────────┐                                │     │
│ TOOL EXECUTION  │                              │     │
└──────────────┘                                │     │
        │                                       │     │
        ▼                                       │     │
┌──────────────┐                                │     │
│ PostToolUse.sh  │ ← Log outcome, capture metrics │     │
│ (required)   │                                │     │
└──────────────┘                                │     │
        │                                       │     │
        ▼                                       │     │
┌──────────────┐                                │     │
│ SubagentStop.sh │ ← Validate handoff (if agent)  │     │
│ (conditional)   │                              │     │
└──────────────┘                                │     │
        │                                       │     │
        └───────────────────────────────────────┴─────┘
                                                │
        ┌───────────────────────────────────────┘
        │
        ▼
┌──────────────┐
│ SessionEnd.sh   │ ← Summary, archival trigger
│ (optional)   │
└──────────────┘
```

## 3.2 PreToolUse.sh Specification

**Purpose:** Enforce routing decisions, validate requests, log decisions

**Exit codes:** | Code | Meaning | Behavior | |——|———|————-| | 0 | PERMIT | Continue with tool execution | | 1 | ERROR | Hook failure, blocks execution | | 2 | BLOCK | Routing violation, blocks with message |

**Environment variables available:**

```
CLAUDE_TOOL_NAME        # Name of tool being invoked
CLAUDE_SESSION_ID       # Current session identifier
CLAUDE_REQUESTED_TIER   # Tier requested (if applicable)
CLAUDE_WORKING_DIR      # Current working directory
```

**Required behavior:** 1. Read complexity score from state file 2. Compare requested operation to tier ceiling 3. Log decision to routing_log.jsonl 4. Return appropriate exit code

**Template:**

```
#!/bin/bash
set -euo pipefail

# Configuration
STATE_DIR="$HOME/.claude/tmp"
LOG_FILE="$STATE_DIR/routing_log.jsonl"

# Read state
SCORE=$(cat "$STATE_DIR/complexity_score" 2>/dev/null || echo "5")
CEILING=$(cat "$STATE_DIR/recommended_tier" 2>/dev/null || echo
"sonnet")

# Enforcement logic
# [implementation here]

# Log decision
echo "{...}" >> "$LOG_FILE"

exit $EXIT_CODE
```

### 3.3 PostToolUse.sh Specification

**Purpose:** Capture outcomes, update metrics, detect anomalies

**Exit codes:** | Code | Meaning | Behavior | |——|———|————-| | 0 | SUCCESS | Continue normally | | 1 | ERROR | Log error, continue |

**Environment variables available:**

```
CLAUDE_TOOL_NAME        # Name of tool that executed
CLAUDE_TOOL_EXIT_CODE   # Exit code from tool
CLAUDE_SESSION_ID       # Current session identifier
CLAUDE_TOOL_DURATION    # Execution time in milliseconds
```

**Required behavior:** 1. Read last routing log entry 2. Update with actual outcome 3. Detect sharp edges (errors, unusual patterns) 4. Queue observations for learning

### 3.4 SubagentStop.sh Specification

**Purpose:** Validate agent completion, verify handoff integrity

**Exit codes:** | Code | Meaning | Behavior | |——|———|————-| | 0 | VALID | Handoff accepted | | 1 | ERROR | Hook failure | | 2 | INVALID | Handoff rejected, blocks |

**Validation checklist:** - [ ] Handoff file exists and is valid JSON - [ ] Required fields present (task_summary, success_criteria) - [ ] Referenced artifacts exist - [ ] Status is “completed” (warn if not) - [ ] Schema version compatible

### 3.5 SessionEnd.sh Specification

**Purpose:** Generate session summary, trigger archival processes

**Exit codes:** | Code | Meaning | Behavior | |——|———|————-| | 0 | SUCCESS | Clean shutdown | | 1 | ERROR | Log error, continue shutdown |

**Required behavior:** 1. Aggregate session metrics from routing log 2. Generate session summary JSON 3. Archive routing log with session ID 4. Trigger memory archivist (background) 5. Clean up temporary state files

---

## 4. State File Specifications

### 4.1 scout_metrics.json

**Location:** `~/.claude/tmp/scout_metrics.json` **Lifetime:** Per-task (refreshed on each scout operation) **Max age:** 5 minutes before considered stale

```
{
  "schema_version": "1.0.0",
  "generated_at": "2026-01-13T10:23:45Z",
  "scout_agent": "haiku",
```

```
  "scout_report": {
    "scope_metrics": {
      "total_files": 15,
      "file_types": {
        ".py": 10,
        ".md": 3,
        ".json": 2
      },
      "estimated_tokens": 45000,
      "largest_file": {
        "path": "src/auth/handlers.py",
        "tokens": 8500
      }
    },
    "complexity_signals": {
      "cross_file_dependencies": 8,
      "module_count": 3,
      "circular_imports": 0,
      "test_coverage_files": 5
    },
    "recommendations": {
      "suggested_tier": "sonnet",
      "gemini_offload_candidates": [],
      "risk_factors": ["large handlers.py may need splitting"]
    }
  }
}
```

### 4.2 complexity_score

**Location:** `~/.claude/tmp/complexity_score` **Lifetime:** Per-task **Format:** Plain text, single decimal number

```
13.50
```

### 4.3 recommended_tier

**Location:** `~/.claude/tmp/recommended_tier` **Lifetime:** Per-task **Format:** Plain text, tier name

```
opus
```

### 4.4 handoff.json

**Location:** `~/.claude/tmp/handoff.json` **Lifetime:** Per-agent-transition **Max size:** 100KB

```
{
  "schema_version": "1.0.0",
  "handoff_id": "550e8400-e29b-41d4-a716-446655440000",
  "from_agent": "architect",
  "to_agent": "executor",
  "created_at": "2026-01-13T10:25:00Z",
  "status": "completed",
  "context": {
    "task_summary": "Implement OAuth refresh token rotation",
    "files_in_scope": [
      "src/auth/tokens.py",
      "src/auth/middleware.py"
    ],
    "critical_constraints": [
      "Must maintain backward compatibility with existing tokens",
      "Refresh rotation must be atomic"
    ],
    "success_criteria": [
      "All existing tests pass",
      "New rotation tests added",
      "No breaking API changes"
    ]
  },
  "artifacts": {
    "specs_path": ".claude/tmp/specs.md",
    "scout_metrics_path": ".claude/tmp/scout_metrics.json"
  },
  "metadata": {
    "estimated_tokens": 25000,
    "estimated_duration_minutes": 15,
    "tier_ceiling": "sonnet"
  }
}
```

### 4.5 routing_log.jsonl

**Location:** `~/.claude/tmp/routing_log.jsonl` **Lifetime:** Per-session (archived on SessionEnd) **Format:** JSON Lines (one JSON object per line)

```
{"timestamp":"2026-01-
13T10:23:45Z","session_id":"abc123","decision_id":"uuid","tool_name":"Edit","routing":
{"complexity_score":8.5,"calculated_tier":"sonnet","requested_tier":"opus","final_tier":"sonnet","decis
 exceeds ceiling"},"cost":
{"estimated_tokens":35000,"estimated_cost_usd":0.105},"outcome":
{"actual_tokens":null,"actual_cost_usd":null,"task_success":null}}
```

---

# 5. Directory Structure

## 5.1 Complete Annotated Tree

```
~/.claude/
├── hooks/                          # Lifecycle hooks (executable
scripts)
│   ├── PreToolUse.sh              # REQUIRED: Routing enforcement
│   ├── PostToolUse.sh             # REQUIRED: Outcome capture
│   ├── SubagentStop.sh            # OPTIONAL: Handoff validation
│   ├── SessionStart.sh            # OPTIONAL: Context loading
│   └── SessionEnd.sh              # OPTIONAL: Summary/archival
│
├── scripts/                        # Utility scripts
│   ├── calculate-complexity.sh    # Complexity formula
implementation
│   ├── validate-routing.sh        # Tier permission validation
│   ├── validate-state.py          # Schema validation
│   ├── query-memory-bm25.py       # Memory retrieval
│   ├── log-observation.py         # Observation capture
│   ├── generate-cost-report.sh    # Cost aggregation
│   ├── weekly-review.sh           # Review orchestrator
│   └── verify-memory-integrity.sh # Integrity checking
│
├── agents/                         # Agent definitions
│   ├── haiku-scout/
│   │   ├── agent.yaml             # Configuration
│   │   ├── agent.md               # Description
│   │   └── CLAUDE.md              # Instructions
│   ├── architect/
│   ├── memory-archivist/
│   ├── memory-synthesis/
│   ├── systems-architect/
│   ├── schema-discovery/          # Phase 4
│   └── [spawned-agents]/          # Phase 5+
│
├── skills/                         # Workflow definitions
│   └── explore/
│       └── SKILL.md
│
├── schemas/                        # Schema definitions
│   ├── state/                     # State file schemas
│   │   ├── scout_metrics.py
│   │   ├── handoff.py
│   │   └── routing_log.py
│   ├── memory/                    # Memory schemas
│   │   ├── decision.py
│   │   └── observation.py
│   └── [version]/                 # Versioned schemas (Phase 4+)
│
├── tmp/                            # Ephemeral state (cleared per
session)
│   ├── scout_metrics.json
│   ├── complexity_score
│   ├── recommended_tier
│   ├── handoff.json
│   ├── routing_log.jsonl
│   ├── specs.md
│   ├── validation_log.jsonl
│   ├── session_summaries/
│   │   └── [session_id].json
│   ├── reviews/
│   │   └── [date]/
│   │       ├── synthesis.json
│   │       ├── architect_report.md
│   │       └── recommendations.json
│   └── metrics/
│       └── cost_report.json
│
├── memory/                         # Persistent learning (git
versioned)
│   ├── decisions/                 # Architectural decisions
│   │   └── YYYY-MM-DD-topic.md
│   ├── sharp-edges/               # Known pitfalls
│   │   └── YYYY-MM-DD-issue.md
│   ├── facts/                     # Verified project facts
│   │   └── topic.md
│   ├── preferences/               # User preferences
│   │   └── category.md
│   ├── observations/              # Raw behavioral observations
│   │   └── YYYY-MM-DD-observations.jsonl
│   ├── audit.jsonl                # Memory access audit log
│   └── index.json                 # Searchable index (generated)
│
├── docs/                           # Documentation
│   ├── memory-format.md           # Memory file standard
│   ├── schema-approval.md         # Schema approval workflow
│   └── review-interview.md        # Human interview template
│
├── settings.json                   # Global configuration
├── routing-schema.json            # Routing rules and thresholds
├── agents-index.json              # Agent registry with lifecycle
└── autonomy-levels.yaml           # Autonomy state per category

~/.gemini-slave/                    # Gemini integration
├── protocols/
```

```
│   ├── handover-protocol.md
│   ├── codebase-analysis.md
│   └── document-synthesis.md
└── config.yaml
```

### 5.2 File Naming Conventions

| Location | Pattern | Example |
|---|---|---|
| Memory decisions | `YYYY-MM-DD-topic.md` | `2026-01-13-jwt-strategy.md` |
| Memory sharp-edges | `YYYY-MM-DD-issue.md` | `2026-01-13-circular-import.md` |
| Memory facts | `topic.md` | `project-structure.md` |
| Observations | `YYYY-MM-DD-observations.jsonl` | `2026-01-13-observations.jsonl` |
| Session summaries | `[session_id].json` | `abc123.json` |
| Archived routing logs | `routing_log_[session_id].jsonl` | `routing_log_abc123.js` |
| Review directories | `YYYY-MM-DD/` | `2026-01-13/` |

### 5.3 Lifecycle Management

| Directory | Retention | Cleanup Trigger |
|---|---|---|
| `tmp/` | Session | SessionEnd hook |
| `tmp/session_summaries/` | 30 days | Weekly cleanup |
| `tmp/reviews/` | 90 days | Monthly cleanup |
| `memory/observations/` | Until processed | Schema discovery |
| `memory/decisions/` | Permanent | Manual archival |
| `memory/sharp-edges/` | Permanent | Manual archival |

---

## 6. API Contracts

### 6.1 Gemini CLI Interface

**Invocation pattern:**

```
gemini-cli <command> [options]
```

**Commands:**

| Command | Purpose | Key Options |
|---|---|---|
| `analyze` | Codebase analysis | `--input-dir`, `--output`, `--max-tokens` |
| `summarize` | Document summarization | `--input`, `--output`, `--format` |
| `synthesize` | Multi-document synthesis | `--inputs`, `--output` |

**Example invocations:**

```
# Codebase analysis
gemini-cli analyze \
  --protocol codebase-analysis \
  --input-dir ./src \
  --output .claude/tmp/analysis.json \
  --max-tokens 100000

# Document synthesis
gemini-cli synthesize \
  --inputs ".claude/tmp/summary_*.json" \
  --output .claude/tmp/synthesis.md \
  --format markdown
```

**Output format:**

```
{
  "status": "success|error",
  "output_path": "/path/to/output",
  "metrics": {
    "input_tokens": 45000,
    "output_tokens": 3000,
    "duration_seconds": 12
  },
  "error": null
}
```

### 6.2 Inter-Agent Message Format

**Standard message envelope:**

```
{
  "message_id": "uuid",
  "timestamp": "ISO8601",
```

```
  "from_agent": "agent-name",
  "to_agent": "agent-name",
  "message_type": "request|response|error",
  "correlation_id": "uuid",
  "payload": {},
  "metadata": {
    "tier": "sonnet",
    "priority": "normal|high",
    "timeout_seconds": 300
  }
}
```

### 6.3 Memory Query Interface

**Function signature:**

```
def query_memory(
    query: str,
    top_k: int = 5,
    category: Optional[str] = None,
    tags: Optional[List[str]] = None,
    min_confidence: Optional[str] = None,
    since: Optional[date] = None
) -> List[MemoryResult]
```

**CLI interface:**

```
python3 ~/.claude/scripts/query-memory-bm25.py \
  "authentication JWT refresh" \
  --top-k 5 \
  --category decisions \
  --json
```

**Result format:**

```
[
  {
    "path": "decisions/2026-01-13-jwt-strategy.md",
    "title": "Decision: JWT Token Refresh Strategy",
    "category": "decisions",
    "tags": ["authentication", "jwt", "security"],
    "score": 8.234,
    "preview": "Implemented sliding window refresh with..."
  }
]
```

---

## Summary

This technical specification provides the concrete implementation details needed to build the Lisan al-Gaib architecture:

1. **Thresholds** — All numeric values with justification and adjustment guidance
2. **Schemas** — Complete JSON Schema definitions for all data structures
3. **Hooks** — Specification for each lifecycle hook with exit codes and behavior
4. **State files** — Format and location for all ephemeral state
5. **Directory structure** — Complete annotated file tree with conventions
6. **APIs** — Contracts for Gemini CLI, inter-agent messaging, and memory queries

These specifications should be treated as the authoritative reference during implementation. Deviations should be documented and justified.

---

*Continue to Part VII: Tradeoff Matrices & Decision Frameworks → #*
Part VII: Tradeoff Matrices & Decision Frameworks

# Structured Decision Support for Architectural Choices

---

## 1. RAG Implementation Tradeoffs

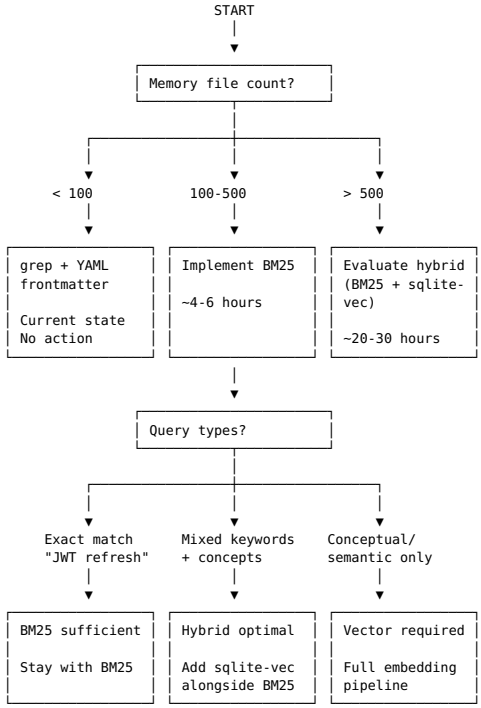
### 1.1 Retrieval Method Comparison

| Method | Recall (top-20) | Latency | Dependencies | Best For |
|---|---|---|---|---|
| grep + frontmatter | ~75% | <10ms | None | <100 files, exact matches |
| BM25 | 89% | 20-50ms | rank_bm25 | 100-1000 files, keyword |
| Vector | | 50- | Embedding | >1000 files, |

| | | | | |
|---|---|---|---|---|
| embeddings | 91.7% | 200ms | model, DB | semantic |
| Hybrid (BM25 + vector) | 93-95% | 100-300ms | Both | Large scale, mixed queries |

*Source: XetHub benchmark, 2024*

### 1.2 Decision Tree: When to Upgrade Retrieval




### 1.3 Implementation Effort vs. Benefit

| Upgrade | Effort | Recall Improvement | When Worth It |
|---|---|---|---|
| grep → BM25 | 6 hours | +14% | Always (>100 files) |
| BM25 → sqlite-vec | 15 hours | +3% | >500 files, semantic queries |
| sqlite-vec → full hybrid | 20 hours | +2% | >2000 files, mixed queries |

**Recommendation:** Implement BM25 in Phase 2. Defer vector embeddings until memory exceeds 500 files or semantic retrieval failures exceed 10% of queries.

---

## 2. Framework Adoption Tradeoffs

### 2.1 Component-by-Component Assessment

| Component | Adopt? | Rationale | Risk Level |
|---|---|---|---|
| `langchain-anthropic` | ✅ Consider | Clean provider abstraction | Low |
| `langchain-openai` | ✅ Consider | Multi-provider flexibility | Low |
| LlamaIndex retrievers | ✅ Consider | Advanced retrieval patterns | Low |
| LangGraph StateGraph | ❌ Avoid | Infrastructure conflicts | High |
| LangChain memory | ❌ Avoid | Token-inefficient | Medium |
| LangChain chains | ❌ Avoid | Debugging complexity | High |
| LangSmith | ✅ Consider | Observability value | Low |

### 2.2 Adoption Decision Tree

```
                    ┌──────────────────────┐
                    │ Component considered? │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ Does it require      │
                    │ PostgreSQL/Redis?    │
                    └──────────────────────┘
                              │
                ┌─────────────┴─────────────┐
                │ YES                       │ NO
                ▼                           ▼
    ┌──────────────────┐        ┌──────────────────────┐
    │ REJECT           │        │ Does it handle state │
    │                  │        │ we already manage?   │
    │ Conflicts with   │        └──────────────────────┘
    │ file-based       │                   │
    │ philosophy       │
┌───┴──────────────────┴──────────┐
│                                 │ YES
│ NO                              ▼
▼
                    ┌──────────────────┐
┌──────────────────┐│ REJECT           │                │ Is it
isolated/       │
                    │                  │                │ stateless
utility?    │
                    │ Duplicates       │
└──────────────────┘│
                    │ existing         │                          │
                    │ capacity         │
┌──────────────────┴──────────────────┐
                    └──────────────────┘          │ YES
│ NO
▼
                                                  ▼
┌──────────────────┐                  ┌──────────────────┐
                                       │ CONSIDER         │
│ EVALUATE         │
                                       │                  │
│ CAREFULLY        │
                                       │ Low risk,        │
│                  │
                                       │ useful utility   │
│ May introduce    │
                                       └──────────────────┘
│ coupling         │
└──────────────────┘
```

### 2.3 Integration Complexity Ratings

| Integration | Complexity | Hours | Reversibility |
|---|---|---|---|
| langchain-anthropic adapter | Low | 2-4 | Easy |
| LangSmith tracing | Low | 1-2 | Easy |
| LlamaIndex BM25 retriever | Medium | 8-12 | Moderate |
| LlamaIndex hybrid search | Medium | 15-20 | Moderate |
| LangGraph for specific workflow | High | 20-40 | Difficult |
| Full LangChain adoption | Very High | 80-120 | Very Difficult |

**Recommendation:** Adopt only isolated utilities (adapters, observability). Never adopt state management or orchestration components.

---

## 3. Parallelization Tradeoffs

### 3.1 When Parallel Processing Helps

| Task Type | Speedup | Quality Impact | Recommendati |
|---|---|---|---|
| Document extraction | 1.4-3x | Neutral | ✅ Parallelize |
| Structured data parsing | 2-4x | Neutral | ✅ Parallelize |
| Multi-file analysis | 1.5-2x | Slight improvement | ✅ Parallelize |
| Creative generation | 0.8-1.0x | Degraded | ❌ Sequential |
| Synthesis/summarization | 1.0-1.5x | Risk of gaps | ⚠ Overlapping windows |
| Reasoning chains | 0.5-0.8x | Degraded | ❌ Sequential |

### 3.2 Parallelization Decision Matrix

```
┌──────────────
```

| | | | |
|---|---|---|---|
| OpenAI | 500 RPM | 10 workers | Linear (1s increments) |

```python
# Semaphore pattern for rate limiting
import asyncio

GEMINI_SEMAPHORE = asyncio.Semaphore(5)

async def rate_limited_call(func, *args):
    async with GEMINI_SEMAPHORE:
        return await func(*args)
```

# 4. Autonomy Level Tradeoffs

## 4.1 Risk vs. Efficiency by Level

| Level | Human Effort | Error Rate Risk | Suitable Tasks |
|---|---|---|---|
| L1 Operator | 100% | Baseline | All (initial) |
| L2 Collaborator | 50-70% | Baseline | Suggestions visible |
| L3 Consultant | 20-40% | +0.5-1% | High-confidence routine |
| L4 Approver | 5-15% | +1-2% | Well-defined routine |
| L5 Observer | <5% | +2-5% | Fully automated domain |

## 4.2 Category-Specific Recommendations

| Decision Category | Max Autonomy | Rationale |
|---|---|---|
| Quick approval (simple tasks) | L5 | Low risk, high frequency |
| Routing decisions | L4 | Formula-based, auditable |
| Scope modifications | L3 | Requires judgment |
| Security-related | L2 | High consequence |
| Architecture changes | L2 | High consequence |
| Memory curation | L3 | Reversible, moderate impact |

## 4.3 Promotion vs. Demotion Thresholds

```
┌──────────────
```

### 4.4 Rollback Triggers

| Trigger | Action | Recovery |
|---|---|---|
| Single failure at L5 | Demote to L4 | 50 successes to re-promote |
| 3 failures at L4 | Demote to L3 | 100 successes to re-promote |
| Success rate drops 5% | Demote one level | Rebuild success history |
| Human override | Immediate demotion | Per human specification |
| Schema change | Reset to L2 | Re-learn patterns |

---

## 5. Cost Optimization Tradeoffs

### 5.1 Tier Routing Sensitivity Analysis

| Routing Strategy | Avg Cost/Session | Quality Impact | When to Use |
|---|---|---|---|
| Always Haiku | $0.25-0.50 | -30% quality | Never (too aggressive) |
| Formula-based (current) | $3-5 | Baseline | Default |
| Always Sonnet | $5-10 | +5% quality | Quality-critical periods |
| Aggressive Opus | $15-30 | +10% quality | Complex refactors only |

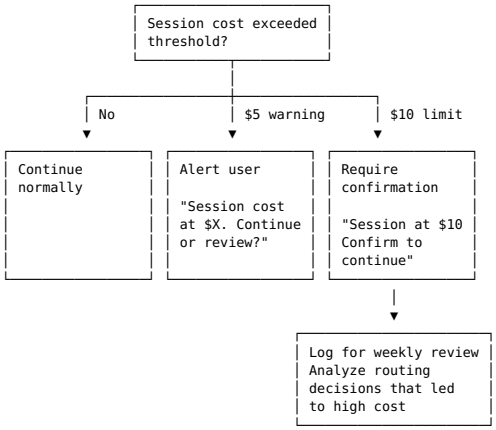
### 5.2 Cost Reduction Levers

| Lever | Cost Impact | Implementation Effort | Risk |
|---|---|---|---|
| Scout-first protocol | -20-40% | Already implemented | Low |
| Stricter Opus threshold | -15-25% | 1 hour (config change) | Medium |
| Memory caching | -10-20% | 8-12 hours | Low |
| Request batching | -5-15% | 4-6 hours | Low |
| Context compression | -20-30% | 20-30 hours | Medium |

### 5.3 Weekly Review Budget Allocation

| Component | Budget | Rationale |
|---|---|---|
| Memory Synthesis (Sonnet) | $0.30 | Aggregation task |
| Systems Architect (Opus) | $1.50 | Complex analysis |
| Schema Discovery (Sonnet) | $0.50 | Pattern detection |
| Buffer | $0.20 | Unexpected needs |
| **Total** | **$2.50** | Infrastructure investment |

### 5.4 Cost Monitoring Decision Framework




### 5.5 Shadow Deployment Cost Ceiling

| Scenario | Max Shadow Cost | Rationale |
|---|---|---|
| New agent validation | $5 over shadow period | Bounded investment |
| Routing threshold | $2 per A/B | Quick validation |

| | | |
|---|---|---|
| test | comparison | |
| Schema migration test | $3 per migration | One-time cost |

**Rule:** Shadow deployments that exceed 2x expected cost are auto-terminated and flagged for review.

---

## 6. Decision Framework Summary

### 6.1 Quick Reference Decision Table

| Decision | Default Choice | Reconsider When |
|---|---|---|
| Retrieval method | BM25 | >500 files or semantic failures |
| Framework adoption | Bespoke | Never for orchestration |
| Parallelization | Sequential | >50K tokens OR extraction task |
| Autonomy level | L2 start | Success thresholds met |
| Cost optimization | Formula routing | Budget pressure |

### 6.2 Reversibility Assessment

| Decision | Reversibility | Lock-in Risk |
|---|---|---|
| Add BM25 retrieval | Easy | None |
| Add vector embeddings | Moderate | Data migration |
| Adopt LangGraph | Difficult | High |
| Increase autonomy | Easy | None |
| Decrease thresholds | Easy | None |
| Shadow new agent | Easy | None |
| Promote agent to production | Moderate | Dependent systems |

### 6.3 Decision Logging

All significant architectural decisions should be logged:

```
# .claude/memory/decisions/YYYY-MM-DD-decision-topic.md
---
title: "Decision: [Brief title]"
created: YYYY-MM-DD
category: decisions
tags: [architecture, tradeoff, relevant-area]
status: active
summary: "One-line summary of what was decided"
---

## Context
Why this decision was needed.

## Options Considered
1. Option A - pros/cons
2. Option B - pros/cons

## Decision
What was chosen.

## Rationale
Why this option was selected.

## Consequences
What changes as a result.

## Review Trigger
When to revisit this decision.
```

---

*Continue to Part VIII: Risk Assessment & Mitigation* → # Part VIII: Risk Assessment & Mitigation

# Comprehensive Risk Analysis and Management Strategies

---

## 1. Risk Assessment Framework

### 1.1 Risk Scoring Matrix

| Impact | Likelihood: Low | Likelihood: Medium | Likelihood: High |
|---|---|---|---|
| **High** | Medium Risk | High Risk | Critical Risk |
| **Medium** | Low Risk | Medium Risk | High Risk |

| | | | |
|---|---|---|---|
| **Low** | Minimal Risk | Low Risk | Medium Risk |

### 1.2 Risk Categories

| Category | Scope | Examples |
|---|---|---|
| Technical | Architecture, implementation | State corruption, integration failures |
| Operational | Day-to-day function | Cost overruns, performance degradation |
| Strategic | Long-term viability | Market shifts, dependency obsolescence |

---

## 2. Technical Risks

### 2.1 Risk: Subagent Proliferation

**Description:** Uncontrolled growth of spawned agents leads to coordination overhead, increased complexity, and diminishing returns.

**Likelihood:** Medium **Impact:** High **Overall:** High Risk

**Evidence:** Research shows coordination overhead dominates with >4 agents operating simultaneously. Error amplification reaches 17.2× with independent agents.

**Indicators:** - Agent count approaching 15 (hard cap) - New agent spawn blocked due to limit - Coordination overhead >30% of task time - Success rates declining despite more agents

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Hard cap enforcement | agents-index.json validates count | Phase 5 | | Deprecation policy | 30-day unused → review | Phase 5 | | Success rate gate | <50% over 20 invocations → deprecate | Phase 5 | | Spawn requires deprecation | 1-in-1-out when at cap | Policy |

**Contingency:** If proliferation occurs despite controls, freeze spawning and conduct agent consolidation review. Merge overlapping capabilities.

---

### 2.2 Risk: Schema Migration Failures

**Description:** Schema version changes break compatibility with existing observations or memory files, causing data loss or retrieval failures.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Evidence:** Schema versioning is a known pain point in production systems. Breaking changes to required fields can invalidate historical data.

**Indicators:** - Validation errors on old memory files - Retrieval returning zero results unexpectedly - Pattern detection failing on backfill

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | BACKWARD_TRANSITIVE compatibility | Never remove required fields | Policy | | Upcasting on read | Transform old → new at retrieval | Phase 4 | | Schema version in all files | `schema_version` field required | Phase 2 | | Migration scripts | Automated upgrade paths | Phase 7 | | Backup before migration | Copy memory/ before schema change | Process |

**Contingency:** If migration fails, restore from backup. Schema changes should be staged: test on copy, then apply.

---

### 2.3 Risk: State Corruption

**Description:** Invalid or corrupted state files cause routing failures, agent handoff errors, or memory integrity issues.

**Likelihood:** Low **Impact:** High **Overall:** Medium Risk

**Evidence:** File-based state is vulnerable to partial writes, concurrent access, and malformed JSON.

**Indicators:** - JSON parse errors in hooks - Validation failures in SubagentStop - Inconsistent routing decisions

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Atomic writes | Write to temp, then rename | All scripts | | Schema validation | Pydantic models for state files | Phase 0 | | Checksum verification | SHA256 in memory files | Phase 2 | | Graceful degradation | Default values if state missing | Hooks |

**Contingency:** Clear `.claude/tmp/` and restart session. State is ephemeral by design.

---

### 2.4 Risk: Gemini Integration Failures

**Description:** Gemini CLI failures (rate limits, API changes, outages) block large-context operations.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Indicators:** - Gemini calls timing out - Rate limit errors increasing - API response format changes

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Exponential backoff | 1s, 2s, 4s, 8s retry | gemini-cli | | Fallback to chunked Claude | If Gemini unavailable, chunk for Opus | Routing | | Rate limit monitoring | Track 429 responses | Logging | | API version pinning | Specify API version in config | Config |

**Contingency:** If Gemini is unavailable, degrade to chunked processing with Claude Opus. Higher cost but maintains functionality.

---

### 2.5 Risk: Hook Execution Failures

**Description:** Hook scripts fail to execute (permissions, dependencies, bugs), breaking the enforcement layer.

**Likelihood:** Low **Impact:** High **Overall:** Medium Risk

**Indicators:** - Tools executing without routing checks - Missing log entries - Hook exit codes not respected

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Hook health check | Verify hooks exist and executable | SessionStart | | Fallback defaults | Conservative routing if hook fails | PreToolUse | | Dependency verification | Check jq, bc availability | Install | | Error logging | All hook errors to stderr and log | All hooks |

**Contingency:** If hooks fail, system should fail closed (block operations) rather than fail open (allow unrestricted access).

---

## 3. Operational Risks

### 3.1 Risk: Cost Overruns

**Description:** Session costs exceed budget due to routing failures, excessive Opus usage, or runaway parallel workers.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Evidence:** LangChain documented 2.7× cost overruns. Without monitoring, costs are invisible until billing.

**Indicators:** - Session cost warnings triggered - Opus usage per session >5 calls - Weekly cost exceeding $50

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Real-time cost tracking | Estimate in PreToolUse, actual in PostToolUse | Phase 1 | | Session cost warnings | Alert at $5, require confirmation at $10 | Phase 1 | | Weekly cost reports | Automated aggregation | Phase 1 | | Opus usage alerts | Warn if >5 Opus calls in session | Routing | | Shadow cost ceiling | Auto-terminate at 2× expected | Phase 5 |

**Contingency:** If cost overrun detected, pause session and review routing decisions. Adjust thresholds if systematic.

---

### 3.2 Risk: Human-in-the-Loop Fatigue

**Description:** Excessive approval requests or feedback prompts cause user to disengage, providing low-quality input or bypassing controls.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Evidence:** Research shows >1 explicit prompt per 10 interactions causes fatigue. Implicit feedback should be >90% of signal.

**Indicators:** - Approval times decreasing (rubber-stamping) - Feedback quality declining - User complaints about interruptions - Override rate increasing without pattern

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Implicit signal priority | Track behavior, not just clicks | Phase 2 | | Batch approvals | Group related decisions | Weekly review | | Progressive trust | Automate as competence proven | Phase 6 | | Friction-free feedback | Optional, ≤1 click | All UI | | Review frequency tuning | Weekly default, adjustable | Config |

**Contingency:** If fatigue detected, reduce explicit prompts. Accept lower-fidelity implicit signals.

---

### 3.3 Risk: Memory Bloat

**Description:** Memory directory grows unbounded, degrading retrieval performance and increasing storage.

**Likelihood:** Low **Impact:** Low **Overall:** Low Risk

**Indicators:** - Memory file count >500 - Retrieval latency increasing - Disk space warnings

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | File count monitoring | Alert at 500, hard warning at 1000 | Weekly review | | Stale content archival | >90 days without access → archive | Phase 7 | | Observation consolidation | Compress into patterns after schema activation | Phase 4 | | Deduplication | Detect similar content | Future |

**Contingency:** Manual curation sprint. Archive low-value content to cold storage.

---

### 3.4 Risk: Observability Blind Spots

**Description:** Insufficient logging or metrics leave performance issues, routing problems, or cost drivers undetected.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Indicators:** - Unable to explain cost spike - Cannot reproduce routing decision - No data for threshold tuning

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | OpenTelemetry enablement | Environment variable | Phase 0 | | Comprehensive routing log | All decisions with context | Phase 1 | | Session summaries | Aggregated metrics per session | Phase 1 | | Weekly reports | Cost, success rate, routing breakdown | Phase 1 | | Decision capture | Full context for learning | Phase 2 |

**Contingency:** If blind spot discovered, add targeted logging immediately. Prioritize observability debt.

---

## 4. Strategic Risks

### 4.1 Risk: Model Capability Shifts

**Description:** Claude or Gemini capabilities change significantly, invalidating routing assumptions or tier assignments.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Evidence:** Model capabilities improve rapidly. Today's Sonnet may exceed yesterday's Opus. Pricing changes affect cost optimization.

**Indicators:** - Routing thresholds feel outdated - Tier cost assumptions invalid - New model releases

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Threshold configurability | All thresholds in routing-schema.json | Architecture | | Tier abstraction | Reference tiers by capability, not model name | Architecture | | Quarterly threshold review | Validate against current models | Process | | Model version tracking | Log model used per session | Logging |

**Contingency:** Rapid threshold recalibration. Architecture supports this by design.

---

### 4.2 Risk: Dependency Obsolescence

**Description:** Key dependencies (Claude Code hooks, Gemini CLI) change APIs or become unavailable.

**Likelihood:** Low **Impact:** High **Overall:** Medium Risk

**Evidence:** Claude Code hook system is documented but could change. Gemini CLI is external dependency.

**Indicators:** - Hook execution method changes - CLI interface breaks - Deprecation notices

**Mitigation:** | Action | Implementation | Owner | |——-|————-|——-| | Minimal dependencies | Prefer standard tools (bash, jq) | Architecture | | Abstraction layers | Wrap external calls | Scripts | | Version pinning | Document compatible versions | Docs | | Alternative identification | Know fallback options | Planning |

**Contingency:** Fork and maintain critical dependencies if necessary. Architecture is designed for minimal lock-in.

---

### 4.3 Risk: Competitive Pressure

**Description:** Alternative solutions (managed agent platforms, improved frameworks) reduce value proposition.

**Likelihood:** Medium **Impact:** Low **Overall:** Low Risk

**Evidence:** Agent frameworks are actively developing. Cloud providers building managed solutions.

**Indicators:** - Framework capabilities converging - Managed solutions matching cost efficiency - User migration away from bespoke

**Mitigation:** | Action | Implementation | Owner | |——–|————-|——-| | Differentiation focus | Novel IP (schema discovery, spawning) | Strategy | | Transparency advantage | Maintain file-based auditability | Architecture | | Local-first commitment | No cloud dependency requirement | Architecture | | Continuous evolution | Weekly reviews drive improvement | Operations |

**Contingency:** This risk is acceptable. Core value proposition (transparency, control, evolution) is defensible.

---

### 4.4 Risk: IP Protection Gaps

**Description:** Novel innovations are replicated without protection, reducing competitive advantage.

**Likelihood:** Medium **Impact:** Medium **Overall:** Medium Risk

**Indicators:** - Similar approaches appearing in literature - Framework features mimicking innovations - No formal IP protection filed

**Mitigation:** | Action | Implementation | Owner | |——–|————-|——-| | Documentation dating | This guide establishes prior art | Immediate | | Provisional patents | File for emergent schema, spawning | Near-term | | Publication consideration | Academic paper for prior art | Consider | | Trade secret identification | Keep specific implementations private | Ongoing |

**Contingency:** If replication occurs, prior art documentation supports defensive position.

---

## 5. Risk Register Summary

| Risk ID | Risk | Category | Likelihood | Impact | Overall |
|---|---|---|---|---|---|
| T-1 | Subagent Proliferation | Technical | Medium | High | High |
| T-2 | Schema Migration Failures | Technical | Medium | Medium | Medium |
| T-3 | State Corruption | Technical | Low | High | Medium |
| T-4 | Gemini Integration Failures | Technical | Medium | Medium | Medium |
| T-5 | Hook Execution Failures | Technical | Low | High | Medium |
| O-1 | Cost Overruns | Operational | Medium | Medium | Medium |
| O-2 | HITL Fatigue | Operational | Medium | Medium | Medium |
| O-3 | Memory Bloat | Operational | Low | Low | Low |
| O-4 | Observability Blind Spots | Operational | Medium | Medium | Medium |
| S-1 | Model Capability Shifts | Strategic | Medium | Medium | Medium |
| S-2 | Dependency Obsolescence | Strategic | Low | High | Medium |
| S-3 | Competitive Pressure | Strategic | Medium | Low | Low |
| S-4 | IP Protection Gaps | Strategic | Medium | Medium | Medium |

---

## 6. Mitigation Implementation Priority

### Phase 0-1 (Foundation)

- T-3: Atomic writes, schema validation
- T-5: Hook health checks
- O-4: OpenTelemetry, comprehensive logging
- O-1: Cost tracking infrastructure

### Phase 2-3 (Capability Building)

- T-2: Schema versioning infrastructure
- O-2: Implicit signal priority
- O-3: File count monitoring

### Phase 4-5 (Evolution Features)

- T-1: Agent population controls
- T-4: Gemini fallback patterns

### Phase 6-7 (Maturity)

- T-2: Automated migration scripts
- O-3: Automated archival

### Ongoing

- S-1: Quarterly threshold review
- S-4: IP documentation maintenance

---

## 7. Monitoring Dashboard Recommendations

### Key Risk Indicators to Track

| Indicator | Warning | Critical | Data Source |
|---|---|---|---|
| Session cost | >$5 | >$10 | routing_log |
| Opus calls/session | >5 | >10 | routing_log |
| Agent count | >12 | >15 | agents-index |
| Memory file count | >500 | >1000 | filesystem |
| Hook failure rate | >1% | >5% | validation_log |
| Routing block rate | >20% | >40% | routing_log |
| Success rate | <90% | <80% | session_summaries |

### Weekly Review Checklist

☐ Session cost within budget?
☐ Routing thresholds effective?
☐ Agent success rates healthy?
☐ Memory growth controlled?
☐ Any new sharp edges?
☐ Override patterns meaningful?
☐ Observability sufficient?

---

*Continue to Part IX: Further Research & Open Questions* → # Part IX: Further Research & Open Questions

## Exploration Priorities and Unresolved Architectural Decisions

---

## 1. Immediate Research Priorities (Months 1-3)

### 1.1 Embedding Model Selection for Hybrid RAG

**Question:** When BM25 proves insufficient, which embedding model provides best cost/performance tradeoff for local deployment?

**Candidates:** | Model | Dimensions | Speed | Quality | License | |——-|————|——-|———|———| | all-MiniLM-L6-v2 | 384 | Fast | Good | Apache 2.0 | | bge-small-en-v1.5 | 384 | Fast | Better | MIT | | nomic-embed-text-v1 | 768 | Medium | Best | Apache 2.0 | | mxbai-embed-large | 1024 | Slow | Best | Apache 2.0 |

**Research tasks:** - [ ] Benchmark recall on sample memory corpus - [ ] Measure embedding generation latency on target hardware - [ ] Evaluate storage requirements per model - [ ] Test hybrid (BM25 + embedding) vs pure embedding

**Success criteria:** Identify model achieving >90% recall with <100ms embedding time per query.

**Priority:** High (blocks Phase 4 hybrid retrieval)

---

### 1.2 sqlite-vec vs LanceDB Evaluation

**Question:** Which vector storage solution best fits the local-first, minimal-dependency philosophy?

**Comparison:** | Aspect | sqlite-vec | LanceDB | |——–|————|———| | Dependencies | Zero (pure C) | Rust runtime | | Disk format | SQLite extension | Lance columnar | | Query speed | Good | Excellent | | Filtering | SQL native | Custom | | Maturity | Newer | Established | | Maintenance | Single developer | Funded company |

**Research tasks:** - [ ] Install and test both on target environment - [ ] Benchmark query performance at 500, 1000, 5000 vectors - [ ] Evaluate hybrid search capabilities (vector + metadata filter) - [ ] Assess maintenance/update burden

**Success criteria:** Select solution with <50ms query time, no external service dependencies.

**Priority:** Medium (supports Phase 4 but not blocking)

---

### 1.3 Gemini Rate Limit Characterization

**Question:** What are actual rate limits and optimal concurrency for Gemini Flash in parallel worker scenarios?

**Known:** - Documented: 60 RPM for free tier - Unknown: Burst behavior, token-based limits, error recovery patterns

**Research tasks:** - [ ] Empirically test sustained throughput - [ ] Characterize rate limit error responses - [ ] Identify optimal backoff parameters - [ ] Test token-based vs request-based limits

**Success criteria:** Document reliable concurrency level and backoff strategy for parallel synthesis.

**Priority:** High (blocks Phase 4-5 parallel processing)

---

### 1.4 Context Compression Techniques

**Question:** Can context compression reduce costs without degrading quality?

**Candidates:** | Technique | Compression | Quality Retention | Implementation | |———–|————-|——————-|————-| | ACON framework | 26-54% | 95%+ | Medium | | LLMLingua | 20-40% | 90%+ | High | | Selective context | Variable | 85-95% | Low | | Summary caching | 50-80% | 80-90% | Medium |

**Research tasks:** - [ ] Implement selective context (drop low-relevance memory) - [ ] Test ACON on sample conversations - [ ] Measure quality degradation per compression level - [ ] Identify break-even point (cost savings vs quality loss)

**Success criteria:** Achieve 30% context reduction with <5% quality degradation.

**Priority:** Medium (cost optimization enhancement)

---

## 2. Medium-Term Exploration (Months 4-6)

### 2.1 RouteLLM Integration Feasibility

**Question:** Can RouteLLM's learned routing provide better cost/quality tradeoff than formula-based routing?

**Background:** RouteLLM demonstrates 85% cost reduction while maintaining 95% performance through learned routing. Current formula is deterministic but may be suboptimal.

**Research approach:** 1. Collect routing decisions with outcomes (already planned in Phase 1) 2. Train RouteLLM classifier on historical decisions 3. A/B test learned routing vs formula routing 4. Measure cost and quality differential

**Considerations:** - Adds ML dependency (acceptable if isolated) - Requires training data accumulation period - May conflict with code-enforced routing philosophy

**Decision framework:** - If learned routing shows >15% cost improvement with <2% quality loss → integrate - If improvement <10% → maintain formula (simpler, more transparent)

**Priority:** Medium (optimization, not critical path)

---

### 2.2 Constitutional AI Principles Adaptation

**Question:** Can Constitutional AI principles improve agent self-correction and reduce human oversight needs?

**Concept:** Define principles as guardrails. Enable agents to self-critique against principles. Train on revised outputs.

**Potential principles for Lisan:** 1. "Prefer lower-cost tier unless quality requirements demand escalation" 2. "Validate assumptions before proceeding with complex operations" 3. "Flag uncertainty explicitly rather than proceeding with low confidence" 4. "Preserve context boundaries in multi-document synthesis" 5. "Capture decision rationale for future learning"

**Research approach:** 1. Define initial principle set (5-10 principles) 2. Implement self-critique prompt in agent definitions 3. Measure impact on routing violations, task failures, human overrides 4. Iterate on principle definitions

**Success criteria:** Reduce human overrides by 20% through principle-guided self-correction.

**Priority:** Medium (quality improvement, supports autonomy progression)

---

### 2.3 Multi-Agent RLHF Implementation

**Question:** How can captured human decisions be used to fine-tune agent behavior beyond pattern matching?

**Current approach:** Apprenticeship learning via pattern matching on captured decisions.

**RLHF enhancement:** Use decisions as preference data for reward model training.

**Challenges:** - Requires substantial decision volume (1000+) - Fine-tuning not available for API models - Proxy approaches (prompt optimization) may suffice

**Research approach:** 1. Accumulate decision corpus through Phases 2-3 2. Evaluate prompt optimization techniques (DSPy, automatic prompt engineering) 3. If effective: implement prompt evolution based on decision outcomes 4. If insufficient: document requirements for future fine-tuning capability

**Success criteria:** Measurable improvement in system recommendations matching human decisions.

**Priority:** Low (long-term enhancement)

---

### 2.4 Distributed Execution Patterns

**Question:** How should architecture evolve if workloads exceed single-machine capacity?

**Current assumption:** All execution on developer machine (local-first).

**Scaling scenarios:** 1. Multiple developers sharing memory (knowledge base) 2. Long-running background tasks (overnight analysis) 3. High-throughput batch processing

**Research areas:** - Shared memory synchronization (git-based already supports) - Background task queue (simple file-based job queue) - Distributed agent coordination (not needed near-term)

**Priority:** Low (premature optimization risk)

---

## 3. Long-Term Research (Months 7-12)

### 3.1 Full Automation Pathways

**Question:** What is the realistic ceiling for autonomous operation?

**Hypothesis:** Level 5 (Observer) autonomy is achievable for well-defined, low-risk decision categories. Most domains will plateau at Level 3-4.

**Research questions:** - What decision categories can safely reach L5? - What safeguards prevent catastrophic autonomous errors? - How to detect novel situations requiring human escalation? - What feedback loops maintain automation quality over time?

**Long-term experiments:** 1. Track L4 categories for extended periods (6+ months) 2. Measure drift in automated decision quality 3. Identify early warning indicators for degradation 4. Document domain-specific automation boundaries

**Priority:** Low (requires foundation from earlier phases)

---

### 3.2 Cross-Session Learning Optimization

**Question:** How to maximize learning from limited human interaction across many sessions?

**Current:** Each session captures observations; weekly review synthesizes.

**Optimization opportunities:** - Real-time pattern matching (surface relevant history immediately) - Active learning (identify high-value decisions to capture explicitly) - Transfer learning across project contexts - Collaborative learning across multiple users (if applicable)

**Research approach:** 1. Instrument learning effectiveness metrics 2. Identify bottlenecks in current learning pipeline 3. Prototype targeted improvements 4. Measure impact on autonomy progression speed

**Priority:** Low (optimization layer)

---

### 3.3 Emergent Capability Detection

**Question:** How to detect when the system has developed capabilities not explicitly designed?

**Context:** Complex adaptive systems can exhibit emergent behaviors. Lisan's evolution mechanisms (schema discovery, agent spawning) could produce unexpected capabilities.

**Monitoring needs:** - Capability inventory (what can each agent do?) - Usage pattern analysis (what is being used that wasn't designed?) - Quality assessment of emergent behaviors - Safety evaluation of new capabilities

**Priority:** Low (philosophical, long-term governance)

---

## 4. Academic References

### 4.1 Key Papers to Monitor

**Multi-Agent Systems:** - "MAST: A Multi-Agent System Testing Taxonomy" (UC Berkeley, 2025) — Failure mode classification - "MegaAgent: Autonomous Cooperation" (2024) — File-based state persistence patterns - "Agent2Agent Protocol" (Google/Linux Foundation) — Standardization efforts

**Retrieval and RAG:** - "Evaluation of Chunking Strategies" (NVIDIA FinanceBench, 2024) — Optimal chunking parameters - "ColBERT v2" (Stanford) — Late interaction retrieval - "Contextual Retrieval" (Anthropic, 2024) — BM25 + embeddings hybrid

**Routing and Efficiency:** - "RouteLLM: Learning to Route LLMs" (2024) — Learned routing classification - "FrugalGPT" (Stanford, 2023) — Cascade routing patterns - "ACON: Adaptive Context Optimization" (2024) — Context compression

**Human-AI Collaboration:** - "Levels of Automation" (Sheridan & Verplank) — Autonomy taxonomy - "Constitutional AI" (Anthropic, 2022) — Principle-based self-correction - "Active Learning from Human Feedback" — Decision capture strategies

### 4.2 Research Groups to Follow

| Group | Focus | Relevance |
|---|---|---|
| Anthropic Research | AI safety, capabilities | Primary model provider |
| Google DeepMind | Multi-agent, large context | Gemini integration |
| Stanford HAI | Human-AI interaction | Autonomy frameworks |
| UC Berkeley AI | Agent architectures | Failure taxonomies |
| LangChain Research | Framework patterns | Anti-patterns to avoid |

### 4.3 Conference Tracks of Interest

| Conference | Track | Timing |
|---|---|---|
| NeurIPS | AI Agents workshop | December |
| ICML | Multi-agent systems | July |
| ACL | Retrieval-augmented generation | Summer |
| CHI | Human-AI collaboration | Spring |
| AAAI | Agent architectures | February |

---

## 5. Open Design Questions

### 5.1 Unresolved Architectural Decisions

**Question 1: Memory Expiration Policy** - Should facts automatically expire? - How to handle contradictory facts (old vs new)? - What triggers re-verification of established facts?

**Current thinking:** Facts should have optional `expires` field. Contradictions surface in weekly review. Verification via explicit user confirmation or external validation.

**Question 2: Agent Communication Protocol** - Direct message passing vs shared state file? - Synchronous vs asynchronous handoffs? - How to handle agent timeout mid-task?

**Current thinking:** File-based (current approach) is simpler and auditable. Async by default. Timeout triggers handoff failure and human escalation.

**Question 3: Schema Evolution Coordination** - Who decides when to evolve schemas? - How to coordinate across active sessions? - What's the rollback strategy?

**Current thinking:** Weekly review proposes; human approves. Session restart required for schema changes. Git-based rollback.

**Question 4: Multi-User Memory Sharing** - Should memory be shared across team members? - How to resolve conflicting preferences? - What's the privacy model?

**Current thinking:** Out of scope for solo dev. Git-based sharing technically possible. Would require explicit namespacing.

### 5.2 Experimentation Recommendations

**A/B Testing Candidates:**

| Experiment | Hypothesis | Metric | Priority |
|---|---|---|---|
| Opus threshold 10 vs 12 | Higher threshold reduces cost | Cost per session | High |
| Scout-first vs direct | Scout saves money on simple tasks | Cost, latency | Medium |
| BM25 k=3 vs k=5 vs k=10 | More results improve context | Task success | Medium |
| Weekly vs biweekly review | Less frequent review is sufficient | Learning velocity | Low |

**Recommended approach:** 1. Implement simple A/B infrastructure (config flag + logging) 2. Run experiments for minimum 2 weeks each 3. Measure with statistical significance 4. Document results in memory/decisions/

### 5.3 Parking Lot (Ideas to Revisit)

| Idea | Why Parked | Revisit When |
|---|---|---|
| Voice interface | Complexity, unclear value | User requests |
| IDE integration | Scope creep | Core stable |
| Cloud backup | Conflicts with local-first | User requests |
| Real-time collaboration | Multi-user complexity | Team adoption |
| Mobile interface | Platform dependency | User requests |

---

## 6. Research Roadmap Summary

```
┌──────────────
```

```
## Research Question
What we were trying to learn.

## Methodology
How we investigated.

## Findings
What we discovered.

## Decision
What we're doing based on findings.

## Open Questions
What remains unresolved.

## References
Sources consulted.
```

---

*Continue to Part X: Appendices* → # Part X: Appendices

# Reference Material and Quick Access Resources

---

## Appendix A: Complete System Diagrams

### A.1 Full Architecture Overview

```
┌──────────────
```

```
│  │  │  haiku-scout │ architect │ memory-archivist │ memory-
synthesis │ ...    │ │ │
│  │  │
│ │ │
│  │
└──────────────────────────────────────────────────────────────────────────────
 │ │
│  │
│ │
│
└──────────────────────────────────────────────────────────────────────────────
 │
│
│
└──────────────────────────────────────────────────────────────────────────────
                                        │

                                        │

┌──────────────────────────────┼──────────────────────────────┐
       │                                │
│
       ▼                                ▼
▼
┌──────────────────┐          ┌──────────────────┐
┌──────────────────┐
│   ENFORCEMENT    │          │      STATE       │         │
MEMORY        │
│      LAYER       │          │      LAYER       │         │
LAYER        │
│                  │          │                  │         │
│
│  .claude/hooks/  │          │  .claude/tmp/    │         │
.claude/memory/  │
│                  │          │                  │         │
│
│  ┌─────────────┐ │          │  ┌────────────┐  │         │
┌─────────────┐ │
│  │ PreToolUse  │ │          │  │scout_metrics│  │         │  │
decisions/  │  │
│  │ PostToolUse │ │          │  │complexity  │  │         │  │
sharp-edges/│  │
│  │ SubagentStop│ │          │  │handoff.json │  │         │  │
facts/      │  │
│  │ SessionEnd  │ │          │  │routing_log  │  │         │  │
preferences/│  │
│  └─────────────┘ │          │  └────────────┘  │         │
│observations/│  │
│                  │          │                  │         │
└─────────────┘  │
│  Routing         │          │  Inter-agent     │         │
│
│  Enforcement     │          │  Communication   │         │
Persistent       │
│  Validation      │          │  Checkpoints     │         │
Learning         │
│  Logging         │          │  Ephemeral       │         │  Git
Versioned    │
│                  │          │                  │         │
│
└──────────────────┘          └──────────────────┘
└──────────────────┘
       │                                │
│

└──────────────────────────────┼──────────────────────────────┘
                                        │
                                        ▼
                                ┌──────────────────┐
                                │   EXTERNAL LAYER  │
                                │                   │
                                │  .gemini-slave/   │
                                │                   │
                                │  CLI Integration  │
                                │  Large Context    │
                                │  Parallel Workers │
                                │                   │
                                └──────────────────┘
```

### A.2 Request Lifecycle Flow

```
┌──────────────────────────────────────────────────────────────────────────────

│                              REQUEST LIFECYCLE
│
└──────────────────────────────────────────────────────────────────────────────


USER REQUEST
     │
     ▼
┌──────────────────────────────────────────────────────────────────────────────

│ PHASE 1: INTENT CLASSIFICATION
│
│
│
```

```
│ • Parse request
│
│ • Determine if explore workflow needed
│
│ • Identify task type and scope
│
└──────────────────────────────────────────────────────────────

    │
    ▼
┌──────────────────────────────────────────────────────────────
│ PHASE 2: SCOUT (Conditional)
│
│
│
│ • Haiku agent enumerates scope
│
│ • Count files, estimate tokens
│
│ • Identify module boundaries
│
│ • Output: scout_metrics.json
│
└──────────────────────────────────────────────────────────────

    │
    ▼
┌──────────────────────────────────────────────────────────────
│ PHASE 3: COMPLEXITY CALCULATION
│
│
│
│ SCORE = (tokens/10000) + (files/5) + (modules×2)
│
│
│
│ Output: complexity_score, recommended_tier
│
└──────────────────────────────────────────────────────────────

    │
    ▼
┌──────────────────────────────────────────────────────────────
│ PHASE 4: ROUTING ENFORCEMENT
│
│
│
│ PreToolUse.sh executes:
│
│ • Check force-tier override
│
│ • Validate against ceiling
│
│ • Log decision
│
│ • Exit 0 (PERMIT) or Exit 2 (BLOCK)
│
└──────────────────────────────────────────────────────────────

    │
    ├─── BLOCK ──► Return to user with explanation
    │
    ▼ PERMIT
┌──────────────────────────────────────────────────────────────
│ PHASE 5: EXECUTION
│
│
│
│ Selected agent executes task
│
│ • Each tool call triggers Pre/PostToolUse
│
│ • SubagentStop validates handoffs
│
│ • Results captured
│
└──────────────────────────────────────────────────────────────

    │
    ▼
┌──────────────────────────────────────────────────────────────
│ PHASE 6: MEMORY ARCHIVAL
│
│
│
│ • PostToolUse logs outcomes
│
│ • SessionEnd triggers archival
│
│ • Memory archivist processes decisions
│
│ • Sharp edges captured
```

```
│
└──────────────────────────────────────────────────────────────────────────────

     │
     ▼
OUTPUT TO USER
```

### A.3 Weekly Review Process

```
┌──────────────────────────────────────────────────────────────────────────────
│                              WEEKLY REVIEW PROCESS
│
└──────────────────────────────────────────────────────────────────────────────


TRIGGER: Monday 9 AM or manual /weekly-review

┌──────────────────────────────────────────────────────────────────────────────
│ PHASE 1: AGGREGATION
│
│
│
│ Memory Synthesis Agent (Sonnet) collects:
│
│ • decisions/*.md (last 7 days)
│
│ • sharp-edges/*.md (last 7 days)
│
│ • session_summaries/*.json
│
│ • routing_log_*.jsonl
│
│ • observations/*.jsonl
│
│
│
│ Output: weekly_synthesis.json
│
└──────────────────────────────────────────────────────────────────────────────

                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────
│ PHASE 2: GAP ANALYSIS
│
│
│
│ Systems Architect Agent (Opus, extended thinking) analyzes:
│
│ • Recurring failure patterns (≥3 occurrences)
│
│ • Tasks exceeding budgets
│
│ • Human override patterns
│
│ • Context boundary issues
│
│
│
│ For each gap: config change OR new subagent?
│
│
│
│ Output: architect_report.md, recommendations.json
│
└──────────────────────────────────────────────────────────────────────────────

                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────
│ PHASE 3: SCHEMA DISCOVERY (if observations ≥ 200)
│
│
│
│ Schema Discovery Agent analyzes observation patterns:
│
│ • Cluster detection (≥10 instances per cluster)
│
│ • Statistical validation (Silhouette > 0.5)
│
│ • Schema proposal generation
│
│
│
│ Output: pattern_analysis.json, proposed_schema (if patterns found)
│
└──────────────────────────────────────────────────────────────────────────────

                                       │
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────
```

```
│ PHASE 4: HUMAN INTERVIEW
│
│
│
│ Orchestrator presents findings:
│
│ • Gap analysis results
│
│ • Recommendations with [Approve] [Modify] [Reject]
│
│ • Schema proposals (if any)
│
│ • Autonomy level assessments
│
│
│
│ Human reviews and decides
│
└──────────────────────────────────────────────────────────────────────────────
                                        │
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────
│ PHASE 5: IMPLEMENTATION
│
│
│
│
│ Approved changes applied:
│
│ • Config updates to routing-schema.json
│
│ • New agents created in .claude/agents/
│
│ • Schemas activated (if approved)
│
│ • Autonomy levels adjusted
│
│
│
│ All changes logged in memory/decisions/
│
└──────────────────────────────────────────────────────────────────────────────
```

### A.4 Autonomy Progression Model

```
┌──────────────────────────────────────────────────────────────────────────────
│                           AUTONOMY LEVEL PROGRESSION
│
└──────────────────────────────────────────────────────────────────────────────


                    INCREASING AUTOMATION
                    ──────────────────────►

┌──────────────┐   ┌──────────────┐   ┌──────────────┐
┌──────────────┐   ┌──────────────┐
│   LEVEL 1    │   │   LEVEL 2    │   │   LEVEL 3    │   │   LEVEL 4
│   │   LEVEL 5   │
│              │   │              │   │              │   │
│   │          │   │
│  OPERATOR    │──►│COLLABORATOR  │──►│ CONSULTANT   │──►│  APPROVER
│──►│  OBSERVER   │
│              │   │              │   │              │   │
│   │          │   │
│ Human does   │   │ System       │   │ System auto-│   │ System
│   │ System runs │
│ everything   │   │ suggests,    │   │ approves     │   │ handles
│   │ autonomously│
│              │   │ human        │   │ high-conf    │   │ routine,
│   │ in defined  │
│ System       │   │ confirms     │   │ decisions    │   │ human
│   │ domain      │
│ observes     │   │              │   │              │   │ reviews
│   │          │   │
│              │   │              │   │              │   │ summary
│   │ Human       │
│ 0% auto      │   │ 0% auto      │   │ ~60% auto    │   │ ~85% auto
│   │ ~95% auto   │
└──────────────┘   └──────────────┘   └──────────────┘
└──────────────┘   └──────────────┘

      │                 │                 │                 │
│
      │                 │                 │                 │
│
      ▼                 ▼                 ▼                 ▼
▼
   100 decisions    200 decisions    500 decisions    1000+
decisions   Domain-specific
   captured         95% success      98% success      99%+ success
Human approves
```

# Appendix B: Threshold Quick Reference

## B.1 Routing Thresholds

| Parameter | Value | Formula/Rule |
|---|---|---|
| Complexity score | `(tokens/10000) + (files/5) + (modules×2)` | |
| Haiku ceiling | Score < 2 | |
| Sonnet ceiling | Score 2-10 | |
| Opus floor | Score > 10 | |
| Gemini trigger | Tokens > 50,000 | |
| Gemini force | Tokens > 100,000 | |

## B.2 Cost Thresholds

| Parameter | Value | Action |
|---|---|---|
| Session warning | $5 | Alert user |
| Session limit | $10 | Require confirmation |
| Weekly budget | $50 | Review trigger |
| Weekly review budget | $2.50 | Infrastructure cost |
| Shadow cost ceiling | 2× expected | Auto-terminate |

## B.3 Pattern Detection Thresholds

| Parameter | Value | Purpose |
|---|---|---|
| Min observations | 200 | Trigger analysis |
| Per-cluster minimum | 30 | Statistical power |
| Silhouette score | > 0.5 | Cluster validity |
| Bootstrap stability | 80% | Reproducibility |
| Confidence level | 99% (α=0.01) | Decision threshold |

## B.4 Autonomy Thresholds

| Transition | Decisions | Success Rate |
|---|---|---|
| L1 → L2 | 100 | N/A |
| L2 → L3 | 200 | 95% |
| L3 → L4 | 500 | 98% |
| L4 → L5 | 1000+ | 99%+ |

## B.5 Shadow Deployment Thresholds

| Parameter | Value |
|---|---|
| Min invocations | 10 |
| Success rate | ≥ 90% |
| Error rate vs baseline | ≤ 0.1% increase |
| Max duration | 14 days |

## B.6 Document Processing

| Parameter | Value |
|---|---|
| Chunk size (default) | 512 tokens |
| Chunk overlap | 15% |
| Max parallel workers | 5 |
| Overlap window size | 40,000 tokens |

---

# Appendix C: Glossary

| Term | Definition |
|---|---|
| **Agent** | An LLM configured with specific instructions and constraints for a task type |
| **Apprenticeship Learning** | Learning to replicate human decision-making through observation |
| **BM25** | Best Matching 25, a probabilistic ranking function for text retrieval |
| **Complexity Score** | Numeric value calculated from tokens, files, and modules to determine routing |
| **Emergent Schema** | A data schema discovered from accumulated observations rather than pre-defined |
| **Explore Workflow** | 7-phase structured process for investigating and modifying codebases |
| **Gap Analysis** | Systematic identification of capability deficiencies based on observed failures |
| **Handoff** | Transfer of task context and state between agents |
| | Bash script executed at specific lifecycle |

| | |
|---|---|
| **Hook** | points (PreToolUse, PostToolUse, etc.) |
| **HITL** | Human-in-the-Loop; pattern where humans approve or reject agent proposals |
| **Memory Archivist** | Agent responsible for processing session outcomes into persistent memory |
| **Observation** | Raw behavioral event captured for pattern analysis |
| **Routing** | Decision process for selecting which agent tier handles a task |
| **Scout** | Haiku agent performing reconnaissance on codebase before main task |
| **Shadow Deployment** | Running new agent in parallel without affecting production, for validation |
| **Sharp Edge** | Known pitfall or unexpected behavior documented for future avoidance |
| **Subagent Spawning** | Automatic generation of new agent definitions from gap analysis |
| **Tier** | Model capability level (Haiku < Sonnet < Opus) |
| **Weekly Review** | Automated process for analyzing system behavior and proposing improvements |

---

# Appendix D: Research Source Index

## Primary Research Documents

| ID | Title | Key Findings Used |
|---|---|---|
| R1 | Hybrid Multi-Agent Architecture Evaluation | MAST taxonomy (41-86.7% failure rate), RouteLLM (85% cost reduction), BM25 recall benchmarks |
| R2 | Evidence-Based Thresholds for Multi-Agent Design | Pattern detection thresholds (30×k formula), autonomy progression criteria, shadow deployment metrics |
| R3 | LangChain/LangGraph vs Bespoke RAG Analysis | Framework overhead (14ms latency, 50% token increase), selective adoption recommendations |

## External Sources Referenced

| Topic | Source | Finding |
|---|---|---|
| BM25 vs Embeddings | XetHub Benchmark 2024 | 89% vs 91.7% recall |
| Chunk Overlap | NVIDIA FinanceBench 2024 | 15% optimal |
| Multi-Agent Coordination | MAST Taxonomy | 35% failures from inter-agent misalignment |
| Routing Efficiency | RouteLLM Paper | 85% cost reduction, 95% performance retention |
| Context Compression | ACON Framework | 26-54% reduction, 95%+ accuracy |
| Autonomy Levels | Sheridan & Verplank | 5-level automation taxonomy |

---

# Appendix E: Verification Commands

## E.1 Installation Verification

```
# Verify Claude Code installation
claude --version

# Verify hooks directory exists
ls -la ~/.claude/hooks/

# Verify hook executability
[[ -x ~/.claude/hooks/PreToolUse.sh ]] && echo "PreToolUse OK" ||
echo "PreToolUse NOT EXECUTABLE"

# Verify required tools
command -v jq >/dev/null && echo "jq OK" || echo "jq MISSING"
command -v bc >/dev/null && echo "bc OK" || echo "bc MISSING"

# Verify telemetry enabled
env | grep CLAUDE_CODE_ENABLE_TELEMETRY
```

## E.2 Component Health Checks

```
# Test complexity calculation
echo '{"scout_report":{"scope_metrics":
```

```
{"total_files":10,"estimated_tokens":25000},"complexity_signals":
{"module_count":2}}}' > ~/.claude/tmp/scout_metrics.json
        ~/.claude/scripts/calculate-complexity.sh
        cat ~/.claude/tmp/complexity_score  # Should output ~7.5

        # Test SubagentStop validation
        echo '{"status":"completed","context":
{"task_summary":"test","success_criteria":["done"]}}' >
~/.claude/tmp/handoff.json
        ~/.claude/hooks/SubagentStop.sh test_agent
        echo "Exit code: $?"  # Should be 0

        # Test memory query
        python3 ~/.claude/scripts/query-memory-bm25.py "test query" --json
```

### E.3 State File Validation

```
        # Validate all state files
        python3 ~/.claude/scripts/validate-state.py

        # Check routing log integrity
        jq empty ~/.claude/tmp/routing_log.jsonl 2>/dev/null && echo
"Routing log valid" || echo "Routing log invalid"

        # Verify memory file format
        for f in ~/.claude/memory/**/*.md; do
            if head -1 "$f" | grep -q "^---$"; then
                echo "✓ $f has frontmatter"
            else
                echo "✗ $f missing frontmatter"
            fi
        done
```

### E.4 Integration Tests

```
        # Full hook chain test
        ~/.claude/scripts/calculate-complexity.sh
        ~/.claude/hooks/PreToolUse.sh
        echo "PreToolUse exit: $?"
        ~/.claude/hooks/PostToolUse.sh
        echo "PostToolUse exit: $?"

        # Memory retrieval test
        python3 ~/.claude/scripts/query-memory-bm25.py "authentication" -k 3

        # Cost report generation
        ~/.claude/scripts/generate-cost-report.sh
```

### E.5 Diagnostic Commands

```
        # View recent routing decisions
        tail -20 ~/.claude/tmp/routing_log.jsonl | jq .

        # Summarize session costs
        jq -s 'map(.cost.estimated_cost_usd // 0) | add'
~/.claude/tmp/routing_log.jsonl

        # Count memory files
        find ~/.claude/memory -name "*.md" | wc -l

        # Check agent registry
        cat ~/.claude/agents-index.json | jq '.[] | {name, status}'

        # View autonomy levels
        cat ~/.claude/autonomy-levels.yaml
```

---

## Appendix F: File Templates

### F.1 Memory File Template

```
        ---
        title: "[Brief descriptive title]"
        created: YYYY-MM-DD
        category: decisions  # or sharp-edges, facts, preferences
        tags: [tag1, tag2, tag3]
        status: active
        summary: "One-line searchable summary"
        ---

        ## Context

        [Why this was created]

        ## Content

        [Main content]

        ## Related

        - [[related-file-1.md]]
        - [[related-file-2.md]]
```

### F.2 Agent Definition Template

```yaml
# agent.yaml
name: agent-name
version: 1.0.0
status: proposed
tier: sonnet
purpose: |
  Brief description of what this agent does.

triggers:
  - condition: "trigger condition"

configuration:
  key: value

constraints:
  max_input_tokens: 50000
  max_output_tokens: 10000
  timeout_minutes: 5
```

### F.3 Decision Log Template

```yaml
---
title: "Decision: [Topic]"
created: YYYY-MM-DD
category: decisions
tags: [architecture, topic]
status: active
summary: "One-line summary"
---

## Context
Why this decision was needed.

## Options Considered
1. Option A
2. Option B

## Decision
What was chosen.

## Rationale
Why this choice.

## Consequences
What changes.
```

---

## Document Version History

| Version | Date | Changes |
|---|---|---|
| 1.0.0 | 2026-01-13 | Initial release |

---

*End of Lisan al-Gaib Architecture & Implementation Guide*